# Physical AI 로봇 제어 실무

Isaac Sim, ROS 2, SO-101, LeRobot와 SmolVLA로 완성하는 시뮬레이션-현실 로봇 제어

Ubuntu 24.04 LTS · ROS 2 Jazzy Jalisco · Python 3 · C++17

- 머리말
- 이 책의 사용법
    - 학습 경로
    - 코드 표기
    - 명령 실행 위치
    - 실물 장비와 시뮬레이션
    - 예상 출력
- 기준 실습 환경
    - 운영체제와 미들웨어
    - 집필 기준 버전과 검증 상태
    - 실습 장비
- 안전 수칙
- 제1장 Physical AI와 Isaac Sim·OpenUSD·ROS 2 기반
    - 장 개요
    - 학습 목표
    - 선수 지식과 준비물
    - 1.1 Physical AI의 문제 정의
        - 1.1.1 정보 처리에서 물리적 상호작용으로
        - 1.1.2 규칙 기반 제어와 학습 기반 정책
        - 1.1.3 인지-정책-행동 루프
        - 1.1.4 책임의 분리
    - 1.2 NVIDIA Isaac Sim 런타임 이해
        - 1.2.1 구성 요소의 역할
        - 1.2.2 Stage와 시뮬레이션 상태를 구분하라
        - 1.2.3 확장 기반 실행과 독립 실행
    - 1.3 OpenUSD 데이터 모델
        - 1.3.1 Stage, Layer, Prim
        - 1.3.2 Attribute와 Relationship
        - 1.3.3 변환과 XformOp
        - 1.3.4 비파괴 합성
        - 1.3.5 예제 장면 생성
    - 1.4 ROS 2 Jazzy 통신 모델
        - 1.4.1 ROS 그래프와 Node
        - 1.4.2 Topic, Service, Action 선택
        - 1.4.3 DDS와 RMW
        - 1.4.4 QoS는 인터페이스 계약이다
        - 1.4.5 시간의 두 종류
    - 1.5 Isaac Sim-ROS 2 브리지 설계
        - 1.5.1 경계에서 명시해야 할 네 가지
        - 1.5.2 권장 인터페이스
        - 1.5.3 단계별 통합 전략
    - 1.6 구현 예제 안내
        - 1.6.1 순수 Python 폐루프
        - 1.6.2 ROS 2 Python 상태 발행기
        - 1.6.3 ROS 2 C++ 안전 게이트

    - 1.7 단계별 실습
    - 1.8 트러블슈팅
    - 1.9 요약
    - 1.10 확인문제
    - 1.11 프로젝트 과제
        - 과제 A: Physical AI 인터페이스 설계서
        - 과제 B: 비파괴 USD 작업 셀
        - 평가 기준
    - 1.12 실습·평가·참고자료
        - 1.12.1 참고문헌
        - [SIM] 실습 1-1: Physical AI 지연 예산
        - [SIM] 실습 1-2: OpenUSD 장면과 비파괴 Override
        - [SIM] 실습 1-3: ROS 2 인터페이스와 QoS
        - [SIM→HW] [SAFETY] 실습 1-4: 시뮬레이터-ROS 2 브리지 계약
        - 제1장 확인문제 해설 및 평가 기준
- 제2장 Isaac Sim 환경 구축과 PhysX 강체 물리 시뮬레이션
    - 장 개요
    - 학습 목표
    - 선수 지식과 준비물
        - 선수 지식
        - 기준 환경
        - 안전 메모
    - 2.1 Isaac Sim 작업 환경 이해하기
        - 2.1.1 Stage, simulation, rendering은 같은 것이 아니다
        - 2.1.2 GUI, Script Editor, Standalone Python
        - 2.1.3 재현 가능한 Stage 규칙
    - 2.2 좌표계와 4×4 동차 변환
        - 2.2.1 pose의 의미부터 고정한다
        - 2.2.2 active rotation과 passive rotation
        - 2.2.3 Euler angle의 convention
        - 2.2.4 quaternion과 순서 문제
    - 2.3 강체 역학과 시뮬레이션 시간
        - 2.3.1 rigid, static, kinematic
        - 2.3.2 이산 시간 적분
        - 2.3.3 solver iteration과 안정성
    - 2.4 충돌 형상 설계
        - 2.4.1 visual mesh와 collision mesh
        - 2.4.2 collision approximation 선택
        - 2.4.3 convex decomposition 튜닝
        - 2.4.4 broad phase와 narrow phase
    - 2.5 Tunneling과 CCD
        - 2.5.1 왜 얇은 물체를 통과하는가
        - 2.5.2 CCD 적용 전략
    - 2.6 물리 재질과 접촉 역학
        - 2.6.1 Coulomb friction의 단순 모델
        - 2.6.2 기울기 실험으로 정마찰 추정

        - 2.6.3 반발계수
        - 2.6.4 physics material과 render material
    - 2.7 질량, 무게중심, 관성 텐서
        - 2.7.1 질량만 맞추면 충분하지 않다
        - 2.7.2 직육면체 관성
        - 2.7.3 density와 mass authoring
    - 2.8 PBR 재질과 조명
        - 2.8.1 물리 기반 렌더링의 목적
        - 2.8.2 핵심 파라미터
        - 2.8.3 Dome, distant, area light
    - 2.9 물리 파라미터 튜닝 절차
        - 2.9.1 관찰량부터 정의한다
        - 2.9.2 한 번에 하나씩 바꾼다
        - 2.9.3 권장 체크리스트
    - 2.10 단계별 통합 실습
        - 실습 A: 변환 체인 검증
        - 실습 B: 충돌 형상과 CCD 비교
        - 실습 C: 마찰과 반발 보정
    - 2.11 예상 출력 예시
    - 2.12 트러블슈팅
        - 물체가 바닥을 통과한다
        - 물체가 폭발하듯 튄다
        - 그리퍼 안의 물체가 계속 미끄러진다
        - 동일한 설정인데 실행마다 결과가 다르다
        - 색은 맞는데 합성 영상이 실물과 다르다
    - 2.13 요약
    - 2.14 확인문제
    - 2.15 프로젝트 과제
        - 과제 1: 작업 셀 디지털 트윈
        - 과제 2: CCD 비용-효과 분석
        - 과제 3: 실측 기반 material calibration
    - 2.16 실습·평가·참고자료
        - 2.16.1 참고문헌
        - 실습 2-A: 좌표 변환 체인과 쿼터니언
        - 실습 2-B: 충돌 근사와 CCD 비용-효과 분석
        - 실습 2-C: 마찰·반발·질량 파라미터 보정
        - 제2장 연습문제 및 해설 지침
- 제3장 SO-101 기구학과 ROS 2 기반 하드웨어 텔레오퍼레이션
    - 장 개요
    - 학습 목표
    - 선수 지식과 준비물
        - 선수 지식
        - 기준 환경
        - 실물 실습 전 안전 원칙
    - 3.1 SO-101 기구학 모델
        - 3.1.1 링크, 관절, 구동기의 구분

        - 3.1.2 URDF에서 읽어야 할 정보
        - 3.1.3 URDF에서 USD Articulation으로
    - 3.2 순기구학
        - 3.2.1 동차변환 체인
        - 3.2.2 표준 DH 표현
        - 3.2.3 기하 자코비안
    - 3.3 DLS 역기구학
        - 3.3.1 선형화와 감쇠
        - 3.3.2 반복 알고리즘
        - 3.3.3 5-DoF 과업 설계
    - 3.4 Leader-Follower 캘리브레이션
        - 3.4.1 데이터 경로
        - 3.4.2 LeRobot 캘리브레이션 절차
        - 3.4.3 그리퍼 표현의 함정
        - 3.4.4 첫 프레임 정렬
    - 3.5 안전한 텔레오퍼레이션 루프
        - 3.5.1 안전 계층
        - 3.5.2 주기와 지연
    - 3.6 ROS 2 JointState 브리지
        - 3.6.1 토픽 계약
        - 3.6.2 QoS 선택
        - 3.6.3 robot_state_publisher와 TF
        - 3.6.4 Isaac Sim 미러링
    - 3.7 단계별 실습
        - [SIM] 실습 1: FK와 DLS IK
        - [SIM] [SAFETY] 실습 2: 안전 필터 dry-run
        - [SIM→HW] [SAFETY] 실습 3: ROS 2 브리지
    - 3.8 트러블슈팅
    - 3.9 요약
    - 3.10 확인문제
    - 3.11 프로젝트 과제
        - 과제: 안전한 SO-101 Digital Twin Bridge
    - 3.12 실습·평가·참고자료
        - 3.12.1 참고문헌
        - [SIM] 실습 3-1: FK와 DLS IK
        - [SIM] [SAFETY] 실습 3-2: Leader-Follower 안전 필터
        - [SIM→HW] [SAFETY] 실습 3-3: ROS 2 JointState 안전 브리지
        - 프로젝트 평가 기준: 안전한 SO-101 Digital Twin Bridge
        - 제3장 확인문제 해설
- 제4장 멀티모달 센서 정합과 데이터 파이프라인
    - 장 개요
    - 학습 목표
    - 선수 지식과 준비물
    - 4.1 핀홀 카메라와 좌표 프레임
        - 4.1.1 투영 모델
        - 4.1.2 렌즈 왜곡

        - 4.1.3 역투영과 깊이의 의미
        - 4.1.4 캘리브레이션 품질
    - 4.2 Isaac Sim/USD, ROS 2, OpenCV 좌표 변환
        - 4.2.1 축 이름을 추측하지 않는다
        - 4.2.2 동차 변환의 방향
        - 4.2.3 쿼터니언과 행렬
        - 4.2.4 프레임 계약 표
    - 4.3 TF2 트리 설계
        - 4.3.1 트리는 하나의 부모를 갖는다
        - 4.3.2 timestamp가 있는 조회
        - 4.3.3 TF 진단
    - 4.4 멀티레이트 센서 시간 정합
        - 4.4.1 네 개의 시간
        - 4.4.2 ExactTime과 ApproximateTime
        - 4.4.3 선형 보간
        - 4.4.4 품질 지표
    - 4.5 rosbag2와 MCAP 원본 기록
        - 4.5.1 왜 원본 메시지를 보존하는가
        - 4.5.2 에피소드 메타데이터
    - 4.6 Domain Randomization
        - 4.6.1 목적과 한계
        - 4.6.2 분포를 버전 관리한다
        - 4.6.3 Replicator 실행 구조
    - 4.7 Rerun 기반 무결성 검사
    - 4.8 구현 예제
        - [SIM] 4.8.1 카메라 수학
        - [SIM] 4.8.2 ApproximateTime ROS 2 Node
        - [SIM] 4.8.3 독립 C++ 보간
    - 4.9 트러블슈팅
    - 4.10 요약
    - 4.11 확인문제
    - 4.12 프로젝트 과제
        - 과제 A: 멀티모달 에피소드 정합기
        - 과제 B: 데이터 품질 리포트
        - 평가 기준
    - 4.13 실습·평가·참고자료
        - 4.13.1 참고문헌
        - [SIM] 실습 4-1: 카메라 투영과 프레임 정합
        - [SIM] 실습 4-2: TF2와 ApproximateTime
        - [SIM] 실습 4-3: rosbag2/MCAP 에피소드 기록
        - [SIM] 실습 4-4: Domain Randomization과 Rerun 품질 검토
        - 제4장 확인문제 해설 및 평가 기준
- 제5장 LeRobot Dataset과 SmolVLA 파인튜닝
    - 장 개요
    - 학습 목표
    - 선수 지식과 준비물

        - 선수 지식
        - 기준 환경
    - 5.1 모방학습과 행동 복제
        - 5.1.1 demonstration dataset
        - 5.1.2 behavior cloning 목적함수
        - 5.1.3 covariate shift와 복구 데이터
    - 5.2 행동 정책의 계보
        - 5.2.1 단일 step 정책
        - 5.2.2 ACT: action sequence를 한 번에 예측
        - 5.2.3 Diffusion Policy
        - 5.2.4 VLA
    - 5.3 LeRobotDataset v3 설계
        - 5.3.1 세 개의 기둥
        - 5.3.2 feature schema
        - 5.3.3 episode 경계와 split
        - 5.3.4 finalize의 의미
    - 5.4 데이터 수집과 품질 관리
        - 5.4.1 다양성과 반복의 균형
        - 5.4.2 성공/실패 label
        - 5.4.3 시간 정렬
        - 5.4.4 영상 인코딩
    - 5.5 정규화와 통계
        - 5.5.1 Z-score와 min-max
        - 5.5.2 작은 분산과 outlier
        - 5.5.3 action representation
        - 5.5.4 Dataset 전역 통계와 정책 학습 통계
    - 5.6 SmolVLA 구조
        - 5.6.1 입력 context
        - 5.6.2 action expert와 flow matching
        - 5.6.3 chunk size
    - 5.7 LoRA 기반 파인튜닝
        - 5.7.1 저랭크 업데이트
        - 5.7.2 rank와 alpha
        - 5.7.3 full fine-tuning과 LoRA 선택
    - 5.8 학습 파이프라인
        - 5.8.1 데이터 사전 점검
        - 5.8.2 기본 fine-tuning 예
        - 5.8.3 LoRA 예
        - 5.8.4 monitoring
    - 5.9 20 Hz 폐루프 추론
        - 5.9.1 주기와 deadline
        - 5.9.2 receding horizon
        - 5.9.3 temporal ensembling
        - 5.9.4 safety filter와 watchdog
        - 5.9.5 최신 LeRobot 배포 backend
    - 5.10 평가 설계

        - 5.10.1 task success
        - 5.10.2 stratified evaluation
        - 5.10.3 offline metric의 한계
    - 5.11 데이터 라이선스, 개인정보, 보안
        - 5.11.1 dataset card와 license
        - 5.11.2 개인정보
        - 5.11.3 token과 Hub 권한
    - 5.12 재현성 체크리스트
    - 5.13 단계별 실습
        - 실습 A: v3 dataset 구조와 무결성 감사
        - 실습 B: normalization과 split 누출 실험
        - 실습 C: SmolVLA LoRA 학습 계획
        - 실습 D: 20 Hz action chunk 실행
    - 5.14 예상 출력 예시
    - 5.15 트러블슈팅
        - dataset이 load되지 않는다
        - 영상과 관절이 어긋난다
        - loss는 감소하지만 로봇이 거의 움직이지 않는다
        - 동작이 떨리거나 chunk 경계에서 튄다
        - GPU memory가 부족하다
        - inference가 20 Hz를 만족하지 못한다
    - 5.16 요약
    - 5.17 확인문제
    - 5.18 프로젝트 과제
        - 과제 1: Dataset v3 품질 보고서
        - 과제 2: ACT 또는 SmolVLA baseline 비교
        - 과제 3: LoRA ablation
        - 과제 4: 20 Hz deployment harness
    - 5.19 실습·평가·참고자료
        - 5.19.1 참고문헌
        - 실습 5-A: LeRobotDataset v3 무결성 감사
        - 실습 5-B: Episode split과 정규화 누출
        - 실습 5-C: SmolVLA LoRA 학습 설계
        - 실습 5-D: 20 Hz Action Chunk 폐루프
        - 제5장 연습문제 및 구현 과제
- 제6장 Sim2Real 배포 최적화와 최종 실증
    - 장 개요
    - 학습 목표
    - 선수 지식과 준비물
    - 6.1 Reality Gap의 구조
        - 6.1.1 오차 분류표
        - 6.1.2 도메인 랜덤화의 역할과 한계
    - 6.2 정량 진단
        - 6.2.1 지연과 jitter
        - 6.2.2 백래시와 deadband
        - 6.2.3 마찰과 그리퍼

        - 6.2.4 카메라 외부 파라미터
    - 6.3 출력 안정화
        - 6.3.1 EMA 저역통과 필터
        - 6.3.2 Temporal Ensembling
    - 6.4 안전 배포 감독기
        - 6.4.1 승격 상태
        - 6.4.2 작업공간 Bounding Box
        - 6.4.3 워치독과 생명주기
    - 6.5 End-to-End Pick & Place 배포
        - 6.5.1 배포 전 고정해야 할 계약
        - 6.5.2 권장 실행 순서
        - 6.5.3 음성/텍스트 명령 경계
    - 6.6 평가 설계
        - 6.6.1 성공 정의
        - 6.6.2 성공률과 Wilson 신뢰구간
        - 6.6.3 층화 실험
    - 6.7 트러블슈팅 매트릭스
    - 6.8 포트폴리오 패키징
        - 필수 구조
        - 보고할 결과
    - 6.9 단계별 실습
        - 실습 1: 로그 진단과 신뢰구간
        - 실습 2: 필터와 안전 승격
        - 실습 3: ROS 2 Workspace Guard
    - 6.10 요약
    - 6.11 확인문제
    - 6.12 최종 프로젝트
        - 안전한 SO-101 Pick & Place 실증
    - 6.13 실습·평가·참고자료
        - 6.13.1 참고문헌
        - 실습 6-1: Sim2Real 로그 진단
        - 실습 6-2: Temporal Filter와 안전 승격 감독기
        - 실습 6-3: ROS 2 Cartesian Workspace Guard
        - 최종 프로젝트 평가 기준
        - 제6장 확인문제 해설
- 부록 A. ROS 2 Lyrical 마이그레이션 노트
    - A.1 왜 본문은 Jazzy를 사용하는가
    - A.2 이식 점검표
    - A.3 Lyrical에서 주목할 기능
    - 참고문헌
- 부록 B. 용어집

# 머리말

Physical AI는 인공지능 모델이 센서와 액추에이터를 통해 물리 세계를 관찰하고, 판단하고, 다시 세계에 영향을 주는 시스템을 다룬다. 언어 모델이나 이미지 분류기처럼 입력과 출력이 디지털 경계 안에 머무르는 시스템과 달리, 로봇의 잘못된 출력은 충돌, 장비 손상, 데이터 오염으로 이어질 수 있다. 따라서 Physical AI 엔지니어에게는 모델 성능뿐 아니라 좌표계, 시간, 물리 파라미터, 통신 지연, 안전 경계까지 하나의 시스템으로 이해하는 능력이 필요하다.

이 책은 컴퓨터공학 전공 졸업자 또는 졸업 예정자가 시뮬레이션에서 시작해 실제 로봇팔의 자율 조작까지 이어지는 전체 파이프라인을 경험하도록 설계했다. 각 장은 개념을 소개하는 데 그치지 않고, 왜 그 문제가 발생하는지, 어떤 설계 선택지가 있는지, 구현 결과를 무엇으로 판단해야 하는지를 함께 설명한다.

책의 기준 환경은 Ubuntu 24.04 LTS와 ROS 2 Jazzy Jalisco다. Jazzy는 Ubuntu 24.04를 Tier 1으로 지원한다(Jazzy 지원 플랫폼, REP 2000). 2026년에 공개된 ROS 2 Lyrical Luth는 장기지원 배포판이지만, 도구·드라이버·시뮬레이터 생태계의 성숙도와 현업 전환 비용을 고려해 Jazzy를 주 환경으로 삼은 것은 이 책의 편집 판단이다. 새 배포판의 주요 차이는 부록의 마이그레이션 노트에서 다룬다.

명령 실행 결과와 학습 로그는 이해를 돕기 위한 **예상 출력 예시**다. 하드웨어, 드라이버, GPU, 라이브러리 버전에 따라 실제 결과는 달라질 수 있다.

# 이 책의 사용법

## 학습 경로

책은 여섯 장이 하나의 프로젝트로 이어지도록 구성되어 있다.

6개 장의 학습 경로: 시스템 기초에서 Sim2Real 배포까지

1. Physical AI 시스템의 전체 구조와 ROS 2·OpenUSD 기초를 익힌다.
2. Isaac Sim에서 물리적으로 일관된 작업 환경을 만든다.
3. SO-101의 기구학과 Leader-Follower 제어를 구현한다.
4. 카메라와 관절 데이터를 동일한 좌표·시간 기준으로 정합한다.
5. 시연 데이터를 LeRobot 형식으로 구성하고 SmolVLA를 학습한다.
6. 학습 정책을 안전한 폐루프 제어기로 배포하고 Sim2Real 성능을 평가한다.

## 코드 표기

- `$`로 시작하는 줄은 일반 사용자 셸 명령이다.
- `#`로 시작하는 셸 명령은 관리자 권한이 필요한 예시일 수 있다.
- Python 코드는 Python 3.12와 타입 힌트를 기준으로 한다.
- ROS 2 C++ 코드는 C++17과 `rclcpp`를 기준으로 한다.
- 코드 블록 위에는 가능한 경우 대응 파일 경로를 적는다.

## 명령 실행 위치

장별 명령은 별도 안내가 없으면 **해당 장의 루트 폴더**에서 실행한다. 예를 들어 제4장 명령의 기준 위치는 다음과 같다.

```
cd book/04_sensor_data_pipeline
```

각 실습 파일도 첫 명령에서 장 루트로 이동한 뒤 `code/...` 경로를 사용한다. EPUB/PDF만 읽는 경우에는 배포된 예제 저장소의 루트에서 같은 경로를 적용한다.

## 실물 장비와 시뮬레이션

각 실습은 다음 표식을 사용한다.

- **[SIM]** 실물 장비 없이 수행 가능한 시뮬레이션 실습
- **[HW]** SO-101 또는 동등한 로봇팔이 필요한 실습
- **[SIM→HW]** 시뮬레이션에서 검증한 뒤 실물로 이식하는 실습
- **[SAFETY]** 사람과 장비 보호를 위해 반드시 확인할 항목

여러 표식이 함께 붙을 수 있다. `[HW]` `[SAFETY]` 실습은 안전 수칙을 확인하고 물리적 전원 차단 수단을 준비하기 전에는 시작하지 않는다.

## 예상 출력

책에 수록된 터미널 출력, 그래프 값, 학습 손실, 성공률은 실행 환경을 설명하기 위한 예시다. 숫자 자체보다 출력의 형태와 판단 기준을 학습하는 데 사용한다.

# 기준 실습 환경

## 운영체제와 미들웨어

| 항목 | 기준 |
|---|---|
| 운영체제 | Ubuntu 24.04 LTS 64-bit |
| ROS 2 | Jazzy Jalisco |
| Python | 3.12 계열 |
| C++ | C++17 |
| 빌드 도구 | colcon, CMake, ament |
| GPU | NVIDIA RTX 계열, Isaac Sim 요구 사양 충족 권장 |
| 컨테이너 | Docker/Podman은 보조 경로로 사용 |

ROS 2 공식 플랫폼 정책에서 Jazzy의 Ubuntu 24.04 `amd64`와 `arm64`는 Tier 1이다. Tier 1은 지속 통합과 패키지 배포가 가장 강하게 보장되는 조합을 뜻한다. 근거는 ROS 2 Jazzy 지원 플랫폼과 REP 2000이다.

## 집필 기준 버전과 검증 상태

아래 표는 2026-10-28에 고정한 집필 기준이다. 문서  확인은 공식 문서와 API를 대조했다는 뜻이고, 구문  확인은 현재 작업 환경에서 Python/XML 또는 독립 C++ 예제까지 확인했다는 뜻이다. GPU·로봇 장비가 필요한 항목은 **예상 실행**이며 실제 장비 검증을 주장하지 않는다.

| 구성 요소 | 집필 기준 | 상태 | 적용 원칙 |
|---|---|---|---|
| Ubuntu | 24.04 LTS | 기준 환경 | 책 전체의 기본 OS |
| ROS 2 | Jazzy Jalisco | 문서 확인·일부 구문 확인 | `/opt/ros/jazzy`와 Jazzy 메시지/API 사용 |
| Python | 3.12 계열 | 구문 확인 | 범용 예제 기준, Isaac Sim 내장 Python은 별도 |
| C++ | C++17 | 독립 예제 컴파일 확인 | ROS 2 예제는 Jazzy 빌드 환경 필요 |
| NVIDIA Isaac Sim | 6.2.0 문서 기준 | 문서 확인·예상 실행 | 설치 릴리스에서 API 경로 재확인 |
| LeRobot | 0.6.1 | 문서 확인·예상 실행 | SO-101 CLI 변경 가능성 확인 |
| Rerun SDK | 0.40.0 | 구문 확인·예상 실행 | 시각화 예제 API 기준 |
| OpenUSD `usd-core` | 26.11 | 범용 예제 구문 확인·예상 실행 | Isaac Sim 번들 USD와 혼용 금지 |

버전 확인 출처: Isaac Sim 6.2 release notes, LeRobot PyPI, Rerun SDK PyPI, OpenUSD usd-core PyPI.

패치 버전이 달라져도 개념과 데이터 계약은 유지한다. 명령이나 import가 달라지면 설치 버전의 release note를 먼저 확인하고, 실습 보고서에 실제 버전을 기록한다.

## 실습 장비

- SO-101 Leader/Follower 한 쌍: 5-DoF 팔 + 그리퍼, 총 6개 구동기
- 대체 장비는 동일한 관절 구성을 권장한다. 6-DoF 팔을 사용하면 제3장의 IK 과업 우선순위가 달라진다.
- 광각 RGB 카메라 2대(상부 및 손목 시점)
- 안정화된 전원 공급 장치와 물리적 전원 차단 수단
- 로봇 고정 마운트, 작업대, 파지 대상체와 트레이
- 최소 60 GB 이상의 여유 저장 공간

실물 장비가 없으면 제1·2장의 Isaac Sim 환경과 제3장의 가상 조인트 인터페이스를 이용해 제4장까지 학습할 수 있다. 제5장의 데이터셋 실습은 제공 예제 또는 공개 데이터셋으로 대체할 수 있다.

# 안전 수칙

로봇팔은 크기가 작더라도 끼임, 충돌, 케이블 감김, 과열 위험이 있다. 아래 수칙은 모든 실물 실습보다 우선한다.

1. 전원을 넣기 전에 로봇을 작업대에 단단히 고정한다.
2. 초기 시험은 낮은 속도와 낮은 토크 제한에서 수행한다.
3. 관절 제한과 작업 공간 경계를 소프트웨어와 하드웨어 양쪽에서 확인한다.
4. 통신 단절이나 명령 타임아웃이 발생하면 마지막 명령을 유지하지 말고 안전 정지 상태로 전환한다.
5. 비상 정지 또는 전원 차단 장치를 작업자의 손이 즉시 닿는 곳에 둔다.
6. 자동 동작 중에는 로봇의 가동 범위 안에 손이나 얼굴을 넣지 않는다.
7. 시뮬레이션에서 검증하지 않은 정책을 실물 로봇에서 곧바로 전체 속도로 실행하지 않는다.
8. 학습 모델 출력은 안전 제약을 우회할 권한을 가져서는 안 된다.

이 책의 예제는 교육 목적이며 특정 장비의 안전 인증을 대체하지 않는다. 실제 장비의 제조사 매뉴얼과 현장 안전 규정을 함께 따라야 한다.

제1장

# Physical AI와 Isaac Sim·OpenUSD·ROS 2 기반

실물 로봇팔과 디지털 트윈이 서로 손을 흔드는 모습

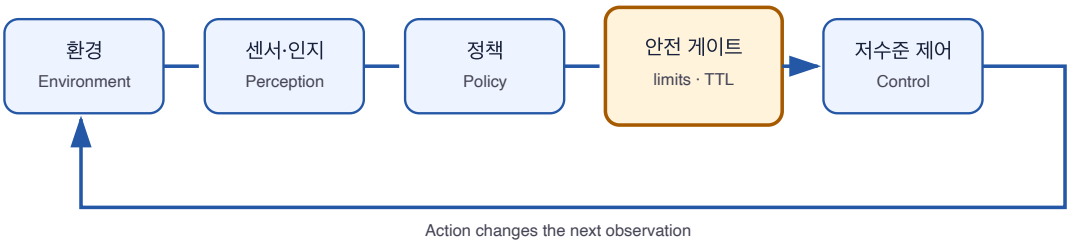
*“디지털 트윈에게 먼저 인사해 두면, 디버깅할 때 덜 어색하다.”*

# 제1장 Physical AI와 Isaac Sim·OpenUSD·ROS 2 기반

## 장 개요

Physical AI는 인공지능 모델이 텍스트나 화면 안에서 답을 생성하는 데 그치지 않고, 센서로 물리 세계를 관측하고 모터를 통해 그 세계를 바꾸는 시스템을 다룬다. 로봇 제어에서는 모델의 정확도만큼 **시간**, **좌표계**, **동역학**, **통신 품질**, **안전 경계**가 중요하다. 같은 신경망이라도 관측이 200 ms 늦거나 관절 명령의 단위가 뒤바뀌면 전혀 다른 기계가 된다.

이 장은 이후 장의 공통 언어를 만든다. 먼저 규칙 기반 제어와 학습 정책을 비교하고, 인지-정책-행동(Perception-Policy-Action) 루프를 확률적 의사결정 문제로 해석한다. 이어 NVIDIA Isaac Sim을 구성하는 Kit, OpenUSD, PhysX, 렌더링 계층을 살펴본다. OpenUSD의 Stage, Layer, Prim, Attribute, Relationship과 합성(composition)을 코드로 다룬 뒤, ROS 2 Jazzy의 그래프와 DDS 기반 통신을 연결한다. 마지막에는 시뮬레이터와 제어기를 잇는 브리지 설계와 실패 격리 원칙을 정리한다.




Physical AI 폐루프 구조

**그림 1-1.** 관측, 추론, 안전 필터, 액추에이터, 환경이 닫힌 루프를 이룬다. 정책 출력은 곧바로 모터 명령이 아니라 안전 계층을 통과해야 한다.

## 학습 목표

이 장을 마치면 다음을 수행할 수 있다.

1. 규칙 기반 제어와 학습 기반 정책의 장단점을 작업 분포와 실패 모드 관점에서 비교한다.
2. Physical AI 제어 주기를 지연 시간 예산(latency budget)이 포함된 폐루프로 설계한다.
3. Isaac Sim의 런타임 구성 요소와 각 구성 요소가 담당하는 책임을 설명한다.
4. OpenUSD Stage를 생성하고 Prim, Attribute, Relationship 및 Layer 합성을 코드로 조작한다.
5. ROS 2의 Node, Topic, Service, Action을 사용 목적에 맞게 선택한다.
6. QoS 정책이 카메라·관절 상태·명령 스트림에 미치는 영향을 추론한다.
7. 시뮬레이터-ROS 2 브리지에서 시간, 좌표계, 단위, 소유권을 명시한다.

## 선수 지식과 준비물

- Python 3의 클래스, 예외, NumPy 배열
- C++17의 클래스, 람다, 스마트 포인터
- 선형대수의 벡터·행렬 기초
- 프로세스와 스레드, TCP/UDP의 기본 개념
- Ubuntu 24.04 LTS, ROS 2 Jazzy Jalisco
- NVIDIA Isaac Sim은 선택 사항이다. OpenUSD 예제는 `usd-core` 또는 Isaac Sim Python 환경에서 실행할 수 있다.

> **버전 주의**: 이 책의 ROS 예제는 Jazzy를 기준으로 한다. Isaac Sim Python API와 확장 이름은 제품 릴리스에 따라 이동할 수 있다. 특정 API 이름보다 Stage, Simulation Context, ROS graph, clock ownership이라는 설계 개념을 우선해서 익힌다.

# 1.1 Physical AI의 문제 정의

## 1.1.1 정보 처리에서 물리적 상호작용으로

일반적인 소프트웨어 함수는 같은 입력에 같은 출력을 내도록 설계할 수 있다. 로봇의 입력은 그렇지 않다. 카메라 노출, 조명, 마찰, 기어 백래시(backlash), 통신 지연, 물체의 미세한 위치가 매 주기 달라진다. 로봇은 부분 관측(partial observation) 아래에서 행동하고, 그 행동이 다음 관측 자체를 바꾼다.

이를 부분 관측 마르코프 의사결정 과정(Partially Observable Markov Decision Process, POMDP)으로 쓰면 다음과 같다.

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, R, \mathcal{O}, O, \gamma \rangle$$

- $s_t \in \mathcal{S}$: 물체 자세, 관절 상태, 마찰처럼 실제 세계의 상태
- $o_t \in \mathcal{O}$: 카메라 이미지, 엔코더, 언어 명령 등 관측
- $a_t \in \mathcal{A}$: 관절 위치·속도·토크 명령
- $T(s_{t+1}|s_t, a_t)$: 행동 뒤 상태가 변하는 동역학
- $R(s_t, a_t)$: 성공, 시간, 에너지, 충돌을 반영하는 보상
- $O(o_t|s_t)$: 상태가 센서 관측으로 나타나는 과정

정책(policy) $\pi(a_t|h_t)$는 현재 관측 하나만이 아니라 관측과 행동의 이력 $h_t$를 사용할 수 있다. Vision-Language-Action(VLA) 모델은 영상과 언어를 조건으로 연속 또는 이산화된 행동을 출력하는 학습 정책의 한 종류다. 그러나 VLA도 센서 드라이버, 좌표 변환, 제어 주기, 안전 제한을 대신하지 않는다.

> **이해 도움말 — 상태(state)와 관측(observation)을 분리해서 생각하기**
>
> 서랍 속 물체의 정확한 자세는 **상태**지만, 로봇이 실제로 받는 것은 카메라 영상과 엔코더 값이라는 **관측**이다. 한 장의 영상에서 가려진 면은 보이지 않으므로 관측만으로 상태를 완전히 알 수 없다. 이때 이전 관측과 행동의 이력 $h_t$는 "방금 어느 방향으로 밀었는가" 같은 단서를 보충한다. POMDP는 수식을 암기하기보다 **세계에는 실제 상태가 있고, 정책에는 제한된 관측만 들어온다**는 구분부터 잡으면 이해하기 쉽다.

## 1.1.2 규칙 기반 제어와 학습 기반 정책

| 관점 | 규칙·모델 기반 | 학습 기반 정책 |
|---|---|---|
| 지식 표현 | 상태기계, 기구학, 경로계획, 제어법칙 | 데이터로 학습한 파라미터 |
| 강점 | 해석 가능, 경계조건 명시, 검증 용이 | 복잡한 시각 특징과 다양한 작업 분포 처리 |
| 약점 | 예외 규칙 증가, 비정형 인식에 취약 | 분포 밖 입력, 설명 곤란, 데이터 편향 |
| 적합 영역 | 안전 제한, 충돌 회피, 정밀 제어 | 인식, 파지 후보, 행동 시퀀스 제안 |
| 실패 대응 | 조건 위반과 상태 전이 추적 | 신뢰도·데이터·관측 분포 분석 필요 |

현업 시스템은 둘 중 하나를 고르는 대신 계층화한다. 학습 정책이 목표 자세나 짧은 행동 청크를 제안하고, 모델 기반 제어기가 추종하며, 독립 안전 계층이 속도·작업공간·충돌 제한을 강제한다. 학습 정책이 실패해도 안전 제한은 유지되어야 한다.

### 1.1.3 인지-정책-행동 루프

한 주기의 이상적인 데이터 흐름은 다음과 같다.

1. **감지(Sensing)**: 카메라, 엔코더, 힘 센서가 타임스탬프가 있는 원시 데이터를 생성한다.
2. **인지(Perception)**: 보정·필터링·특징 추출로 정책 입력을 만든다.
3. **정책(Policy)**: 목표 또는 행동 청크를 추론한다.
4. **안전 필터(Safety filter)**: 관절 한계, 속도, 작업공간, 오래된 명령을 검사한다.
5. **저수준 제어(Control)**: 보간, PID/임피던스 제어, 모터 프로토콜 변환을 수행한다.
6. **행동과 피드백(Action & feedback)**: 물리 상태가 바뀌고 다음 관측이 들어온다.

주기 $T$는 구성 요소 시간의 합과 대기 시간으로 결정된다.

$$T_{loop} = T_{sensor} + T_{transport} + T_{pre} + T_{infer} + T_{safety} + T_{actuator} + T_{wait}$$

20 Hz 정책 루프의 주기 예산은 50 ms다. 추론이 평균 35 ms여도 최악 지연이 80 ms이면 주기를 놓친다. 따라서 평균뿐 아니라 p95/p99 지연, 센서 데이터의 나이(age), 명령 만료 시간(time-to-live)을 기록해야 한다.

### 1.1.4 책임의 분리

좋은 로봇 소프트웨어는 다음 네 가지 책임을 섞지 않는다.

- **세계 모델**: 좌표 프레임, 장면, 로봇 상태
- **의사결정**: 무엇을 할 것인가
- **제어**: 목표를 어떤 주기로 추종할 것인가
- **안전**: 어떤 행동을 절대 허용하지 않을 것인가

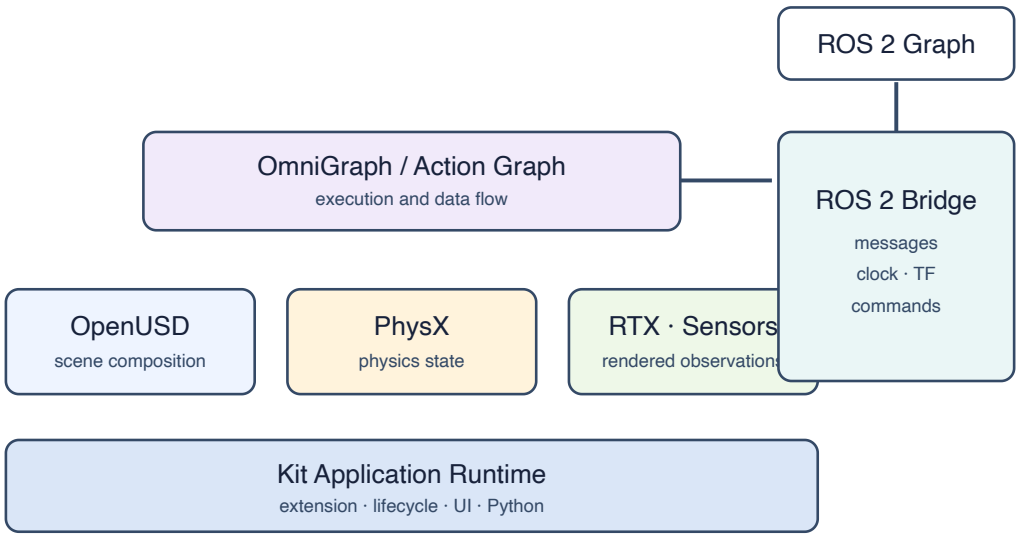
예제 `code/physical_ai_loop.py`는 이 분리를 최소한의 코드로 보여 준다. 정책은 목표를 제안하고 `SafetyGate`가 범위와 명령 나이를 검사한다. 실제 로봇에서는 소프트웨어 제한과 별도로 하드웨어 비상 정지와 전류 제한이 필요하다.

# 1.2 NVIDIA Isaac Sim 런타임 이해

## 1.2.1 구성 요소의 역할

Isaac Sim은 하나의 물리 엔진이 아니라 여러 계층이 결합된 애플리케이션이다.

- **Kit 애플리케이션 런타임**: 확장(extension), UI, 명령, Python 환경의 생명주기를 관리한다.
- **OpenUSD**: 장면의 구조, 속성, 참조, 변형을 표현한다.
- **PhysX**: 강체, 관절, 접촉, 충돌을 시간에 따라 적분한다.
- **RTX 렌더러와 센서 모델**: 카메라 관측과 합성 데이터를 생성한다.
- **OmniGraph/Action Graph**: 데이터 흐름과 실행 노드를 연결한다.
- **ROS 2 Bridge**: ROS 메시지와 시뮬레이터의 상태·명령을 변환한다.




Isaac Sim 계층

**그림 1-2.** USD는 장면의 정적·구성적 표현을, PhysX는 시간에 따른 상태 변화를, ROS 2 Bridge는 외부 시스템과의 메시지 교환을 담당한다.

## 1.2.2 Stage와 시뮬레이션 상태를 구분하라

Stage에 `translate = (0, 0, 1)`을 작성하는 것은 장면 기술을 바꾸는 일이다. 반면 물리 스텝 뒤 강체가 낙하해 현재 자세가 달라지는 것은 런타임 상태 변화다. 두 경로를 무분별하게 섞으면 물리 엔진이 계산한 값을 매 프레임 덮어쓰거나, 저장한 USD가 예상과 달라진다.

다음 질문으로 소유권을 정한다.

- 초기 자세는 어느 Layer가 작성하는가?
- 시뮬레이션 중 관절 명령은 ROS와 그래프 중 누가 소유하는가?
- 물리 결과를 USD에 저장할 것인가, 별도 로그로 남길 것인가?
- Reset 시 어떤 값이 초기화되는가?

### 1.2.3 확장 기반 실행과 독립 실행

GUI에서의 작업은 탐색과 디버깅에 유리하다. 반복 가능한 데이터 생성이나 자동 실험은 독립 실행(standalone) Python이 적합하다. 일반적인 순서는 애플리케이션 생성, 필요한 확장 활성화, Stage 준비, 물리 초기화, 반복 스텝, 종료다.

```
# 개념적 생명주기 예시: 릴리스별 import 경로를 공식 문서에서 확인한다.
simulation_app = SimulationApp({"headless": True})
try:
    # USD/Isaac 모듈은 앱 생성 후 import하는 버전이 많다.
    prepare_stage()
    reset_world()
    while simulation_app.is_running():
        step_world(render=False)
finally:
    simulation_app.close()
```

앱 생성 전에 렌더러 의존 모듈을 import하거나, 종료 시 `close()`를 호출하지 않으면 플러그인 초기화·정리 문제를 만들 수 있다. 이 책의 예제 `code/create_usd_scene.py`는 Isaac Sim 생명주기와 독립적으로 OpenUSD 데이터 모델만 연습하도록 구성했다.

# 1.3 OpenUSD 데이터 모델

## 1.3.1 Stage, Layer, Prim

**Layer**는 USD 의견(opinion)을 저장하는 단위다. 파일일 수도 있고 메모리 안의 익명 Layer일 수도 있다. **Stage**는 여러 Layer와 composition arc를 합성해 애플리케이션이 보는 하나의 장면을 제공한다. **Prim**은 `/World/Robot/arm_link`처럼 경로를 갖는 장면 객체다.

파일 시스템 비유는 유용하지만 완전하지 않다. Prim은 단순 폴더가 아니라 타입, 속성, 관계, 메타데이터, 자식 Prim을 가진다. 여러 Layer가 같은 Prim 경로에 서로 다른 의견을 낼 수 있으며, Stage는 합성 규칙으로 최종 값을 결정한다.

```
Stage
└── /World                          (Xform Prim)
    ├── /World/Worktable            (Cube Prim)
    ├── /World/Robot                (Reference로 가져온 Prim)
    │   └── /World/Robot/Tool
    └── /World/Target               (Sphere Prim)
```

Prim 경로는 절대 경로로 명시하면 `/`로 시작한다. 이름 변경은 코드·관계·센서 설정에 영향을 줄 수 있으므로 장면 경로를 공개 API처럼 관리한다.

## 1.3.2 Attribute와 Relationship

**Attribute**는 시간 샘플을 가질 수 있는 값이다. 위치, 색, 질량처럼 수치나 토큰을 저장한다. **Relationship**은 다른 Prim 또는 Property 경로를 가리킨다. 센서가 어떤 링크에 부착되었는지, 재질이 어떤 대상과 연결되는지처럼 값 복사보다 참조가 자연스러운 경우에 쓴다.

```
prim = stage.GetPrimAtPath("/World/Target")
purpose = prim.CreateAttribute("robotics:purpose", Sdf.ValueTypeNames.Token)
purpose.Set("pick_target")

rel = prim.CreateRelationship("robotics:observedBy")
rel.SetTargets([Sdf.Path("/World/Camera")])
```

사용자 정의 namespace `robotics:`는 표준 스키마와 충돌을 피하고 의미를 묶는다. 다만 팀 프로젝트에서는 임의 Attribute를 늘리기 전에 Typed/API Schema로 만들 필요가 있는지 검토한다.

## 1.3.3 변환과 XformOp

`UsdGeom.Xformable`은 변환 연산(XformOp)의 순서를 명시한다. 단일 4×4 행렬, translate/orient/scale의 조합 등 여러 표현이 가능하다. 단순히 속성 이름만 찾아 값을 바꾸면 기존 `xformOpOrder`와 충돌할 수 있다.

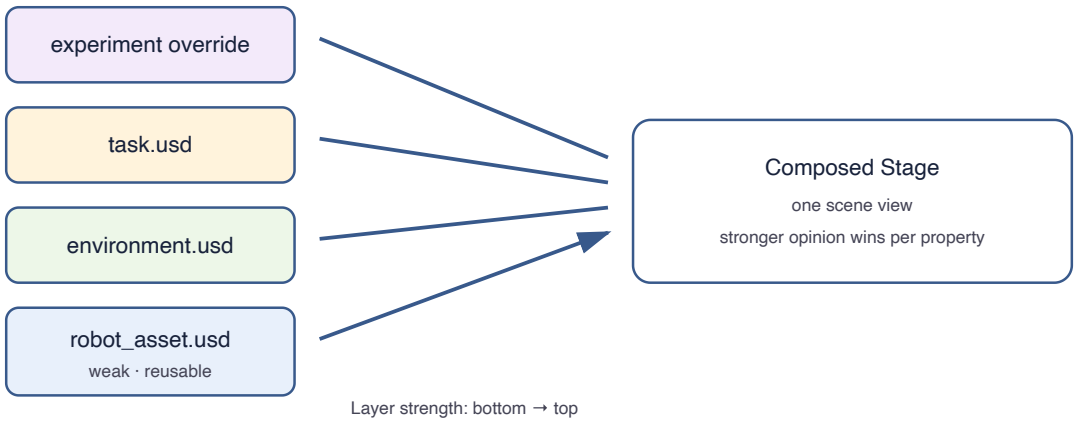
쿼터니언 순서도 주의한다. USD의 `Gf.Quatd(real, imaginary)`는 실수부 $w$를 먼저 받는 반면, ROS 메시지 `geometry_msgs/Quaternion` 필드는 `x, y, z, w` 순서다. 배열을 그대로 복사하지 말고 이름 있는 필드로 변환한다.

### 1.3.4 비파괴 합성

OpenUSD의 핵심은 원본 자산을 수정하지 않고 의견을 겹치는 것이다.

- **SubLayer**: Layer 스택에 다른 Layer를 포함한다. 환경, 조명, 작업 설정을 계층화할 때 유용하다.
- **Reference**: 특정 Prim 아래에 외부 자산을 가져온다. 한 로봇 자산을 여러 장면에 재사용한다.
- **Payload**: Reference와 비슷하지만 필요할 때 적재해 대형 장면의 작업 집합을 줄인다.
- **Inherit**: 클래스 Prim의 의견을 상속한다.
- **VariantSet**: 같은 자산의 그리퍼, 재질, 상세도 같은 선택지를 제공한다.
- **Specialize**: 기반 정의를 특수화한다.

강한 Layer의 override가 약한 Layer의 값을 가릴 수 있다. "파일에는 값이 맞는데 Stage에서 다르게 보인다"면 먼저 Prim Stack과 Layer Stack을 조사한다. 값을 반복해서 덮어써 증상을 숨기지 않는다.




OpenUSD 합성

**그림 1-3.** 로봇 자산, 작업 환경, 실험별 override를 분리하면 원본 자산을 보존하면서 여러 실험을 재현할 수 있다.

### 1.3.5 예제 장면 생성

`code/create_usd_scene.py`는 다음을 수행한다.

1. 메모리 또는 파일 Stage를 생성한다.
2. `/World`를 default prim으로 지정한다.
3. 바닥, 작업대, 목표 물체, 카메라 Prim을 만든다.
4. 사용자 Attribute와 Relationship을 작성한다.
5. 별도 session Layer에서 목표 색을 override한다.

**예상 출력 예시**

```
stage: physical_ai_scene.usda
default prim: /World
target purpose: pick_target
observed by: [/World/Camera]
composed display color: [(0.9, 0.2, 0.1)]
```

생성된 USDA를 텍스트 편집기로 열어 계층과 값이 어떻게 기록되는지 확인한다. 바이너리 USDC는 배포에 유리하지만 학습 단계에서는 USDA가 디버깅하기 쉽다.

# 1.4 ROS 2 Jazzy 통신 모델

## 1.4.1 ROS 그래프와 Node

ROS 2 그래프는 실행 중인 Node와 그 사이의 인터페이스를 나타낸다. Node는 일반적으로 하나의 논리적 책임을 갖는다. 카메라 드라이버, 상태 추정, 정책 추론, 안전 필터를 하나의 거대한 Node로 묶으면 장애 격리와 주기 측정이 어려워진다.

Node는 서로 다른 프로세스·호스트에 있을 수도 있고 component로 같은 프로세스에 합성될 수도 있다. 프로세스를 나누면 장애 격리는 좋아지지만 직렬화와 스케줄링 비용이 늘 수 있다. 초기에는 책임 경계를 올바르게 나누고, 측정 후 필요한 경로만 최적화한다.

## 1.4.2 Topic, Service, Action 선택

| 프리미티브 | 의미 | 로봇 예 | 피해야 할 사용 |
|---|---|---|---|
| Topic | 지속적인 비동기 데이터 스트림 | 카메라, JointState, 속도 명령 | 한 번의 요청에 반드시 한 번 응답해야 하는 설정 |
| Service | 짧은 요청-응답 RPC | 캘리브레이션 상태 조회, gripper reset | 수 초 이상 걸리거나 진행률·취소가 필요한 작업 |
| Action | 장시간 Goal, Feedback, Result, Cancel | Pick-and-place, 경로 추종 | 100 Hz 연속 센서 스트림 |

Action은 내부적으로 여러 Topic과 Service를 사용하지만, 사용자는 Goal의 상태와 취소 의미를 명시적인 인터페이스로 다룬다. “시작” Service와 “상태” Topic을 임의로 조합하는 것보다 장기 작업에는 Action이 일관적이다.

> **잠깐 점검 — 어떤 ROS 2 인터페이스를 고를까?**
>
> 1. 30 fps 카메라 영상은 Topic, Service, Action 중 무엇이 적절한가?
> 2. 20초 동안 진행되며 진행률 표시와 취소가 필요한 Pick-and-place는 무엇이 적절한가?
>
> **확인 포인트:** 지속 데이터 스트림인 카메라는 **Topic**, 목표·피드백·결과·취소가 필요한 장기 작업은 **Action**이 적절하다.

## 1.4.3 DDS와 RMW

ROS 2 클라이언트 라이브러리 `rclpy`와 `rclcpp` 아래에는 ROS middleware interface(RMW)가 있다. RMW 구현은 DDS 또는 다른 전송 계층과 ROS 타입·발견·통신을 연결한다. 애플리케이션은 보통 동일한 ROS API를 사용하지만, 발견 방식, 설정, 성능 특성은 구현에 따라 달라질 수 있다.

DDS의 데이터 중심 publish/subscribe는 중앙 브로커 없이 참가자를 발견하고, Topic 단위의 QoS를 협상한다. 이는 네트워크 설정이 필요 없다는 뜻이 아니다. 멀티캐스트가 차단된 네트워크, 컨테이너, VPN에서는 발견이 실패할 수 있다.

## 1.4.4 QoS는 인터페이스 계약이다

중요 QoS 정책은 다음과 같다.

- **Reliability**: `RELIABLE`은 전달 재시도를, `BEST_EFFORT`는 최신성 우선 전달을 뜻한다.

- **Durability**: `TRANSIENT_LOCAL`은 늦게 참여한 구독자에게 보관 표본을 제공할 수 있다.
- **History/Depth**: 큐에 보관할 표본 수를 제한한다.
- **Deadline**: 표본이 기대 주기 안에 도착해야 한다는 계약을 표현한다.
- **Liveliness**: 발행자가 살아 있음을 판단한다.
- **Lifespan**: 오래된 표본의 유효 기간을 제한한다.

카메라 미리보기는 손실된 과거 프레임을 재전송하느라 현재 프레임이 늦어지는 것보다 `BEST_EFFORT`, 작은 depth가 나을 수 있다. 반대로 낮은 빈도의 설정 변경은 `RELIABLE`과 `TRANSIENT_LOCAL`이 적절할 수 있다. 발행자와 구독자의 QoS가 호환되지 않으면 그래프에 이름이 보여도 데이터가 흐르지 않는다.

### 1.4.5 시간의 두 종류

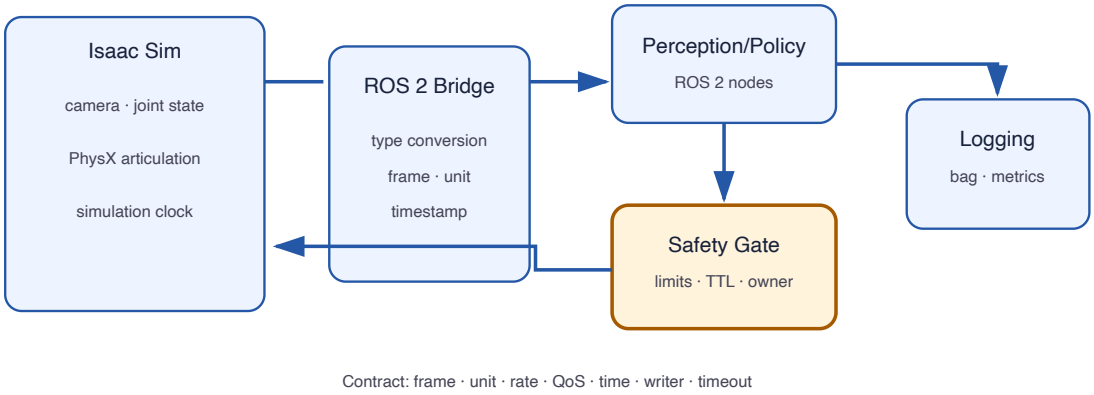
ROS Node는 시스템 시간과 시뮬레이션 시간을 구분한다. `/clock`을 사용하고 `use_sim_time=true`이면 시뮬레이터가 시간의 권위자(authority)가 된다. 시뮬레이션을 일시정지하거나 reset하면 시간도 멈추거나 되돌아갈 수 있다.

- 메시지 헤더는 측정 시각을 기록한다.
- 처리 완료 시각을 헤더에 넣지 않는다.
- `steady_clock`은 타임아웃 측정에, ROS clock은 데이터 정렬에 구분해 사용한다.
- reset 뒤 이전 epoch의 버퍼를 비운다.

# 1.5 Isaac Sim-ROS 2 브리지 설계

## 1.5.1 경계에서 명시해야 할 네 가지

1. **좌표 프레임**: `world`, `base_link`, `tool0`, 카메라 optical frame 사이의 TF 관계
2. **단위**: radian/degree, metre/millimetre, N/N·m
3. **시간**: 시뮬레이션 clock과 메시지 timestamp의 소유자
4. **명령 소유권**: 같은 관절을 그래프, ROS controller, GUI가 동시에 쓰지 않도록 단일 writer 지정




ROS 2 브리지 데이터 흐름

**그림 1-4.** 관측 경로와 명령 경로를 분리하고 안전 게이트를 시뮬레이터 밖의 독립 Node로 두면 실물 전환 시 같은 계약을 유지하기 쉽다.

## 1.5.2 권장 인터페이스

- `/joint_states` (`sensor_msgs/msg/JointState`): 관절 위치·속도·힘
- `/tf`, `/tf_static`: 좌표 프레임 관계
- 카메라 영상과 `CameraInfo`: 영상과 내부 파라미터
- `/clock`: 시뮬레이션 시간
- 제어 명령: controller가 요구하는 trajectory 또는 명시적 target message
- 작업 수준: 자체 Action 또는 표준 controller Action

JointState의 `name`과 각 배열은 인덱스가 대응한다. 빈 velocity를 허용하는지, 모든 관절을 매번 포함하는지 계약을 명시한다. 이름 순서를 고정된 하드코딩 인덱스로 가정하지 말고 map을 구성한다.

## 1.5.3 단계별 통합 전략

1. 시뮬레이터를 정지한 상태에서 Stage 경로와 관절 이름을 확인한다.
2. `/clock`, `/joint_states`, `/tf`처럼 읽기 전용 관측 경로를 먼저 연결한다.
3. 단일 관절을 낮은 속도로 명령하고 부호와 단위를 확인한다.
4. 명령 만료 watchdog과 범위 제한을 활성화한다.

5. reset, pause, bridge 재시작, 구독자 지연을 의도적으로 발생시킨다.
6. 정상 경로보다 실패 뒤 복구가 결정적인지 확인한다.

## 1.6 구현 예제 안내

### 1.6.1 순수 Python 폐루프

`code/physical_ai_loop.py`는 외부 로봇 프레임워크 없이 1차원 관절을 제어한다.

```
python3 code/physical_ai_loop.py
```

**예상 출력 예시**

```
step=00 q=+0.000 proposed=+0.700 safe=+0.700 age_ms=0.0
step=05 q=+0.331 proposed=+0.700 safe=+0.700 age_ms=0.0
step=10 q=+0.505 proposed=+0.700 safe=+0.700 age_ms=0.0
step=15 q=+0.597 proposed=+0.700 safe=+0.700 age_ms=0.0
watchdog: stale command rejected
```

핵심은 수치 결과가 아니라 정책과 안전 필터의 독립성이다.

### 1.6.2 ROS 2 Python 상태 발행기

`code/ros2_jazzy/physical_ai_demo/physical_ai_demo/joint_state_source.py`는 가상의 관절 상태를 발행한다. 센서형 QoS와 명시적 timestamp를 사용한다.

```
cd code/ros2_jazzy/physical_ai_demo
colcon build --symlink-install
source install/setup.bash
ros2 run physical_ai_demo joint_state_source
```

다른 터미널:

```
ros2 topic echo /demo/joint_states
ros2 topic hz /demo/joint_states
```

**예상 출력 예시**

```
average rate: 49.9
name: [joint_1]
position: [0.31]
```

### 1.6.3 ROS 2 C++ 안전 게이트

`code/ros2_jazzy/safety_gate_cpp/`는 목표 JointState를 구독해 관절 한계와 메시지 나이를 검사한 뒤 안전한 명령을 발행한다. 예제는 교육용이며, 실물의 인증된 안전 장치를 대체하지 않는다.

장 루트에서 두 패키지를 함께 빌드한다.

```
cd code/ros2_jazzy
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select physical_ai_demo safety_gate_cpp
source install/setup.bash
```

터미널 A에서 게이트, 터미널 B에서 현재 timestamp가 있는 교육용 명령 발생기, 터미널 C에서 안전 명령을 실행한다.

```
# 터미널 A
ros2 run safety_gate_cpp safety_gate

# 터미널 B
ros2 run physical_ai_demo proposed_command_source

# 터미널 C
ros2 topic echo /demo/safe_command
```

**예상 출력 예시**

```
name: [joint_1]
position: [0.8]
```

`proposed_command_source`의 목표를 1.5 rad로 바꾸면 기본 상한 1.0 rad로 clamp된다. timestamp를 0으로 바꾸면 게이트가 stale/future command로 거부해야 한다.

## 1.7 단계별 실습

상세 절차와 제출물은 다음 파일에 있다.

- `[SIM]` 실습 1-1: 제어 주기와 지연 예산 설계
- `[SIM]` 실습 1-2: OpenUSD 장면과 override 생성
- `[SIM]` 실습 1-3: Topic/Service/Action 및 QoS 관찰
- `[SIM→HW]` `[SAFETY]` 실습 1-4: 시뮬레이터-ROS 브리지 인터페이스 명세

실습은 먼저 시뮬레이션 또는 가상 데이터로 수행한다. 실물 로봇에 적용하기 전 작업 공간을 비우고, 속도·토크를 최소로 제한하며, 물리적 비상 정지와 전원 차단 담당자를 정한다.

## 1.8 트러블슈팅

| 증상 | 가능 원인 | 진단 | 해결 방향 |
|---|---|---|---|
| Topic 이름은 보이나 데이터 없음 | QoS 비호환 | `ros2 topic info -v` | reliability/durability 계약 맞춤 |
| 다른 PC의 Node가 보이지 않음 | 도메인 ID, 방화벽, 멀티캐스트 | 동일 호스트와 네트워크를 분리 점검 | `ROS_DOMAIN_ID`와 DDS discovery 설정 확인 |
| 시뮬레이션 pause 중 timeout 폭주 | ROS clock과 steady clock 혼용 | clock source 로그 | 데이터 시간과 watchdog 시간 분리 |
| USD 파일 값과 화면 값이 다름 | 강한 Layer override | Layer/Prim stack 확인 | 의견이 작성된 Layer 수정 또는 mute |
| 물체가 매 프레임 원위치 | USD transform writer와 physics 동시 소유 | writer 목록과 graph 확인 | 단일 writer 원칙 적용 |
| 관절이 반대로 움직임 | 축/부호/단위 불일치 | 한 관절 저속 시험 | 이름 기반 mapping과 단위 변환 명시 |
| 카메라가 느려짐 | reliable 재전송, queue 누적 | topic delay와 queue 측정 | sensor QoS, 작은 depth, 소비율 확보 |
| bridge 재시작 후 명령 급변 | 오래된 명령 재사용 | timestamp/sequence 확인 | lifespan, watchdog, reset 시 버퍼 비움 |

## 1.9 요약

- Physical AI는 관측-정책-안전-제어-환경이 닫힌 시간 의존 시스템이다.
- 학습 정책은 모델 기반 제어와 독립 안전 계층을 보완하며, 이를 대체하지 않는다.
- Isaac Sim에서 OpenUSD는 장면 구성, PhysX는 물리 상태 변화, ROS 2 Bridge는 외부 메시지 변환을 담당한다.
- Stage는 여러 Layer의 의견을 합성한 뷰이며, Prim은 경로 기반 장면 객체다.
- ROS 2 Topic, Service, Action은 각각 스트림, 짧은 RPC, 취소 가능한 장기 작업에 사용한다.
- QoS, timestamp, frame, unit, command ownership은 브리지의 핵심 계약이다.
- 정상 동작뿐 아니라 pause, reset, 지연, 재시작 뒤의 결정적 복구를 설계해야 한다.

## 1.10 확인문제

1. POMDP에서 상태와 관측을 구분해야 하는 로봇 예를 두 가지 제시하라.
2. 20 Hz 루프에서 카메라 8 ms, 전송 3 ms, 전처리 5 ms, 추론 26 ms, 안전 필터 2 ms가 걸린다. 남은 예산과 위험 요인을 계산하라.
3. 학습 정책의 출력이 관절 토크일 때 위치 목표 출력보다 안전 검증이 어려운 이유를 설명하라.
4. Stage와 Layer를 Git working tree와 commit에 비유할 때 맞는 점과 틀린 점을 각각 쓰라.
5. Reference와 SubLayer를 각각 선택할 장면 구성 사례를 제시하라.
6. `geometry_msgs/Quaternion`과 `Gf.Quatd` 사이 변환에서 발생할 수 있는 오류를 설명하라.
7. 카메라 Topic에 `RELIABLE`, depth 100을 사용했을 때 느린 구독자가 만드는 현상을 예측하라.
8. Service보다 Action이 적합한 로봇 작업 두 가지와 그 이유를 쓰라.
9. 시뮬레이션 reset 뒤 동기화 버퍼를 비워야 하는 이유를 시간축 관점에서 설명하라.
10. bridge의 단일 writer 원칙이 깨졌을 때 관찰 가능한 증상을 세 가지 제시하라.

정답 및 평가 기준은 제1장 확인문제 해설 및 평가 기준에 있다.

# 1.11 프로젝트 과제

## 과제 A: Physical AI 인터페이스 설계서

SO-101이 테이블의 색상 블록을 집어 트레이에 놓는 시스템을 설계한다. 다음을 포함한 4~6쪽 설계서를 작성하라.

- Node와 Topic/Service/Action 목록
- 각 메시지의 frame, unit, rate, QoS, owner
- 50 ms 정책 주기의 latency budget
- timeout, stale command, camera dropout 처리
- 실물과 시뮬레이터에서 동일하게 유지할 계약

## 과제 B: 비파괴 USD 작업 셀

로봇 자산을 수정하지 않고 `environment`, `task`, `experiment_override` Layer를 분리한다. 두 종류의 테이블 재질과 세 종류의 목표 위치를 variant 또는 override로 표현하고, Stage의 최종 합성 값을 출력한다.

## 평가 기준

| 항목 | 배점 |
|---|---|
| 인터페이스 의미와 책임 경계 | 25 |
| 시간·좌표·단위의 명시 | 20 |
| QoS와 실패 처리 | 20 |
| USD 합성 구조 | 20 |
| 코드 가독성·문서·출처 | 15 |

# 1.12 실습·평가·참고자료

이 절 아래에 장별 참고문헌, 단계별 실습, 확인문제 해설과 평가 기준을 한데 모았다.

## 1.12.1 참고문헌


1. ROS 2 Documentation, Nodes, Topics, Services, Actions, Jazzy.
2. ROS 2 Design, ROS on DDS 및 Clock and Time.
3. ROS 2 Documentation, Quality of Service settings, Jazzy.
4. OpenUSD, Introduction to USD, Glossary, USD API.
5. Pixar Animation Studios, Universal Scene Description.
6. NVIDIA, Isaac Sim Documentation, ROS 2 Tutorials.
7. NVIDIA, Omniverse Kit Documentation.
8. Kober, J., Bagnell, J. A., Peters, J., Reinforcement Learning in Robotics: A Survey, IJRR, 2013.
9. Levine, S. et al., End-to-End Training of Deep Visuomotor Policies, JMLR, 2016.
10. Brohan, A. et al., RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control, 2023.


## [SIM] 실습 1-1: Physical AI 지연 예산

```
cd book/01_physical_ai_isaac_ros2
```

**목표**

50 ms 정책 주기를 구성 요소별로 배분하고 평균이 아닌 최악 지연을 기준으로 실패를 판단한다.

**절차**

1. `code/physical_ai_loop.py`의 감지, 정책, 안전, plant 단계를 구분한다.
2. 각 단계에 `perf_counter_ns()` 측정을 추가한다.
3. 1,000회 실행해 평균, p95, p99를 계산한다.
4. 정책 단계에 0~40 ms의 무작위 지연을 추가한다.
5. 50 ms deadline miss 비율과 command age를 표로 작성한다.
6. 큐를 1개만 유지하는 latest-only 전략과 모든 명령을 순서대로 실행하는 전략을 비교한다.

**제출물**

- 단계별 p50/p95/p99 표
- deadline miss 원인 3가지
- watchdog TTL 선정 근거

**토의**

평균 지연이 50 ms 이하인데도 로봇이 불안정할 수 있는 이유를 설명하라.

## [SIM] 실습 1-2: OpenUSD 장면과 비파괴 Override

```
cd book/01_physical_ai_isaac_ros2
```

**목표**

Stage, Layer, Prim, Attribute, Relationship을 코드로 생성하고 합성 강도를 관찰한다.

**절차**

1. OpenUSD Python 환경에서 `python3 code/create_usd_scene.py`를 실행한다.
2. 생성된 USDA에서 `/World/Target`을 찾는다.
3. Session Layer override가 파일에 저장되지 않지만 합성 결과에는 나타나는 이유를 설명한다.
4. 별도 `experiment.usda`를 만들고 target 위치에 더 강한 의견을 작성한다.
5. target에 `robotics:taskId` Attribute를 추가한다.
6. 카메라 두 대를 만들고 Relationship target을 복수로 설정한다.

**예상 결과 예시**

```
root layer color: blue
session composed color: red
relationship targets: [/World/Camera, /World/WristCamera]
```

**제출물**

- USDA 2개
- Layer stack과 최종 값 설명
- Prim 경로 설계 원칙 3개

## [SIM] 실습 1-3: ROS 2 인터페이스와 QoS

```
cd book/01_physical_ai_isaac_ros2
```

**목표**

Topic의 빈도와 QoS를 관찰하고 Service/Action의 적용 범위를 구분한다.

**절차**

1. `physical_ai_demo`를 빌드하고 `joint_state_source`를 실행한다.
2. `ros2 node list`, `ros2 topic list -t`, `ros2 topic info -v /demo/joint_states`를 기록한다.
3. `ros2 topic hz`와 `ros2 topic delay`로 주기와 지연을 관찰한다.
4. 구독자의 reliability를 reliable로 강제했을 때 sensor-data publisher와 호환되는지 확인한다.
5. 다음 기능을 Topic/Service/Action 중 하나로 분류한다: 카메라, 비상 정지 요청, 홈 이동, 캘리브레이션 조회, pick-and-place.

#### 제출물

- ROS graph 스케치
- QoS 호환성 관찰 결과
- 인터페이스 선택 표와 근거

## [SIM→HW] [SAFETY] 실습 1-4: 시뮬레이터-ROS 2 브리지 계약

```
cd book/01_physical_ai_isaac_ros2
```

#### 목표

구현 전에 데이터 경계의 frame, unit, rate, QoS, owner, timeout을 문서화한다.

#### 절차

SO-101 디지털 트윈을 가정하고 아래 표를 완성한다.

| 데이터 | 타입 | Frame | Unit | Rate | QoS | Writer | Timeout |
|---|---|---|---|---|---|---|---|
| Joint state | JointState | - | rad | 50 Hz | sensor | bridge | 100 ms |
| Wrist image | Image | camera_optical | pixel | 30 Hz | sensor | bridge | 100 ms |
| Joint target | 미정 | - | rad | 20 Hz | reliable | safety gate | 100 ms |

1. `/clock` authority와 reset 동작을 명시한다.
2. GUI와 ROS가 동시에 관절을 쓰지 못하게 ownership 규칙을 작성한다.
3. pause, reset, bridge crash, stale command의 복구 절차를 작성한다.
4. 실물 전환 시 달라지는 부분과 유지되는 계약을 색으로 구분하지 말고 열로 표시한다.

## 제1장 확인문제 해설 및 평가 기준

1. 상태는 직접 접근할 수 없는 실제 물리량, 관측은 센서가 제공한 불완전한 값이다. 예: 물체의 실제 6D pose와 가려진 RGB 영상, 실제 접촉력과 모터 전류 기반 추정값.
2. 명목상 6 ms가 남는다. OS 스케줄링, GPU 동기화, 전송 jitter를 포함하지 않아 여유가 작고 p99 추론이 26 ms보다 클 수 있다.
3. 토크는 동역학에 직접 작용해 짧은 오류도 큰 가속도와 충돌을 만들며, 모델·주기·전류 제한이 필요하다.
4. Layer가 의견을 누적한다는 점은 유사하지만 Stage 합성은 단일 선형 이력도 아니고 Layer가 commit과 일대일 대응하지 않는다.
5. Reference는 로봇 자산을 특정 Prim 아래 재사용할 때, SubLayer는 환경·조명 등 장면 전반 의견을 층으로 포함할 때 적합하다.
6. ROS는 필드 순서가 x,y,z,w이고 `Gf.Quatd` 생성자는 실수부 w가 먼저다. 순서를 그대로 복사하면 전혀 다른 회전이 된다.
7. 느린 구독자와 손실 네트워크에서 재전송·큐 누적으로 오래된 프레임이 늦게 도착해 폐루프 지연이 증가할 수 있다.
8. 경로 추종과 pick-and-place처럼 장시간 진행되고 feedback, cancel, result가 필요한 작업이 Action에 적합하다.
9. reset으로 ROS time이 이전 timestamp보다 작아질 수 있다. 이전 epoch 표본이 남으면 잘못된 보간과 미래 데이터 판정이 발생한다.
10. 관절 떨림, 명령이 되돌아감, 물리 상태가 매 프레임 순간이동함, 로그상 목표가 번갈아 나타남 등이 가능하다.

서술형 답은 용어 암기보다 구체적 실패 경로와 완화책의 연결을 평가한다.

제2장

# Isaac Sim 환경 구축과 PhysX 강체 물리 시뮬레이션

마찰과 충돌을 설명하는 물리 유령과 로봇 그리퍼

*“물체가 미끄러지면 버그일 수도 있고, 마찰계수가 솔직한 것일 수도 있다.”*

# 제2장 Isaac Sim 환경 구축과 PhysX 강체 물리 시뮬레이션

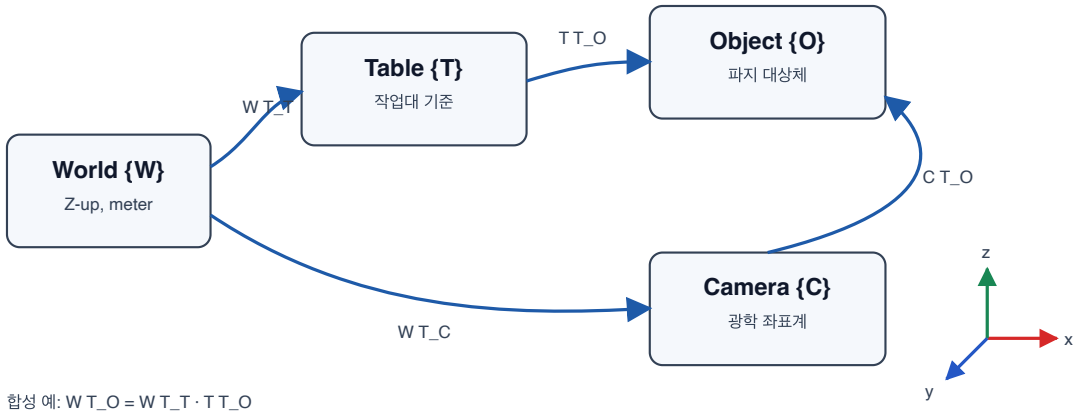
## 장 개요

로봇 정책이 시뮬레이터에서 성공했다는 사실만으로 실물에서도 성공한다고 말할 수는 없다. 시뮬레이터가 계산한 접촉, 질량, 마찰, 카메라 영상이 실제 장비와 충분히 닮아 있어야 학습된 정책의 행동이 현실로 이어진다. 이 장에서는 NVIDIA Isaac Sim의 장면을 단순한 3D 그림이 아니라 **측정 가능하고 반복 가능한 물리 실험 장치**로 구축한다.

먼저 작업대, 트레이, 파지 대상체를 서로 다른 좌표계에 배치하고 4×4 동차 변환 행렬(homogeneous transformation matrix)과 쿼터니언(quaternion)을 이용해 pose를 계산한다. 이어서 PhysX의 강체(rigid body), 충돌 형상(collision shape), 연속 충돌 검출(continuous collision detection, CCD), 마찰과 반발, 질량과 관성 텐서(inertia tensor)를 다룬다. 마지막으로 물리 기반 렌더링(physically based rendering, PBR)과 조명을 설정하고, 물리 파라미터를 실측값에 맞추는 체계적인 튜닝 절차를 설계한다.

이 장의 핵심 관점은 다음 한 문장으로 요약된다.

> 보이는 메시(render mesh), 충돌을 계산하는 메시(collision mesh), 질량을 대표하는 모델(mass model)은 서로 다른 공학적 역할을 가진다.




작업 셀의 좌표계 관계

**그림 2-1.** 월드, 작업대, 카메라, 대상체 좌표계의 변환 체인. 화살표는 부모 좌표계에서 자식 좌표계 pose를 표현하는 변환을 뜻한다.

## 학습 목표

이 장을 마치면 학습자는 다음을 수행할 수 있다.

1. Isaac Sim의 Stage와 PhysicsScene을 구성하고 GUI 및 Standalone Python 개발 방식을 선택할 수 있다.

2. 로컬 좌표계와 월드 좌표계 사이의 4×4 동차 변환을 계산하고, 좌표계 방향을 명시할 수 있다.
3. Euler angle과 quaternion의 장단점 및 convention 차이를 설명할 수 있다.
4. static collider, dynamic rigid body, kinematic body를 용도에 맞게 구분할 수 있다.
5. primitive, convex hull, convex decomposition, triangle mesh 충돌 근사의 비용과 정확도를 비교할 수 있다.
6. discrete collision detection에서 tunneling이 발생하는 이유를 설명하고 CCD 적용 범위를 결정할 수 있다.
7. 마찰, 반발, 질량, 무게중심, 관성 텐서를 측정 또는 계산해 시뮬레이터 초기값으로 사용할 수 있다.
8. 렌더 재질과 물리 재질을 분리해 관리하고, 조명 및 PBR 파라미터를 재현 가능한 형태로 기록할 수 있다.
9. 관찰 가능한 지표를 기준으로 물리 파라미터를 한 번에 하나씩 튜닝할 수 있다.

# 선수 지식과 준비물

## 선수 지식

- Python과 NumPy의 배열 연산
- 벡터, 행렬 곱, 전치행렬, 역행렬
- 고전역학의 힘, 질량, 속도, 운동에너지
- 제1장의 USD Stage, Prim, Attribute 개념

## 기준 환경

- Ubuntu 24.04 LTS
- NVIDIA RTX GPU와 호환 드라이버
- NVIDIA Isaac Sim 최신 안정 릴리스
- Python 3.12(범용 수학 예제), Isaac Sim 내장 Python(시뮬레이터 예제)

Isaac Sim 공식 요구사항은 Ubuntu 22.04와 24.04를 지원 대상으로 명시하고, 복잡한 장면에는 충분한 RAM과 VRAM이 필요하다고 안내한다. 정확한 GPU/드라이버 요구사항은 릴리스마다 달라지므로 설치 직전에 Isaac Sim Requirements를 확인한다. 이 책의 코드는 특정 GPU 성능을 보장하지 않으며 출력은 예시이다.

## 안전 메모

이 장은 주로 시뮬레이션을 다루지만 결과를 실물 로봇에 옮길 때는 별도 안전 검증이 필요하다. 물체 질량, 마찰, 토크 한계가 잘못되면 시뮬레이션에서 안전했던 궤적도 실물에서 미끄러짐이나 충돌을 일으킨다. 실물 적용 전에는 속도와 토크를 낮추고, 작업 공간을 비우며, 비상 정지 장치를 손이 닿는 위치에 둔다.

# 2.1 Isaac Sim 작업 환경 이해하기

## 2.1.1 Stage, simulation, rendering은 같은 것이 아니다

Isaac Sim 장면을 구성할 때 세 층을 구분하면 문제를 빠르게 진단할 수 있다.

- **USD Stage**: Prim과 Attribute로 저장되는 장면 명세다. 파일로 저장할 수 있는 선언적 상태이다.
- **Physics simulation**: Stage의 물리 schema를 읽어 PhysX가 시간에 따른 상태를 계산한다.
- **Rendering**: Stage의 geometry, material, light, camera를 읽어 픽셀을 생성한다.

Stage에 큐브가 보인다고 해서 그 큐브가 중력의 영향을 받는 것은 아니다. USD Prim에는 물리 기능이 기본으로 활성화되지 않으며, rigid body와 collision API를 별도로 적용해야 한다. 반대로 viewport에서 숨긴 충돌 형상도 collision API가 유지되면 물리 계산에 참여할 수 있다. 이 분리는 Isaac Sim Physics Simulation Fundamentals의 기본 원칙이다.

## 2.1.2 GUI, Script Editor, Standalone Python

Isaac Sim은 여러 개발 방식을 제공한다. 공식 Development Methods는 GUI/Extension과 Standalone Python의 차이를 설명한다.

| 방식 | 적합한 작업 | 시간 제어 | 주요 위험 |
|---|---|---|---|
| GUI | 첫 장면 구성, 속성 탐색, 육안 디버깅 | Play 버튼 중심 | 수작업 변경이 기록되지 않으면 재현하기 어렵다 |
| Script Editor | 실행 중 Stage 수정, 짧은 실험 | 비동기 실행 고려 | physics/render step과 callback 순서를 놓치기 쉽다 |
| Standalone Python | 반복 실험, 데이터 생성, headless 실행 | 코드가 step을 직접 제어 | 앱 초기화 전에 Omniverse 모듈을 import하면 실패한다 |

Standalone 스크립트에서는 `SimulationApp`을 가장 먼저 생성한 뒤 `pxr`, `omni` 모듈을 import한다. `code/create_physx_scene.py`가 이 패턴을 사용한다. API는 릴리스에 따라 모듈 경로가 바뀔 수 있으므로, 책의 예제를 그대로 복사하기 전에 설치된 릴리스의 SimulationApp API와 release notes를 확인한다.

## 2.1.3 재현 가능한 Stage 규칙

팀 프로젝트에서는 다음 규칙을 권장한다.

1. `/World/PhysicsScene`, `/World/Robot`, `/World/Environment`, `/World/Objects`, `/World/Sensors`, `/World/Lights`, `/World/Materials`처럼 역할별 namespace를 정한다.
2. Stage의 up axis와 meters-per-unit을 명시한다. 이 책은 Z-up, 1 unit = 1 m를 사용한다.
3. asset 파일은 reference로 연결하고 실험별 변경은 별도 layer에 override한다.
4. 랜덤화 seed, physics step, render step, solver 설정을 실험 메타데이터에 기록한다.
5. 시각 재질과 물리 재질의 Prim 경로를 분리한다.

# 2.2 좌표계와 4×4 동차 변환

## 2.2.1 pose의 의미부터 고정한다

3차원 pose는 위치(position)와 방향(orientation)의 쌍이다. 회전행렬 ${}^{A}R_{B}$는 B 좌표계의 벡터를 A 좌표계에서 표현하도록 바꾸고, 변위 ${}^{A}t_{B}$는 A 원점에서 본 B 원점 위치다. 두 값을 합치면 다음 동차 변환을 얻는다.

$$ {}^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}t_{B} \\ 0\,0\,0 & 1 \end{bmatrix} $$

B 좌표계 점 ${}^{B}p$를 동차 좌표 $[x, y, z, 1]^{T}$로 만들면 A 좌표계 표현은 다음과 같다.

$$ {}^{A}p = {}^{A}T_{B}\,{}^{B}p $$

변환 체인의 곱셈 순서는 오른쪽부터 적용된다.

$$ {}^{W}T_{O} = {}^{W}T_{T}\,{}^{T}T_{O} $$

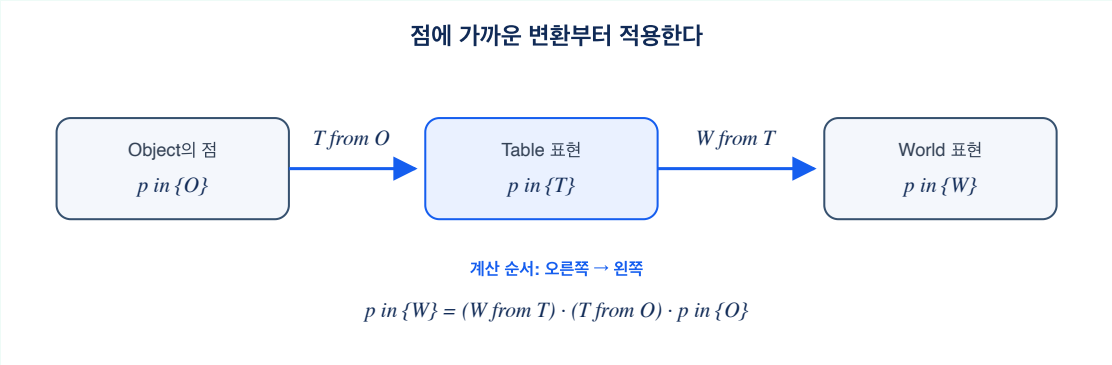
여기서 W는 world, T는 table, O는 object 좌표계다. `code/transforms.py`의 `Transform.compose()`가 이 계산을 구현한다.

역변환은 일반 4×4 역행렬을 구하지 않고 회전행렬의 정규 직교 성질을 이용해 안정적으로 계산한다.

$$ ({}^{A}T_{B})^{-1} = {}^{B}T_{A} = \begin{bmatrix} R^{T} & -R^{T}t \\ 0 & 1 \end{bmatrix} $$

**이해 도움말 — 변환 이름을 문장처럼 읽기**

${}^{A}T_{B}$를 "B에서 A로 좌표를 표현하는 변환"이라고 읽는다. 점 ${}^{B}p$ 오른쪽에 가장 가까운 변환부터 적용되므로 ${}^{W}T_{T}\,{}^{T}T_{O}\,{}^{O}p$는 "Object의 점을 Table로, 다시 World로" 옮긴다. 첨자를 지워도 되는지 확인하려면 인접한 $T$와 ${}^{T}$, $O$와 ${}^{O}$가 연결되는지 본다. 코드 변수도 `world_from_table`처럼 같은 방향으로 읽히게 만들면 역변환 실수를 줄일 수 있다.




변환식을 오른쪽부터 읽는 방법

**그림 2-2.** 점에 가까운 변환부터 적용하고, 인접 프레임 이름이 이어지는지 확인한다.

## 2.2.2 active rotation과 passive rotation

같은 행렬이 두 문맥에서 사용되기 때문에 혼동이 생긴다.

- **Active rotation**: 한 좌표계 안에서 물체 또는 벡터를 회전한다.

- **Passive rotation**: 같은 물리 벡터를 다른 좌표계 성분으로 다시 표현한다.

두 연산은 서로 전치 관계다. API 문서가 `rotate vector`, `transform pose`, `express in frame` 중 어떤 의미를 쓰는지 확인해야 한다. 코드 리뷰에서는 변수명을 `world_from_camera`, `camera_from_object`처럼 읽을 수 있게 정한다. `T_wc`처럼 짧은 이름만 쓰면 첨자의 방향을 팀원이 다르게 해석할 수 있다.

### 2.2.3 Euler angle의 convention

Euler angle은 roll, pitch, yaw 세 값으로 방향을 설명해 사람이 이해하기 쉽다. 그러나 다음 정보를 함께 기록하지 않으면 값만으로 회전을 재현할 수 없다.

- 축 순서: XYZ, ZYX 등
- intrinsic 회전인지 extrinsic 회전인지
- right-handed/left-handed 좌표계
- radian/degree 단위

이 장의 예제는 오른손 좌표계에서 고정 world 축에 대한 extrinsic Z-Y-X 합성, 즉 $R = R_z(yaw)R_y(pitch)R_x(roll)$을 사용한다. pitch가 ±90° 근처가 되면 서로 다른 두 회전축이 정렬되어 자유도 하나를 구분할 수 없게 된다. 이것이 짐벌 락(gimbal lock)이다. 실제 물체가 자유도를 잃는 것이 아니라 Euler 파라미터화가 특이해지는 것이다.

### 2.2.4 quaternion과 순서 문제

단위 쿼터니언은 네 성분으로 3차원 회전을 표현한다.

$$q = (x, y, z, w), \qquad x^2 + y^2 + z^2 + w^2 = 1$$

쿼터니언은 Euler angle처럼 특정 자세에서 자유도를 잃지 않고, spherical linear interpolation(SLERP)을 사용할 수 있다. 그러나 다음 함정이 있다.

1. 라이브러리마다 `(x,y,z,w)`와 `(w,x,y,z)` 순서가 다르다.
2. $q$와 $-q$는 같은 회전을 나타낸다.
3. 수치 적분 후 norm이 1에서 벗어날 수 있어 정규화가 필요하다.
4. 두 쿼터니언의 선형 보간은 일정 각속도 회전을 보장하지 않는다.

`code/transforms.py`는 외부 인터페이스를 `(x,y,z,w)`로 고정하고 입력을 정규화한다. 실제 Isaac Sim/ROS/OpenCV 경계에서는 각 API의 순서를 어댑터 함수 하나로 격리하는 것이 좋다.

# 2.3 강체 역학과 시뮬레이션 시간

## 2.3.1 rigid, static, kinematic

강체는 변형을 무시하고 질량 중심의 병진 운동과 회전 운동만 계산하는 모델이다. 일반적인 actor 유형은 다음과 같다.

- **Static collider**: 작업대, 벽처럼 움직이지 않는 환경이다. 질량 상태를 적분하지 않는다.
- **Dynamic rigid body**: 중력, 힘, 토크, 접촉 충격량의 영향을 받아 움직인다.
- **Kinematic body**: 사용자가 pose를 지정해 움직이며, 다른 물체에 영향을 줄 수 있지만 일반적인 힘 적분으로 움직이지 않는다.

동적 물체를 매 frame 강제로 teleport하면 접촉 solver가 현실적인 힘을 계산하기 어렵다. 목표 pose를 따라가야 하는 장치는 joint drive, force/torque 또는 kinematic target 중 목적에 맞는 방법을 선택한다.

## 2.3.2 이산 시간 적분

시뮬레이터는 연속 시간 방정식을 step 크기 $\Delta t$로 나눠 계산한다. 가장 단순화한 병진 운동은 다음과 같다.

$$v_{k+1} = v_k + \Delta t\, M^{-1} f_k, \qquad x_{k+1} = x_k + \Delta t\, v_{k+1}$$

step이 작을수록 빠른 접촉을 포착하기 쉽지만 계산량이 증가한다. render rate와 physics rate는 같을 필요가 없다. 예를 들어 physics를 120 Hz, render를 30 Hz로 두면 render frame 하나 동안 네 번 물리를 계산한다. 학습 데이터에는 physics timestamp와 sensor timestamp를 따로 기록해야 한다.

## 2.3.3 solver iteration과 안정성

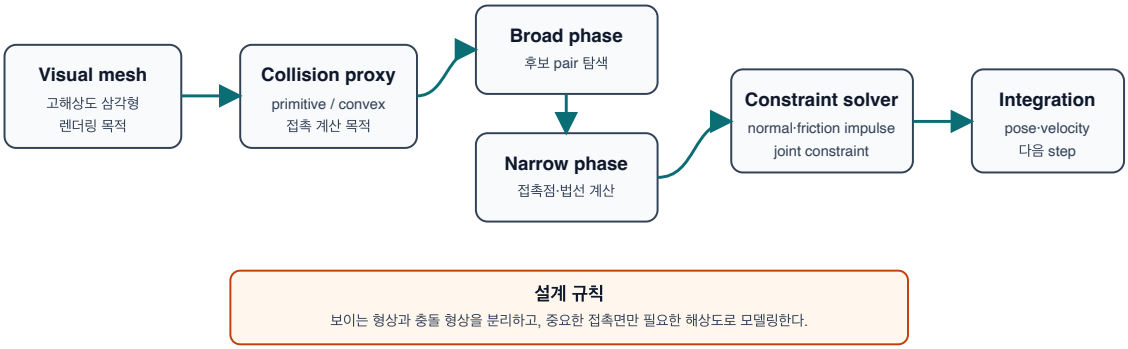
접촉과 joint는 이상적으로 정확한 힘을 직접 구하기보다 제한된 반복 횟수 안에서 constraint를 만족하도록 푼다. 관절이 많은 로봇, 질량비가 큰 물체, 작은 timestep에서 지나치게 큰 stiffness는 solver를 어렵게 만든다. 무조건 iteration을 높이기 전에 다음을 확인한다.

- 충돌 geometry가 지나치게 복잡하지 않은가?
- mass ratio가 비현실적으로 크지 않은가?
- contact offset이 장면 scale과 맞는가?
- 매 frame pose를 순간 이동시키고 있지 않은가?
- controller gain이 timestep에 비해 너무 큰가?

PhysX의 Rigid Body Dynamics는 mass property, force/torque, friction, solver 설정을 설명한다.

> **잠깐 점검 — 더 정밀하게 계산하면 항상 해결될까?**
>
> 1. 빠른 물체가 얇은 벽을 통과할 때 먼저 줄여 볼 값은 physics step $\Delta t$인가, render frame 간격인가?
> 2. 물체가 접촉 순간 폭발하듯 튈 때 solver iteration만 높이기 전에 확인할 항목 하나를 고르라.
>
> **확인 포인트:** 충돌 판정은 physics step마다 일어나므로 먼저 $\Delta t$와 CCD를 검토한다. 불안정 접촉은 비현실적인 질량비, 과도한 stiffness/controller gain, 복잡한 collision geometry, 순간이동 중 하나가 원인일 수 있다.

# 2.4 충돌 형상 설계




충돌 처리 파이프라인

**그림 2-3.** visual mesh와 collision proxy를 분리하고 broad phase, narrow phase, solver를 거쳐 상태를 갱신하는 흐름.

## 2.4.1 visual mesh와 collision mesh

visual mesh는 픽셀 품질을 위해 세밀한 곡면과 작은 부품을 포함한다. collision mesh는 접촉 계산을 위해 단순하고 닫힌 형상을 선호한다. visual mesh를 그대로 충돌에 사용하면 triangle 수뿐 아니라 접촉점과 friction patch 수가 증가해 solver 비용과 jitter가 커질 수 있다.

## 2.4.2 collision approximation 선택

| 근사 | 장점 | 단점 | 권장 대상 |
|---|---|---|---|
| Box/Sphere/Capsule | 매우 빠르고 안정적 | 형상 오차가 큼 | 작업대, 손잡이, 단순 물체 |
| Convex hull | 안정적이고 동적 물체에 적합 | 오목한 홈을 메움 | 블록, 병, 단순 공구 |
| Convex decomposition | 오목한 형상을 여러 convex part로 근사 | cooking 시간과 part 수 비용 | 그리퍼, 트레이, 복잡한 동적 소품 |
| Triangle mesh | 원형상에 가까움 | 접촉 비용과 안정성 부담 | 주로 정적 환경 |

PhysX geometry의 종류와 cooking 요구사항은 PhysX Geometry에 정리되어 있다. 동적 강체는 가능한 한 primitive 또는 convex 계열을 사용하고, 정적 환경에만 triangle mesh를 제한적으로 사용한다.

## 2.4.3 convex decomposition 튜닝

V-HACD 계열 분해는 오목한 메시를 여러 convex hull로 바꾼다. hull 수를 늘리면 원형상에 가까워지지만 항상 좋은 것은 아니다. 다음 지표를 함께 본다.

- 파지에 중요한 홈이나 테두리가 보존되는가?
- hull 수와 hull당 vertex 수가 예산 이내인가?
- 접촉 시 jitter가 줄었는가?

- 한 physics step의 평균/상위 99% 시간이 허용 범위인가?

중요한 접촉면은 별도 primitive collider로 직접 모델링하는 편이 자동 분해보다 예측 가능하다. 예를 들어 그리퍼 고무 패드는 얇은 box collider, 손가락 본체는 capsule/box 조합으로 만든다.

### 2.4.4 broad phase와 narrow phase

충돌 검사는 보통 두 단계로 생각할 수 있다.

1. **Broad phase**: AABB 등 싼 표현으로 충돌 가능성이 있는 pair를 찾는다.
2. **Narrow phase**: 실제 geometry를 사용해 접촉점, 법선, penetration을 계산한다.

scene에 collider가 많으면 broad phase pair 수가 증가한다. 로봇 링크끼리 절대 접촉할 필요가 없는 조합은 collision filtering으로 제외한다. 하지만 실제 self-collision 가능성이 있는 pair까지 제거하면 안전성이 과대평가된다.

# 2.5 Tunneling과 CCD

## 2.5.1 왜 얇은 물체를 통과하는가

이산 충돌 검출(discrete collision detection)은 각 step의 위치에서 겹침을 검사한다. 반지름 $r$인 작은 물체가 한 step 동안 $v\Delta t$만큼 이동하고 그 거리가 물체 두께보다 크면, 이전 pose와 다음 pose 모두에서 충돌하지 않은 채 장애물을 건너뛸 수 있다. 이것이 tunneling이다.

다음 조건은 위험 신호다.

$$v\Delta t > \alpha d_{min}$$

여기서 $d_{min}$은 가장 얇은 충돌 형상의 특성 길이, $\alpha$는 보수 계수다. 이 식은 PhysX 내부 판정식이 아니라 설계자가 CCD 필요성을 판단하기 위한 휴리스틱이다.

## 2.5.2 CCD 적용 전략

PhysX/Omniverse는 sweep 기반 CCD와 speculative CCD를 제공한다. 공식 Omniverse PhysX Rigid Bodies는 sweep 기반 방식이 선형 이동 경로를 검사하고, speculative 방식이 motion에 따라 contact offset을 조정한다고 설명한다. sweep 기반 CCD는 scene과 body 양쪽에서 활성화해야 한다. speculative 방식은 빠른 각운동에도 유용하지만 ghost contact 가능성을 평가해야 한다.

권장 순서는 다음과 같다.

1. 실제 scale과 physics timestep을 먼저 확인한다.
2. collider가 지나치게 얇다면 물리적으로 허용되는 범위에서 두께를 조정한다.
3. 고속 또는 얇은 **필요한 body에만** CCD를 켠다.
4. 같은 initial state와 seed로 penetration rate와 step time을 비교한다.
5. CCD를 켰다고 collision filter나 contact offset 문제를 감추지 않는다.

CCD는 무료가 아니다. 빠른 body가 많을수록 검사 비용이 커지고 일부 경로는 CPU에서 처리된다. 또한 sweep 기반 CCD는 주로 선형 운동을 고려하므로 빠르게 회전하는 긴 물체에는 speculative CCD 또는 작은 timestep이 더 적합할 수 있다.

# 2.6 물리 재질과 접촉 역학

## 2.6.1 Coulomb friction의 단순 모델

접촉면의 법선력 크기를 $N$이라 할 때 단순 Coulomb 모델은 다음과 같다.

$$\| F_t \| \le \mu_s N \quad \text{(정지)}, \qquad \| F_t \| = \mu_d N \quad \text{(미끄러짐)}$$

$\mu_s$는 정마찰계수(static friction), $\mu_d$는 동마찰계수(dynamic friction)다. PhysX는 개별 점 하나가 아니라 contact patch와 friction anchor를 사용해 충격량 기반으로 제약을 푼다. 따라서 교과서 식은 방향과 규모를 이해하는 모델이며 solver의 정확한 내부 결과와 같다고 가정하면 안 된다.

마찰계수는 재질 pair의 속성이다. 고무-플라스틱, 고무-금속은 서로 다른 값을 갖는다. 두 material의 값을 결합하는 combine mode도 결과에 영향을 준다. 프로젝트에서 `rubber_mu=0.9`처럼 단일 숫자만 기록하지 말고 두 표면, 측정 방법, 습도/오염 상태를 기록한다.

## 2.6.2 기울기 실험으로 정마찰 추정

물체가 경사면에서 막 미끄러지기 시작하는 각도를 $\theta_c$라 하면 단순 모델에서

$$\mu_s \approx \tan\theta_c$$

이다. `code/contact_estimator.py`가 이 계산을 수행한다. 최소 5회 반복해 평균과 분산을 기록하고, 표면을 닦은 조건과 그대로 둔 조건을 구분한다. 실제 그리퍼에는 압력 분포, 패드 변형, 곡면 접촉이 추가되므로 이 값은 초기값이지 정답이 아니다.

## 2.6.3 반발계수

충돌 법선 방향 상대속도를 기준으로 반발계수(coefficient of restitution) $e$를 단순화하면

$$e = -\frac{v_{after}}{v_{before}}$$

이다. 공기 저항과 회전을 무시한 낙하 실험에서는 $e \approx \sqrt{h_r / h_d}$ 로 추정할 수 있다. pick-and-place 대상체는 보통 큰 bounce가 필요하지 않으므로 낮은 값을 시작점으로 삼되, 임의의 범위를 정답처럼 고정하지 않는다. 바닥 재질과 대상체 재질을 실제 조합으로 측정한다.

## 2.6.4 physics material과 render material

- **Physics material**: friction, restitution, density 등 접촉 계산에 사용된다.
- **Render material**: base color, roughness, metallic, normal 등 픽셀 생성에 사용된다.

둘은 서로 독립적이다. 반짝이는 금속처럼 보이게 만들었다고 마찰이 자동으로 낮아지지 않는다. Isaac Sim의 material API 중 일부는 experimental로 표시되어 있어 변경될 수 있다. 안정성이 중요한 asset pipeline에서는 `UsdPhysics.MaterialAPI`와 명시적인 binding을 사용하고, high-level wrapper를 쓸 때는 설치 버전 문서를 확인한다.

# 2.7 질량, 무게중심, 관성 텐서

## 2.7.1 질량만 맞추면 충분하지 않다

동적 강체의 상태 응답에는 다음 세 속성이 모두 필요하다.

- 질량 $m$
- 무게중심/질량중심(center of mass, COM) 위치
- COM 기준 관성 텐서 $I$

병진 운동은 $F = ma$, 회전 운동은 단순한 주축 조건에서 $\tau = I\alpha$로 설명된다. 질량이 같아도 질량이 중심에 모였는지 바깥에 퍼졌는지에 따라 각가속도가 다르다. COM이 실제보다 높으면 물체가 쉽게 넘어지고, 낮으면 지나치게 안정적으로 보인다.

## 2.7.2 직육면체 관성

질량 $m$, 변 길이 $a, b, c$인 균일 직육면체의 중심/주축 기준 관성은 다음과 같다.

$$I_{xx} = \frac{m}{12}(b^2 + c^2), \quad I_{yy} = \frac{m}{12}(a^2 + c^2), \quad I_{zz} = \frac{m}{12}(a^2 + b^2)$$

`code/inertia_properties.py`는 이 식과 평행축 정리를 구현한다.

$$I_P = I_C + m(\parallel d \parallel^2 E - dd^T)$$

여기서 $d$는 COM에서 새 기준점까지의 벡터다. CAD가 제공하는 관성 텐서의 단위와 좌표축이 USD asset과 같은지 반드시 확인한다. mm 기반 CAD 관성값을 m 기반 Stage에 그대로 넣으면 큰 오차가 난다.

## 2.7.3 density와 mass authoring

geometry와 density로 mass/inertia를 자동 계산하면 asset 수정에 따라 값이 함께 바뀌어 편리하다. 그러나 빈 용기, 내부 배터리, 비균질 물체는 균일 density 가정과 맞지 않는다. 이때는 실측 mass, COM, inertia를 명시한다. PhysX 문서는 mass와 inertia를 별개 속성으로 다루므로 mass만 바꾸고 inertia를 그대로 두지 않도록 한다. PxRigidBody API도 mass 설정만으로 inertia가 갱신되지 않는 경우를 명시한다.

# 2.8 PBR 재질과 조명

## 2.8.1 물리 기반 렌더링의 목적

PBR은 광원, 표면 재질, 카메라의 관계를 일관된 에너지 모델로 계산해 다양한 조명에서도 그럴듯한 영상을 만든다. 로봇 데이터 생성에서는 단순히 아름다운 이미지를 만드는 것이 아니라 다음을 재현하는 것이 목표다.

- 대상체와 배경의 명암 대비
- specular highlight와 반사
- 카메라 노출과 white balance 변화
- 그림자 경계와 occlusion
- 표면 거칠기와 작은 형상 변화

## 2.8.2 핵심 파라미터

- **Base color/albedo**: 조명 효과를 제외한 표면 색의 근사
- **Roughness**: 미세 표면 법선 분포의 폭; 높을수록 highlight가 넓고 흐리다
- **Metallic**: 금속/비금속 workflow를 구분하는 파라미터
- **Normal map**: geometry를 늘리지 않고 미세 법선 변화를 표현

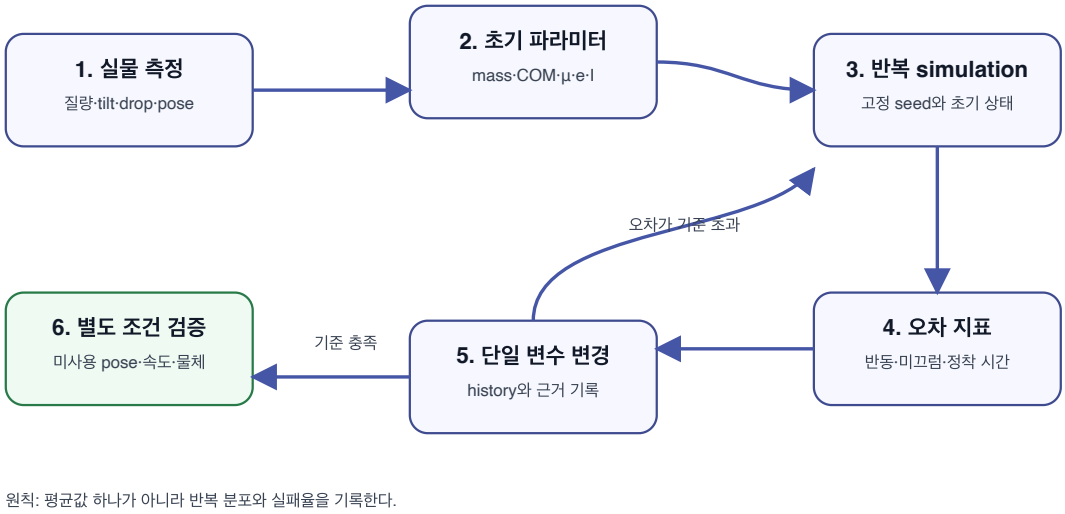
roughness와 friction은 이름이 비슷한 직관을 주지만 서로 다른 모델의 변수다. 시각적으로 거친 표면이 항상 높은 물리 마찰을 갖는 것은 아니다.

## 2.8.3 Dome, distant, area light

- **Dome light**: HDR 환경맵 또는 전방위 배경광을 제공한다.
- **Distant light**: 태양처럼 먼 방향광을 근사한다.
- **Rect/Disk/Sphere light**: 가까운 면광원/점광원에 가까운 효과를 낸다.

실험에서 광원 종류, intensity, color temperature, pose를 기록한다. 카메라 auto exposure를 켜면 조명 변화가 픽셀 변화로 직접 나타나지 않을 수 있으므로 데이터 생성 목적에 따라 exposure를 고정하거나 메타데이터에 기록한다.

# 2.9 물리 파라미터 튜닝 절차




물리 파라미터 튜닝 루프

**그림 2-4.** 실측, 단일 파라미터 변경, 반복 실험, 오차 평가를 분리한 튜닝 루프.

## 2.9.1 관찰량부터 정의한다

파라미터를 바꾸기 전에 비교할 관찰량(observable)을 정한다.

| 현상 | 관찰량 예 | 우선 점검 파라미터 |
|---|---|---|
| 낙하 | 충돌 시각, 최대 반동 높이, 정착 시간 | scale, gravity, restitution, timestep |
| 미끄러짐 | 임계 경사각, 정지 거리 | friction pair, combine mode |
| 넘어짐 | 임계 외력, 각속도 | COM, inertia, contact geometry |
| 파지 | slip rate, 필요한 finger force | finger/object friction, pad geometry, mass |
| 고속 관통 | penetration count, 최소 거리 | timestep, collider thickness, CCD |

## 2.9.2 한 번에 하나씩 바꾼다

재현 가능한 초기 상태와 random seed를 고정한 뒤 한 파라미터만 바꾼다. grid search를 하더라도 실험 ID와 전체 configuration을 저장한다. 손으로 GUI 값을 바꾼 뒤 기억에 의존하지 않는다.

## 2.9.3 권장 체크리스트

1. Stage 단위와 geometry scale을 확인한다.
2. collision proxy를 시각화해 gap/penetration을 확인한다.

3. mass와 COM을 실측값에 맞춘다.
4. inertia를 계산하거나 CAD 값으로 입력한다.
5. restitution을 낙하 실험으로 맞춘다.
6. friction을 tilt/pull 실험으로 맞춘다.
7. physics timestep과 solver 설정을 조정한다.
8. 필요 body에만 CCD를 적용한다.
9. 여러 초기 pose와 속도에서 검증한다.
10. 평균뿐 아니라 실패율과 worst case를 기록한다.

## 2.10 단계별 통합 실습

### 실습 A: 변환 체인 검증

1. `code/transforms.py`를 실행한다.
2. 출력된 `world_from_object`의 translation을 손 계산과 비교한다.
3. `world_from_object.inverse().compose(world_from_object)`가 identity에 가까운지 확인하는 코드를 추가한다.
4. pitch를 89°, 90°, 91°로 바꾸고 rotation matrix와 quaternion을 비교한다.
5. quaternion 순서를 `(w,x,y,z)`로 잘못 전달했을 때 결과가 어떻게 달라지는지 기록한다.

상세 활동지는 실습 2-A: 좌표 변환 체인과 쿼터니언을 사용한다.

### 실습 B: 충돌 형상과 CCD 비교

1. 얇은 platform과 작은 고속 cube를 만든다.
2. CCD를 끈 상태에서 초기 높이와 속도를 바꾸며 tunneling 횟수를 센다.
3. physics timestep을 절반으로 줄이고 다시 측정한다.
4. sweep CCD, speculative CCD를 각각 적용한다.
5. penetration rate와 frame time을 표로 비교한다.

상세 활동지는 실습 2-B: 충돌 근사와 CCD 비용-효과 분석을 사용한다.

### 실습 C: 마찰과 반발 보정

1. 실제 재질 pair의 임계 경사각을 반복 측정한다.
2. `code/contact_estimator.py`로 초기 friction과 restitution을 구한다.
3. Isaac Sim physics material에 값을 입력한다.
4. 동일한 tilt/drop scenario를 실행해 오차를 측정한다.
5. 평균 오차뿐 아니라 trial별 분산을 기록한다.

상세 활동지는 실습 2-C: 마찰·반발·질량 파라미터 보정을 사용한다.

## 2.11 예상 출력 예시

아래 값은 코드와 환경에 대한 **예상 출력 예시**다. 부동소수점 연산, 라이브러리 버전, solver 설정에 따라 실제 결과가 달라도 된다.

`python3 code/transforms.py`

```
world_from_object =
[[ 0.8529 -0.5     0.1504  0.9665]
 [ 0.4924  0.866   0.0868  0.0116]
 [-0.1736  0.      0.9848  0.79  ]
 [ 0.      0.      0.      1.    ]]
quaternion xyzw = [-0.0226  0.0842  0.2578  0.9623]
round-trip max error = 1.11e-16
```

`python3 code/inertia_properties.py`

```
COM inertia [kg m^2]:
 [[0.0004929 0.        0.       ]
  [0.        0.0006067 0.       ]
  [0.        0.        0.0002596]]
grasp-point inertia [kg m^2]:
 [[0.0017529 0.        0.       ]
  [0.        0.0018667 0.       ]
  [0.        0.        0.0002596]]
kinetic energy: 0.008213 J
```

`python3 code/contact_estimator.py`

```
estimated static friction: 0.625
estimated restitution: 0.100
minimum normal force per finger (SF=2): 3.923 N
```

## 2.12 트러블슈팅

### 물체가 바닥을 통과한다

- 바닥과 물체에 collision API가 모두 적용됐는가?
- dynamic body가 visual mesh parent와 다른 Prim에 있지는 않은가?
- Stage scale과 collider thickness가 올바른가?
- 빠른 물체라면 timestep을 줄이거나 CCD를 적용했는가?
- collision filtering이 pair를 제외하지 않았는가?

### 물체가 폭발하듯 튄다

- 초기 pose에서 collider가 깊게 겹쳐 있지 않은가?
- mass ratio, restitution, contact offset이 비현실적이지 않은가?
- 매 frame pose를 강제로 덮어쓰고 있지 않은가?
- controller stiffness와 timestep 조합이 지나치게 크지 않은가?

### 그리퍼 안의 물체가 계속 미끄러진다

- physics material이 visual material과 혼동되지 않았는가?
- 그리퍼 pad와 object 양쪽 material 및 combine mode를 확인했는가?
- finger normal force가 물체 중량과 가속도에 충분한가?
- collision proxy가 visual pad보다 안쪽/바깥쪽으로 어긋나지 않았는가?
- 실물의 패드 변형을 rigid contact 하나로 과도하게 단순화하지 않았는가?

### 동일한 설정인데 실행마다 결과가 다르다

- GPU/CPU pipeline, thread scheduling, solver 설정이 같은가?
- 초기 조건을 physics step 경계에서 적용했는가?
- random seed와 asset version을 기록했는가?
- 결과가 chaotic contact에 민감하다면 평균뿐 아니라 분포로 비교했는가?

### 색은 맞는데 합성 영상이 실물과 다르다

- base color만 맞추고 roughness/normal/exposure를 무시하지 않았는가?
- light 크기와 방향 때문에 그림자 경계가 다른가?
- 카메라 intrinsics, tone mapping, white balance가 같은가?
- HDRI에 학습 label과 상관된 배경 정보가 들어가 있지 않은가?

## 2.13 요약

- USD Stage, physics simulation, rendering은 서로 연결되지만 동일한 계층이 아니다.
- 변환은 `parent_from_child` 의미와 좌표축, 단위, quaternion 순서를 함께 기록해야 한다.
- Euler angle은 사람이 읽기 쉽지만 convention과 singularity가 있고, quaternion은 정규화와 성분 순서 관리가 필요하다.
- visual mesh와 collision proxy를 분리하면 성능과 접촉 안정성을 함께 개선할 수 있다.
- CCD는 tunneling에 효과적이지만 모든 body에 무조건 켜는 기능이 아니다.
- friction과 restitution은 재질 pair 및 측정 조건에 의존하며, 단순 식은 초기값 산정 도구다.
- mass, COM, inertia를 함께 맞춰야 병진과 회전 응답을 재현할 수 있다.
- PBR 속성과 physics material 속성은 독립적이다.
- 튜닝은 관찰량 정의, 단일 변수 변경, 반복 측정, 오차 평가의 순서로 수행한다.

## 2.14 확인문제

1. Stage에서 보이는 mesh가 중력으로 떨어지지 않는 이유를 세 계층 관점에서 설명하라.
2. ${}^{W}T_{C}$와 ${}^{C}T_{O}$가 있을 때 ${}^{W}T_{O}$를 쓰고 곱셈 순서를 설명하라.
3. `Rz(yaw) Ry(pitch) Rx(roll)`과 `Rx Ry Rz`가 다른 이유를 예로 설명하라.
4. $q$와 $-q$가 같은 회전을 나타내는 성질이 interpolation 구현에 주는 영향을 설명하라.
5. triangle mesh collider를 동적 그리퍼 링크에 그대로 쓰는 것의 단점을 세 가지 쓰라.
6. physics rate 60 Hz에서 속도 3 m/s인 물체가 한 step 동안 이동하는 거리를 계산하라.
7. sweep CCD와 speculative CCD의 차이를 설명하고 빠르게 회전하는 긴 막대에 적합한 대안을 논하라.
8. 정마찰계수가 0.5이고 물체 질량이 0.4 kg일 때, 정지 상태를 유지하기 위한 평행 그리퍼 손가락당 최소 법선력을 단순 모델로 구하라. 안전계수 2를 적용하라.
9. 질량만 변경하고 관성을 그대로 둘 때 회전 응답에 어떤 불일치가 생기는가?
10. roughness를 높였는데 friction이 변하지 않는 이유를 설명하라.

추가 연습문제와 해설 지침은 제2장 연습문제 및 해설 지침에 있다.

## 2.15 프로젝트 과제

### 과제 1: 작업 셀 디지털 트윈

작업대, 트레이, cube/cylinder, 두 개의 광원을 포함한 Stage를 만든다. 다음 산출물을 제출한다.

- Stage hierarchy와 layer 구성 설명
- world/table/object 좌표계 변환표
- visual/collision mesh 분리 근거
- mass/COM/inertia 값과 산출 방법
- physics/render material 표
- 10초 simulation의 frame time과 penetration 관찰 결과

### 과제 2: CCD 비용-효과 분석

물체 크기, 속도, timestep을 독립 변수로 정하고 CCD off/on 실험을 수행한다. tunneling rate, 평균 step time, 99th percentile step time을 비교해 어떤 조건에서 CCD를 적용할지 정책을 작성한다.

### 과제 3: 실측 기반 material calibration

서로 다른 두 재질 pair에 대해 tilt test와 drop test를 수행한다. 실측 분포를 바탕으로 simulation parameter를 정하고, 시뮬레이션/실물 오차가 남는 원인을 rigid Coulomb model의 한계와 연결해 분석한다.

## 2.16 실습·평가·참고자료

이 절 아래에 장별 참고문헌, 단계별 실습, 연습문제와 해설 지침을 한데 모았다.

### 2.16.1 참고문헌


1. NVIDIA, Isaac Sim Requirements, 최신 문서.
2. NVIDIA, Isaac Sim Development Methods, 최신 문서.
3. NVIDIA, Physics Simulation Fundamentals, 최신 문서.
4. NVIDIA, Scene Setup Snippets and Examples, 최신 문서.
5. NVIDIA PhysX, Rigid Body Dynamics, PhysX 5.4 documentation.
6. NVIDIA PhysX, Geometry, PhysX 5.4.1 documentation.
7. NVIDIA Omniverse PhysX, Rigid Bodies, 최신 문서.
8. NVIDIA PhysX, Advanced Collision Detection, PhysX 5.1.2 documentation.
9. OpenUSD, UsdGeomXformable Class Reference, 최신 API 문서.
10. David H. Eberly, *3D Game Engine Design*, 2nd ed., Morgan Kaufmann, 2006.
11. Roy Featherstone, *Rigid Body Dynamics Algorithms*, Springer, 2008.


### 실습 2-A: 좌표 변환 체인과 쿼터니언

**[SIM]** 아래 명령은 저장소 루트가 아니라 `book/02_physx_simulation` 장 루트에서 실행한다.

#### 목표

- 변환 행렬의 곱셈 순서를 검증한다.
- Euler convention과 quaternion 성분 순서를 명시한다.
- round-trip 오차를 수치로 평가한다.

#### 준비

```
cd book/02_physx_simulation
python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy
python code/transforms.py
```

#### 절차

1. `world_from_table`, `table_from_object`의 translation을 그림으로 그린다.
2. `world_from_object = world_from_table.compose(table_from_object)` 결과를 기록한다.
3. 다음 검사를 코드에 추가한다.

```
identity = world_from_object.inverse().compose(world_from_object).matrix()
print(np.max(np.abs(identity - np.eye(4))))
```

4. `table_from_object.compose(world_from_table)`처럼 순서를 바꿔 결과가 왜 달라지는지 설명한다.
5. yaw만 0°, 30°, 90°로 바꾸고 object 원점의 world 좌표를 표로 작성한다.
6. pitch를 89°, 90°, 91°로 바꾸고 matrix/quaternion이 연속적으로 변하는지 확인한다.
7. quaternion을 다른 API로 전달한다고 가정하고 `(xyzw) -> (wxyz)` adapter를 작성한다.

**기록표**

| 조건 | world position | quaternion xyzw | round-trip max error |
|---|---|---|---|
| yaw 0° | | | |
| yaw 30° | | | |
| yaw 90° | | | |

**토의**

1. 행렬 원소 오차가 0이 아닌 이유는 무엇인가?
2. Euler angle이 불연속적으로 보이더라도 실제 회전은 연속적일 수 있는 이유는 무엇인가?
3. 좌표계 이름을 변수명에 포함시키면 어떤 종류의 결함을 예방하는가?

**완료 조건**

- 변환 방향과 convention을 보고서에 명시했다.
- inverse/compose round-trip 오차를 수치로 제시했다.
- 잘못된 quaternion 순서가 만드는 오류 사례를 하나 제시했다.

## 실습 2-B: 충돌 근사와 CCD 비용-효과 분석

**[SIM]** Isaac Sim 안에서 수행한다. 프로젝트 경로가 필요한 명령은 `book/02_physx_simulation` 장 루트를 기준으로 한다.

**목표**

- discrete collision의 tunneling 조건을 관찰한다.
- collider, timestep, CCD가 정확도와 성능에 미치는 영향을 비교한다.

**장면**

- Z-up, metersPerUnit = 1
- 두께 0.005 m인 static platform
- 한 변 0.02 m인 dynamic cube 또는 radius 0.01 m sphere
- 초기 높이 0.5 m
- 아래 방향 초기속도: 1, 3, 6, 10 m/s

**독립 변수**

- physics rate: 60, 120, 240 Hz
- collision approximation: primitive, convex hull
- CCD: off, sweep, speculative

### 종속 변수

- platform 아래로 통과한 trial 수 / 전체 trial 수
- 최대 penetration depth
- 평균 physics step time
- 99th percentile physics step time

### 절차

1. CCD off, 60 Hz를 baseline으로 설정한다.
2. 각 속도에서 20회 반복한다. initial horizontal offset에 작은 고정 seed 난수를 준다.
3. physics rate만 바꿔 반복한다.
4. 원래 rate로 돌아가 CCD 방식만 바꾼다.
5. visual mesh는 유지하고 collision approximation만 바꾼다.
6. 결과를 CSV로 저장하고 configuration 전체를 함께 기록한다.

### 안전한 판정 코드 예

```
passed_through = final_z < platform_bottom_z - object_half_extent
```

한 frame만 보지 말고 trajectory 전체의 최소 z와 contact event를 함께 검사한다.

### 결과표

| rate | speed | CCD | collider | tunnel rate | mean step ms | p99 step ms |
|---|---|---|---|---|---|---|
| 60 | 1 | off | primitive | | | |
| 60 | 10 | off | primitive | | | |
| 60 | 10 | sweep | primitive | | | |

### 토의

1. timestep 감소와 CCD 중 어느 방법이 더 일반적인 개선을 제공했는가?
2. 빠른 각운동을 갖는 긴 물체에서 sweep CCD만으로 부족할 수 있는 이유는 무엇인가?
3. CCD를 모든 물체에 적용하지 않을 근거를 결과에서 찾으라.

## 실습 2-C: 마찰·반발·질량 파라미터 보정

**[SIM→HW] [SAFETY]** 실물 경사·낙하 시험은 파손 위험이 없는 시편과 낮은 에너지 조건에서 수행한다. 명령은 `book/02_physx_simulation` 장 루트를 기준으로 한다.

### 목표

- 실측에서 simulation 초기값을 얻는다.
- 한 번에 한 파라미터만 변경하는 보정 절차를 수행한다.
- 평균과 분산으로 Reality Gap을 표현한다.

### 실물 측정

#### Tilt test

1. 대상체와 경사판 표면을 기록하고 사진을 남긴다.
2. 판을 천천히 기울여 물체가 지속적으로 움직이기 시작하는 각도를 측정한다.
3. 최소 10회 반복한다.
4. 평균, 표준편차, 최소/최대값을 구한다.

#### Drop test

1. 안전한 작은 높이에서 물체를 낙하시킨다.
2. high-frame-rate 영상 또는 눈금을 이용해 첫 반동 높이를 구한다.
3. 물체가 손상될 수 있는 높이나 표면은 사용하지 않는다.
4. 최소 10회 반복한다.

### 초기값 계산

```
cd book/02_physx_simulation
python code/contact_estimator.py \
  --tilt-deg 32 \
  --drop-m 0.30 \
  --rebound-m 0.003 \
  --mass-kg 0.25
```

### 시뮬레이션 보정

1. geometry scale, mass, gravity를 먼저 고정한다.
2. restitution만 바꿔 rebound height error를 줄인다.
3. restitution을 고정하고 friction만 바꿔 critical angle error를 줄인다.
4. COM과 inertia를 바꾸기 전 tipping 실험을 별도로 설계한다.
5. 최종 파라미터로 사용하지 않은 pose/속도 조건을 검증한다.

### 오차 지표

$$E_h = |h_{sim} - h_{real}|, \qquad E_\theta = |\theta_{sim} - \theta_{real}|$$

### 제출물

- raw measurement CSV
- 측정 환경과 재질 pair 설명
- 파라미터 변경 history
- calibration 조건과 validation 조건의 오차 비교
- rigid Coulomb contact model로 설명하지 못한 현상 목록

## 제2장 연습문제 및 해설 지침

### 개념 문제

1. render mesh, collision mesh, mass model을 분리하는 이유를 각각 쓰라.
2. static collider와 kinematic body가 같은 점과 다른 점을 설명하라.
3. `metersPerUnit=0.01`인 Stage에 m 단위 관성값을 그대로 넣었을 때 발생할 수 있는 문제를 설명하라.
4. $q$와 $-q$가 같은 회전이라는 사실을 데이터셋 loss 설계에서 어떻게 처리할 수 있는가?
5. CCD를 켜도 tunneling이나 ghost collision이 완전히 사라진다고 보장할 수 없는 이유를 쓰라.

### 계산 문제

1. 120 Hz에서 4 m/s로 움직이는 물체가 한 step 동안 이동하는 거리를 구하라.
2. 0.3 kg, 크기 0.1×0.05×0.04 m 직육면체의 COM 기준 대각 관성 성분을 구하라.
3. 임계 경사각 측정값이 29°, 31°, 30°, 32°, 28°일 때 각 trial의 마찰 추정값과 평균을 구하라.
4. 0.4 m에서 낙하한 물체가 0.016 m 반동했다. 단순 모델의 반발계수를 구하라.
5. 0.5 kg 물체를 마찰계수 0.6인 두 평행 finger로 잡는다. 안전계수 2.5일 때 finger당 최소 법선력을 구하라.

### 구현 문제

1. `Transform`에 `from_quaternion_translation()` classmethod를 추가하라.
2. 두 quaternion 사이의 shortest-path SLERP를 구현하라. dot product가 음수일 때 처리에 주의한다.
3. cylinder와 solid sphere의 관성 텐서 함수를 추가하라.
4. `contact_estimator.py`가 CSV trial을 읽어 평균과 95% bootstrap interval을 출력하도록 확장하라.
5. collision experiment configuration을 YAML로 저장하고 실험 ID의 hash를 만드는 도구를 작성하라.

### 해설 작성 기준

- 숫자 답만 쓰지 말고 단위와 가정을 표시한다.
- quaternion 문제는 성분 순서와 multiplication convention을 명시한다.
- 시뮬레이션 문제는 정확도뿐 아니라 계산 비용과 failure mode를 논의한다.

제3장

# SO-101 기구학과 ROS 2 기반 하드웨어 텔레오퍼레이션

서로의 자세를 따라 하는 Leader와 Follower 로봇팔

*"Leader가 길을 잃으면 Follower는 아주 충실하게 같이 길을 잃는다."*

# 제3장 SO-101 기구학과 ROS 2 기반 하드웨어 텔레오퍼레이션

## 장 개요

로봇팔 제어 소프트웨어는 세 가지 세계를 동시에 다룬다. 첫째는 링크와 관절로 구성된 기하학 모델, 둘째는 엔코더와 버스 서보가 있는 실제 장비, 셋째는 메시지와 시간으로 표현되는 분산 소프트웨어다. 어느 한쪽의 좌표축, 단위, 관절 순서가 어긋나도 가상 로봇은 그럴듯하게 움직이면서 실물은 반대 방향으로 돌 수 있다.

이 장에서는 SO-101을 사례로 기구학 사슬(kinematic chain)을 모델링하고, 순기구학(Forward Kinematics, FK)과 감쇠 최소제곱 역기구학(Damped Least Squares Inverse Kinematics, DLS IK)을 구현한다. 이어 Leader 암의 관절 상태를 Follower 암의 안전한 명령으로 변환하고, ROS 2 Jazzy의 `sensor_msgs/msg/JointState`를 이용해 실물과 시뮬레이터를 연결한다.

SO-101은 자세를 만드는 5개의 팔 관절과 그리퍼 구동기 1개를 가진다. 따라서 모터는 6개지만, 그리퍼를 제외한 팔은 5-DoF이다. 3차원 위치 3개와 방향 3개를 동시에 임의로 지정하려면 일반적으로 6개의 독립 자유도가 필요하므로, SO-101의 역기구학은 위치 우선 또는 방향의 일부만 추종하는 식으로 과업 우선순위(task priority)를 정해야 한다. LeRobot의 공식 Isaac Teleop 설명도 SO-101을 5-DoF 팔로 다루며 방향 오차 가중치를 낮추는 위치 우선 IK를 사용한다.[Hugging Face, 2026]

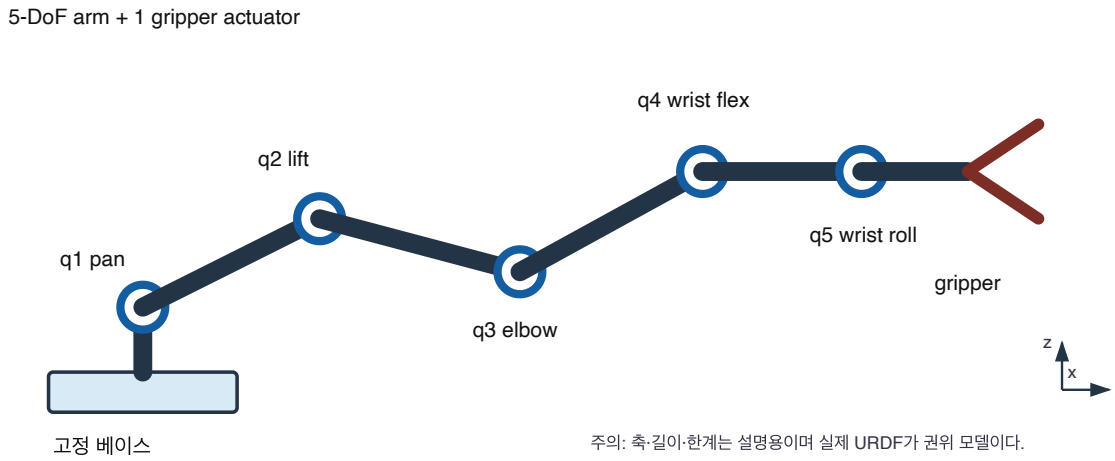



그림 3-1. SO-101의 5개 팔 관절과 그리퍼 구동기

*그림 3-1*은 교육용 개념도다. 정확한 축, 링크 오프셋, 한계는 사용 중인 URDF와 캘리브레이션 파일에서 읽어야 한다.

## 학습 목표

이 장을 마치면 학습자는 다음을 수행할 수 있다.

1. URDF의 `link`, `joint`, `origin`, `axis`, `limit`를 기구학 트리로 해석한다.
2. URDF 모델을 USD 물리 관절(articulation)로 옮길 때 보존해야 하는 정보를 설명한다.
3. 동차변환행렬과 DH(Denavit-Hartenberg) 파라미터로 순기구학을 계산한다.
4. 자코비안(Jacobian)과 DLS를 사용해 특이점 근처에서도 제한된 관절 증분을 계산한다.

5. Leader/Follower 캘리브레이션, 관절 순서, 부호, 단위를 일관되게 관리한다.
6. ROS 2 `JointState` 토픽을 이용해 실물과 시뮬레이터를 동기화한다.
7. 관절 제한, 속도 제한, 워치독(watchdog), 데드맨 스위치(dead-man switch), 비상 정지를 계층적으로 설계한다.

# 선수 지식과 준비물

## 선수 지식

- Python, NumPy 및 C++17 기초
- 4×4 동차변환행렬과 회전행렬
- ROS 2 노드, 토픽, QoS(Quality of Service)의 기본 개념
- Linux 직렬 장치와 사용자 권한의 기본 개념

## 기준 환경

- Ubuntu 24.04 LTS
- ROS 2 Jazzy Jalisco
- Python 3.12, NumPy
- C++17, `rclcpp`, `sensor_msgs`
- LeRobot 최신 안정 릴리스: CLI와 설정 필드는 릴리스에 따라 바뀔 수 있으므로 실행 전 공식 SO-101 문서를 확인한다.
- 선택: SO-101 Leader/Follower, 각 암의 전원 공급 장치와 USB 직렬 어댑터
- 선택: NVIDIA Isaac Sim과 SO-101 URDF/USD 자산

## 실물 실습 전 안전 원칙

> **위험:** 텔레오퍼레이션 코드는 사람의 손동작을 즉시 모터 명령으로 바꾼다. 잘못된 관절 부호, 첫 프레임의 큰 점프, 통신 지연은 충돌과 끼임을 일으킬 수 있다.

실물 전원을 넣기 전에 다음을 확인한다.

1. 암을 단단한 작업대에 고정하고 예상 작업 공간에서 사람, 케이블, 공구를 치운다.
2. 모터와 전원 공급 장치의 정격 전압이 일치하는지 확인한다. 변형별 정격이 다를 수 있다.
3. 하드웨어 전원 차단 수단을 조작자의 손이 닿는 곳에 둔다. 소프트웨어의 `Ctrl+C`는 비상 정지가 아니다.
4. 최초 시험은 토크 비활성 또는 저속·무부하 조건에서 수행한다.
5. 관절별 소프트 리밋(software limit)을 실제 기계 한계보다 안쪽에 둔다.
6. Leader와 Follower의 첫 자세가 크게 다르면 즉시 미러링하지 말고 제한된 속도로 정렬한다.
7. 명령 스트림이 끊기면 현재 자세 유지 또는 토크 해제 중 장비에 맞는 안전 상태로 전이한다.

# 3.1 SO-101 기구학 모델

## 3.1.1 링크, 관절, 구동기의 구분

URDF(Unified Robot Description Format)는 로봇을 링크(link)와 관절(joint)의 트리로 표현한다. SO-101의 일반적인 의미론적 관절 순서는 다음과 같다.

| 순서 | 의미 | 대표 이름 | 역할 |
|---|---|---|---|
| 1 | 베이스 회전 | `shoulder_pan` | 작업 공간의 방위각 변경 |
| 2 | 어깨 굽힘 | `shoulder_lift` | 팔의 높이와 반경 변경 |
| 3 | 팔꿈치 굽힘 | `elbow_flex` | 도달 거리 변경 |
| 4 | 손목 피치 | `wrist_flex` | 그리퍼 접근 각도 변경 |
| 5 | 손목 롤 | `wrist_roll` | 그리퍼 축 주위 회전 |
| 6 | 그리퍼 | `gripper` | 손가락 개폐, 팔의 pose DoF에는 미포함 |

이름은 LeRobot/URDF 버전에 따라 다를 수 있다. 이름을 정수 인덱스로 암묵적으로 치환하지 말고, 항상 메시지의 `name` 배열을 기준으로 재정렬한다. `JointState`의 `position[i]`, `velocity[i]`, `effort[i]`는 같은 인덱스의 `name[i]`에 대응하며, 각 배열은 비어 있을 수 있다. 타임스탬프는 모든 관절 값이 획득된 시각 또는 가능한 한 가까운 시각을 나타내야 한다.[ROS 2 `JointState` 명세]

## 3.1.2 URDF에서 읽어야 할 정보

회전 관절의 핵심 URDF 요소는 다음과 같다.

```
<joint name="elbow_flex" type="revolute">
  <parent link="upper_arm"/>
  <child link="forearm"/>
  <origin xyz="0 0 0.115" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit lower="-1.6" upper="1.6" effort="..." velocity="..."/>
</joint>
```

- `origin`은 부모 링크에서 관절 프레임으로 가는 고정 변환이다.
- `axis`는 관절 프레임에서 표현된 회전축이다.
- `lower`, `upper`는 radian 단위의 위치 한계다.
- `velocity`와 `effort`는 허용 가능한 변화율과 힘/토크 한계를 표현한다.

공식 SO-ARM100 저장소는 새로운 캘리브레이션 모델에서 관절 가상 영점(virtual zero)을 범위 중앙에 두며, 과거 모델은 팔을 수평으로 편 자세를 영점으로 사용한다. 서로 다른 영점 규약의 URDF와 캘리브레이션 JSON을 섞으면 같은 숫자가 다른 실제 자세를 의미한다.[TheRobotStudio, 2026]

### 3.1.3 URDF에서 USD Articulation으로

OpenUSD의 `UsdPhysicsArticulationRootAPI`는 여러 강체와 관절을 축약 좌표(reduced coordinate) 관절계로 묶는다. 회전 관절은 축, 상·하한, 위치/속도 드라이브의 강성(stiffness)과 감쇠(damping)를 가진다. URDF를 Isaac Sim으로 가져올 때 다음을 검토한다.

1. **단위:** URDF 길이는 관례상 m, 각도는 radian이다. USD 스테이지의 `metersPerUnit`, 회전 관절 한계의 단위와 임포터 변환을 확인한다.
2. **축과 부호:** 화면에서 회전 방향만 보고 맞다고 판단하지 말고, 작은 양의 관절 명령으로 축을 검증한다.
3. **루트:** 고정 베이스인지 부유 베이스(floating base)인지 명시한다.
4. **질량과 관성:** 시각 메시만 있는 링크에 임의의 물리 질량이 생성되지 않았는지 확인한다.
5. **충돌 형상:** 시각 메시와 충돌 메시를 분리한다. 복잡한 삼각 메시를 동적 충돌체로 그대로 쓰지 않는다.
6. **드라이브:** 너무 큰 stiffness는 수치적으로 불안정하거나 실물보다 과도하게 단단한 응답을 만든다.
7. **Mimic/그리퍼:** 단일 구동기로 양쪽 손가락을 묶는 관계가 임포트 후 유지되는지 확인한다.

USD Physics 명세는 관절계 루트, 관절, 강체 목록으로 articulation을 구성하며 회전 관절 한계와 드라이브를 명시한다.[OpenUSD, 2026]

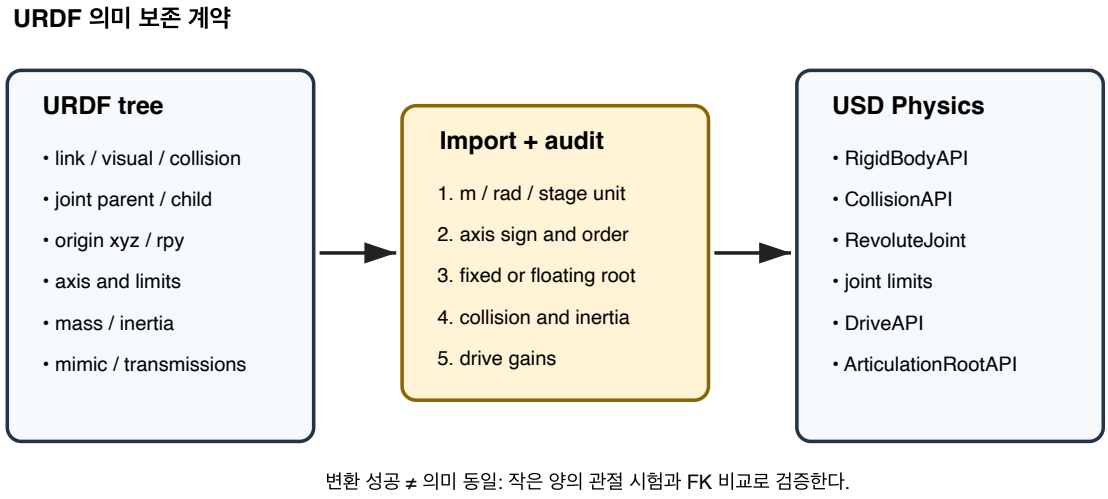



그림 3-2. URDF에서 USD articulation으로 보존해야 할 정보

# 3.2 순기구학

## 3.2.1 동차변환 체인

관절 벡터를 $q = [q_1, \dots, q_n]^T$라 하자. 베이스 프레임 $\{0\}$에서 엔드이펙터 프레임 $\{E\}$까지의 변환은

$$ {}^0T_E(q) = {}^0T_1(q_1)\,{}^1T_2(q_2)\cdots{}^{n-1}T_E(q_n) $$

이다. 변환행렬은

$$ T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, $$

여기서 $R \in SO(3)$는 회전, $p \in \mathbb{R}^3$는 위치다. 행렬 곱은 교환법칙이 성립하지 않으므로 관절 순서가 곧 모델이다.

## 3.2.2 표준 DH 표현

표준 DH 파라미터 $(a_i, \alpha_i, d_i, \theta_i)$의 한 링크 변환은

$$ A_i = R_z(\theta_i)\,T_z(d_i)\,T_x(a_i)\,R_x(\alpha_i) $$

이며, 회전 관절에서는 $\theta_i = q_i + \theta_{0,i}$가 된다. DH는 계산과 교육에 편리하지만, URDF의 임의 축·오프셋을 변환하는 과정에서 실수가 생길 수 있다. 실무에서는 다음 두 모델을 나란히 둔다.

- **권위 모델(authoritative model):** 버전을 고정한 URDF와 캘리브레이션 파일
- **계산 모델(computational model):** DH 또는 Product of Exponentials로 정리한 코드

무작위 관절 자세 여러 개에서 두 모델의 엔드이펙터 위치와 회전을 비교해야 한다. 이 장의 `code/fk_dls_ik.py`는 알고리즘 학습용 5축 DH 예제이며, 실제 SO-101 치수를 주장하지 않는다.

## 3.2.3 기하 자코비안

베이스 좌표계에서 관절 $i$의 회전축을 $z_{i-1}$, 관절 원점을 $o_{i-1}$, 엔드이펙터 원점을 $o_E$라 하면 회전 관절의 자코비안 열은

$$ J_{v,i} = z_{i-1} \times (o_E - o_{i-1}), \qquad J_{\omega,i} = z_{i-1}. $$

따라서

$$ \begin{bmatrix} v_E \\ \omega_E \end{bmatrix} = J(q)\dot{q}. $$

5축 팔의 전체 pose 자코비안은 $6 \times 5$다. 모든 6차원 twist를 독립적으로 만들 수 없으므로 위치만 사용하는 $3 \times 5$ 자코비안이나 방향 오차에 낮은 가중치를 둔 가중 자코비안을 선택한다.

# 3.3 DLS 역기구학

## 3.3.1 선형화와 감쇠

현재 pose에서 작은 관절 변화 $\Delta q$에 대한 작업 공간 변화는

$$\Delta x \approx J(q)\Delta q$$

이다. 단순 의사역행렬은 특이값이 0에 가까울 때 큰 관절 변화를 만들 수 있다. DLS는

$$\min_{\Delta q} \; \| J\Delta q - e \|^2 \; + \lambda^2 \| \Delta q \|^2$$

를 풀어 다음 갱신을 얻는다.

$$\Delta q = J^T(JJ^T + \lambda^2 I)^{-1} e.$$

$\lambda$가 커지면 관절 변화는 작아지지만 목표 추종 오차가 늘어난다. Wampler는 자코비안 기반 IK가 특이점에서 겪는 문제를 DLS 문제로 재정식화해 모든 자세에서 유용한 보정량을 얻는 방법을 제시했다.[Wampler, 1986]

> **이해 도움말 — DLS의 감쇠는 '브레이크'에 가깝다**
>
> 특이점 근처에서는 엔드이펙터를 아주 조금 움직이기 위해 일부 관절을 매우 크게 움직여야 한다. 이때 $\lambda$는 관절 증분이 폭주하지 않도록 제동을 건다. $\lambda$가 작으면 목표를 더 적극적으로 추종하지만 노이즈와 모델 오차에도 민감하고, 크면 움직임은 안정적이지만 목표에 덜 정확하게 접근한다. 따라서 $\lambda$는 "정확도 설정값"이 아니라 **추종 오차와 관절 움직임 사이의 절충(trade-off)**이다.

## 3.3.2 반복 알고리즘

1. 현재 $q_k$에서 FK로 $x_k$를 구한다.
2. 오차 $e_k = x^* - x_k$를 계산한다.
3. $J(q_k)$와 DLS 증분 $\Delta q$를 구한다.
4. $\| \Delta q \|_\infty$ 를 한 스텝 최대값으로 제한한다.
5. $q_{k+1} = \mathrm{clip}(q_k + \Delta q, q_{min}, q_{max})$로 관절 한계를 적용한다.
6. 허용 오차, 최대 반복 수, 진전 부족(stagnation)을 검사한다.

`np.linalg.inv`로 역행렬을 직접 만들지 말고 `np.linalg.solve`를 사용한다. 목표가 작업 공간 밖이면 "가장 가까운 자세"가 성공처럼 보일 수 있으므로 수렴 여부와 최종 오차를 함께 반환한다.

## 3.3.3 5-DoF 과업 설계

SO-101에 6차원 pose 오차를 같은 가중치로 강제하면 서로 양립할 수 없는 목표 때문에 진동할 수 있다. 다음 전략을 선택할 수 있다.

- 위치 3차원만 정확히 추종하고 손목 각도는 별도 명령한다.
- $W = \mathrm{diag}(1, 1, 1, w_r, w_r, w_r)$에서 $0 < w_r \ll 1$로 방향을 부드럽게 추종한다.
- 주 과업으로 위치를 풀고 영공간(null space) 또는 남는 자유도에서 관절 중앙화 비용을 최적화한다.
- 접근 방향 1~2개 축만 제약하고 축 주위 롤은 자유롭게 둔다.

# 3.4 Leader-Follower 캘리브레이션

## 3.4.1 데이터 경로

Leader-Follower 제어는 단순 복사처럼 보이지만 실제 경로는 다음과 같다.

$$q_F^{cmd} = \mathrm{Safe}\big(S\, f_C(r_L) + b\big).$$

- $r_L$: Leader의 원시 엔코더 값
- $f_C$: 캘리브레이션 $C$에 따른 정규화 또는 각도 변환
- $S$: 관절 순서와 방향을 나타내는 부호/순열 변환
- $b$: 기구 조립과 영점 차이를 보정하는 오프셋
- `Safe`: 위치·속도·작업 공간·통신 상태 제한

12-bit 엔코더라면 한 회전을 4096 카운트로 표현할 수 있지만, 실제 유효 범위와 래핑 방식은 서보 설정에 의존한다. “2048이 항상 0 rad”처럼 하드코딩하지 않는다.

## 3.4.2 LeRobot 캘리브레이션 절차

공식 SO-101 안내의 핵심은 Leader와 Follower가 같은 물리 자세일 때 같은 의미의 위치 값을 갖도록 각각 캘리브레이션하는 것이다. 중앙 자세를 기록한 뒤 모든 관절을 전체 안전 범위로 움직여 범위를 수집한다.[Hugging Face SO-101]

```
# 포트 이름은 예시다. 한 번에 한 장치만 연결해 식별한다.
lerobot-find-port

lerobot-setup-motors \
  --robot.type=so101_follower \
  --robot.port=/dev/ttyACM0

lerobot-calibrate \
  --robot.type=so101_follower \
  --robot.port=/dev/ttyACM0 \
  --robot.id=lab_follower_01

lerobot-calibrate \
  --teleop.type=so101_leader \
  --teleop.port=/dev/ttyACM1 \
  --teleop.id=lab_leader_01
```

`id`는 캘리브레이션 파일을 찾는 키이므로 이후 명령에서도 동일하게 사용한다. 캘리브레이션 중에는 기계 스토퍼를 세게 누르지 말고 관절을 안전 범위 안에서 천천히 훑는다.

### 3.4.3 그리퍼 표현의 함정

공식 SO-101 시뮬레이션 설명에서 LeRobot 그리퍼는 0을 완전 닫힘, 100을 완전 열림으로 나타낸다. 그러나 이 매핑이 URDF/MJCF에 그대로 반영되지 않을 수 있다.[TheRobotStudio SO-101 Simulation]

따라서 다음 세 값을 구분한다.

- 원시 서보 tick
- LeRobot 정규화 값(예: 0~100)
- URDF의 회전 또는 선형 관절 위치(rad 또는 m)

그리퍼 변환은 일반 팔 관절의 radian 배열에 섞지 말고 명시적인 함수와 단위로 관리한다.

### 3.4.4 첫 프레임 정렬

Leader가 접힌 상태이고 Follower가 펼쳐진 상태에서 1:1 복사를 시작하면 첫 명령 한 번으로 큰 이동이 발생한다. 안전한 시작 절차는 다음과 같다.

1. 두 암의 현재 상태를 읽는다.
2. 관절 이름, 개수, 유한값, 캘리브레이션 ID를 검증한다.
3. Follower를 Leader의 첫 자세로 제한 속도 보간(slew)한다.
4. 정렬 오차가 임계값 아래일 때만 실시간 미러링을 연다.
5. 데드맨 입력이 놓이면 마지막 명령이 아니라 측정된 현재 자세를 유지한다.

LeRobot의 공식 Isaac Teleop 예제도 기본적으로 Follower를 Leader의 첫 자세까지 일정 시간에 걸쳐 정렬하고, 입력 스트림이 오래되면 측정 자세를 유지하는 계약을 둔다.[Hugging Face Isaac Teleop]

> **잠깐 점검 — Leader 값을 곧바로 복사해도 될까?**
>
> 1. 두 로봇의 관절 이름은 같지만 한 관절의 회전 방향이 반대라면 어느 변환을 확인해야 하는가?
> 2. Leader 메시지가 제한 시간보다 오래되었다면 Follower는 마지막 명령을 계속 실행해야 하는가?
>
> **확인 포인트:** 첫 질문은 부호·순열 변환 $S$와 영점 오프셋 $b$를 확인해야 한다. 두 번째는 아니다. 워치독(watchdog)이 오래된 명령을 거부하고 측정 자세 유지 또는 정지 상태로 전환해야 한다.

## 3.5 안전한 텔레오퍼레이션 루프

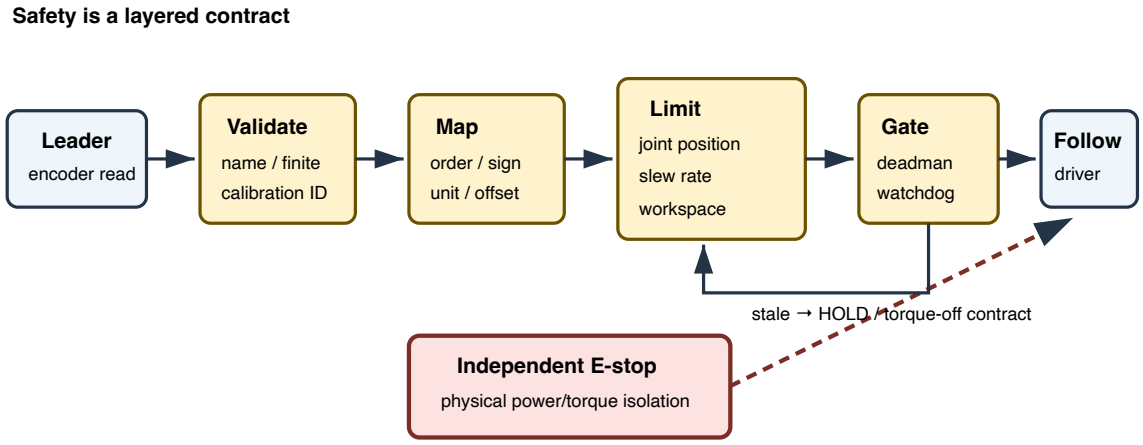



그림 3-3. 안전 계층을 포함한 Leader-Follower 제어 루프

### 3.5.1 안전 계층

안전은 하나의 `if` 문이 아니라 독립적인 방어층으로 구성한다.

| 계층 | 검사 | 실패 시 동작 |
|---|---|---|
| 입력 무결성 | 이름, 길이, NaN/Inf, 타임스탬프 | 프레임 폐기 |
| 캘리브레이션 | 장치 ID, 버전, 관절 순서 | 제어 시작 거부 |
| 위치 제한 | 관절별 soft limit | clamp 및 경고 |
| 속도 제한 | 프레임 간 최대 변화 | slew-rate 제한 |
| 통신 워치독 | 마지막 유효 명령의 나이 | hold 또는 torque off |
| 데드맨 스위치 | 조작자 지속 입력 | hold |
| 독립 하드웨어 | 전원 차단/E-stop | 모터 전력 차단 |

`code/safe_teleop_core.py`는 하드웨어와 분리된 순수 Python 예제다. `JointSafetyFilter`가 이름 기반 재정렬, 위치 clamp, 속도 제한, timeout을 적용한다. `--dry-run`과 같은 단계 없이 실물 송신 함수를 연결하지 않는다.

### 3.5.2 주기와 지연

목표 제어 주기가 50 Hz라면 한 주기는 20 ms다. 전체 지연은 대략

$$T_{loop} = T_{read} + T_{map} + T_{safety} + T_{write} + T_{sleep}$$

이다. 평균뿐 아니라 최악 지연과 jitter를 기록해야 한다. 다음 원칙을 적용한다.

- 단조 시계(monotonic clock)로 timeout을 판단한다.
- 수신 메시지의 ROS time과 로컬 수신 시각을 모두 기록한다.

- 늦은 명령을 한꺼번에 재생하지 않고 가장 최신 유효 상태만 사용한다.
- 통신 실패 후 자동 재개하지 말고 재정렬/재승인을 요구한다.

# 3.6 ROS 2 JointState 브리지

## 3.6.1 토픽 계약

이 장의 예제 토픽은 다음과 같다.

| 토픽 | 타입 | 발행자 | 구독자 | 의미 |
|---|---|---|---|---|
| `/leader/joint_states` | `sensor_msgs/msg/JointState` | Leader 드라이버 | 안전 브리지 | 측정 관절 상태 |
| `/follower/joint_command` | `sensor_msgs/msg/JointState` | 안전 브리지 | 교육용 드라이버 어댑터 | 제한된 목표 위치 |
| `/follower/joint_states` | `sensor_msgs/msg/JointState` | Follower 드라이버 | 시각화/Isaac Sim | 실측 상태 |

표준 `JointState`는 상태 표현이다. 실제 제어기에서는 `trajectory_msgs/msg/JointTrajectory`, `control_msgs/action/FollowJointTrajectory`, `ros2_control` command interface 또는 장치 전용 API가 더 적절할 수 있다. 여기서 명령 토픽에 `JointState`를 쓰는 것은 메시지 정렬과 안전 필터를 학습하기 위한 교육용 계약이며, 생산 시스템의 표준 명령 인터페이스라고 주장하지 않는다.

## 3.6.2 QoS 선택

관절 상태는 최신 값이 중요하므로 작은 큐 깊이와 best-effort를 고려할 수 있다. 반면 명령 손실이 위험한 시스템은 reliable을 선택하되 지연 증가와 backpressure를 측정한다. 시뮬레이터, DDS 구현, 네트워크에 따라 결과가 다르므로 QoS를 하드코딩된 정답이 아니라 인터페이스 계약으로 문서화한다.

`code/ros2_ws/src/so101_training_bridge/`는 두 노드를 제공한다.

- `safe_joint_bridge.py`: Leader 상태를 이름으로 재정렬하고 위치·속도·timeout을 제한해 교육용 명령 토픽으로 발행
- `joint_state_monitor.cpp`: Follower 상태의 관절 누락, 비유한 값, 오래된 타임스탬프를 감시

## 3.6.3 robot_state_publisher와 TF

`robot_state_publisher`는 URDF와 `joint_states`를 사용해 링크 TF를 발행한다. 실물과 시뮬레이터가 같은 TF 트리에 동시에 같은 프레임 이름을 발행하면 충돌하므로 접두사를 둔다.

- 실물: `real/base_link`, `real/gripper_frame_link`
- 시뮬레이터: `sim/base_link`, `sim/gripper_frame_link`

또는 서로 다른 ROS domain/namespace를 사용하고 브리지에서 필요한 토픽만 연결한다. 정적 카메라 외부 파라미터와 동적 관절 TF를 중복 발행하지 않는다.

## 3.6.4 Isaac Sim 미러링

실물 측정값을 가상 articulation에 적용하는 기본 흐름은 다음과 같다.

1. `/follower/joint_states`를 구독한다.
2. 이름 배열을 USD articulation의 DoF 이름으로 매핑한다.

3. radian/degree와 그리퍼 단위를 변환한다.
4. 시뮬레이션 시간에 맞춰 최신 상태를 적용한다.
5. 측정값을 시각화할 때는 물리 드라이브 목표와 순간 상태 설정의 차이를 구분한다.

센서 기반 디지털 트윈은 “완벽한 동시성”이 아니라 타임스탬프가 있는 두 상태의 정렬 문제다. 네트워크 지연을 숨기기 위해 미래를 추정하면 모니터링 화면은 부드러워지지만 실제 측정과 달라질 수 있으므로 추정 상태를 별도 색상/프레임으로 표시한다.

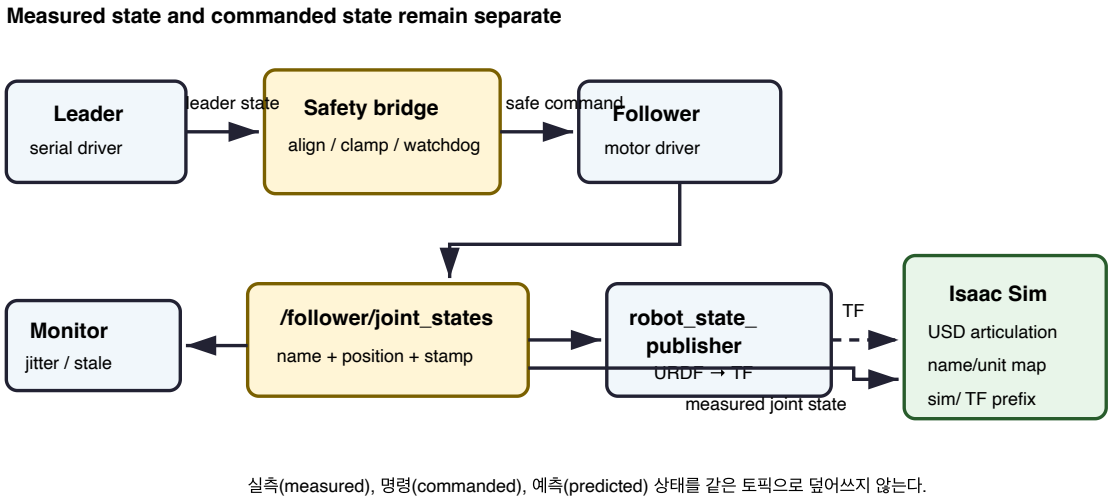



그림 3-4. ROS 2를 통한 실물-시뮬레이터 상태 동기화

## 3.7 단계별 실습

### [SIM] 실습 1: FK와 DLS IK

상세 지시서는 실습 3-1: FK와 DLS IK에 있다.

```
cd book/03_so101_kinematics_teleop
python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy
python code/fk_dls_ik.py
```

**예상 출력 예시**

```
target position [m] : [0.2200 0.0600 0.1800]
converged           : True
iterations          : 7
position error [m]  : 0.000640
solution [deg]      : [ 15.3  10.6  63.0 -43.3   0.0]
```

수치는 NumPy 버전, 초기 자세, 교육용 파라미터에 따라 달라질 수 있다.

### [SIM] [SAFETY] 실습 2: 안전 필터 dry-run

상세 지시서는 실습 3-2: Leader-Follower 안전 필터에 있다.

```
python code/safe_teleop_core.py
```

**예상 출력 예시**

```
frame=0 status=ALIGNING command=[0.01, -0.01, 0.01, -0.01, 0.02, 2.00]
frame=1 status=ALIGNING command=[0.02, -0.02, 0.02, -0.03, 0.04, 4.00]
frame=2 status=ALIGNING command=[0.03, -0.03, 0.04, -0.04, 0.06, 6.00]
frame=3 status=ALIGNING command=[0.04, -0.04, 0.05, -0.06, 0.08, 8.00]
watchdog status=HOLD reason=leader stream stale
```

### [SIM→HW] [SAFETY] 실습 3: ROS 2 브리지

상세 지시서는 실습 3-3: ROS 2 JointState 안전 브리지에 있다.

```
cd code/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 run so101_training_bridge safe_joint_bridge
```

다른 터미널에서 교육용 Leader 상태를 한 번 발행한다.

```
ros2 topic pub --once /leader/joint_states sensor_msgs/msg/JointState \
"{header: {frame_id: 'leader'}, name:
        ['shoulder_pan','shoulder_lift','elbow_flex','wrist_flex','wrist_roll','gripper'],
        position: [0.1,-0.2,0.4,-0.1,0.0,50.0]}"
```

**예상 출력 예시**

```
[INFO] [safe_joint_bridge]: accepted first complete frame; entering TRACKING
[WARN] [safe_joint_bridge]: input timeout; publishing stopped and HOLD requested
```

## 3.8 트러블슈팅

| 증상 | 가능한 원인 | 진단 | 조치 |
|---|---|---|---|
| 한 관절만 반대로 움직임 | `axis` 또는 캘리브레이션 부호 불일치 | 작은 양의 명령, 시뮬레이션/실물 비교 | 매핑 행렬의 해당 부호 수정, 재검증 |
| 시작 순간 크게 튐 | 첫 자세 불일치 | 시작 직전 Leader/Follower 차이 출력 | 제한 속도 정렬 단계 추가 |
| 그리퍼가 반대로 작동 | 0/100 극성 또는 URDF 매핑 혼동 | 명령 단위를 분리해 로그 | 명시적 변환 함수 사용 |
| IK가 특이점에서 진동 | 감쇠 부족, 방향 과제 과구속 | 최소 특이값과 스텝 크기 기록 | $\lambda$ 증가, 위치 우선, step clamp |
| IK가 수렴하지 않음 | 목표가 작업 공간 밖 | 최종 오차와 joint clamp 횟수 확인 | 목표 투영, 실패 상태 반환 |
| `JointState`가 뒤섞임 | 배열 인덱스를 이름 없이 복사 | `name` 배열 비교 | 사전 기반 이름 재정렬 |
| 주기적 timeout | USB/전원/케이블 또는 너무 짧은 timeout | 모터 LED, 포트 로그, 주기 히스토그램 | 배선/전원 확인 후 timeout 재설계 |
| RViz 형상이 튐 | 중복 TF 또는 오래된 stamp | `ros2 run tf2_tools view_frames` | namespace/frame prefix 분리 |
| Isaac Sim 관절이 과도하게 튐 | drive stiffness/damping 또는 단위 오류 | 한 관절씩 저속 시험 | 단위/한계 확인, gain 하향 |

통신 timeout을 무조건 재시도로 덮지 않는다. 공식 LeRobot 안내도 SO 계열 timeout에서 먼저 데이지 체인 LED, 케이블, 전원 변형을 확인하도록 안내한다. 물리 문제를 빠른 재전송으로 악화시키지 않는다.

## 3.9 요약

- SO-101은 5개 팔 관절과 1개 그리퍼 구동기로 이해해야 하며, 5-DoF IK의 과업 우선순위가 필요하다.
- URDF의 링크, 관절축, 원점, 한계와 캘리브레이션 영점은 하나의 버전 계약이다.
- FK는 순서가 있는 변환 체인이고, 자코비안은 관절 속도와 엔드이펙터 twist를 연결한다.
- DLS는 특이점 근처에서 관절 폭주를 줄이지만 정확도와 움직임 크기의 절충을 만든다.
- Leader-Follower 제어에는 이름 매핑, 단위 변환, 첫 자세 정렬, 위치·속도 제한, 워치독, 데드맨, 독립 전원 차단이 필요하다.
- `JointState`는 이름 기반 상태 계약이며 실제 명령 인터페이스는 `ros2_control` 또는 장치 드라이버에 맞게 선택한다.
- 디지털 트윈은 지연 없는 복제가 아니라 타임스탬프와 좌표 프레임이 명시된 상태 정합 시스템이다.

## 3.10 확인문제

1. SO-101의 모터 수가 6인데 팔 pose 자유도를 5라고 하는 이유는 무엇인가?
2. URDF의 `origin`과 `axis`는 각각 어느 프레임에서 어떤 의미를 갖는가?
3. $6 \times 5$ 자코비안으로 임의의 6차원 pose 속도를 만들 수 없는 이유를 rank로 설명하라.
4. DLS의 $\lambda$를 크게 했을 때 장점과 비용은 무엇인가?
5. Leader와 Follower의 첫 자세를 즉시 복사하면 왜 위험한가?
6. `JointState.position`을 배열 인덱스로만 복사하면 안 되는 이유는 무엇인가?
7. ROS time 대신 monotonic clock으로 워치독을 판단해야 하는 이유를 설명하라.
8. 소프트웨어 비상 정지만으로 충분하지 않은 이유는 무엇인가?
9. LeRobot의 그리퍼 0~100과 URDF 관절 값 사이에 어떤 변환 계층이 필요한가?
10. 디지털 트윈 화면을 부드럽게 만들기 위한 예측 상태가 실제 측정과 혼동되지 않게 하는 방법을 제안하라.

해설은 제3장 확인문제 해설에 있다.

# 3.11 프로젝트 과제

## 과제: 안전한 SO-101 Digital Twin Bridge

다음 기능을 가진 ROS 2 패키지를 설계한다.

- Leader/Follower/Sim의 관절 이름을 명시적인 설정 파일로 매핑
- 위치, 속도, timeout 제한
- 첫 자세 정렬 상태 머신: `DISCONNECTED → ALIGNING → TRACKING → HOLD → FAULT`
- 실측 상태와 예측 상태를 분리한 토픽/프레임
- 장비 없이 사용할 수 있는 fake Leader/Follower 노드
- 10분 운전에서 주기, jitter, timeout 횟수를 요약한 보고서
- E-stop 이후 자동 재개 금지와 수동 재승인 절차

평가 기준은 안전한 SO-101 Digital Twin Bridge 프로젝트 평가 기준에 있다.

# 3.12 실습·평가·참고자료

이 절 아래에 장별 참고문헌, 단계별 실습, 확인문제 해설과 프로젝트 평가 기준을 한데 모았다.

## 3.12.1 참고문헌


1. Hugging Face, SO-101 공식 안내, 캘리브레이션 및 텔레오퍼레이션, 2026년 접근.
2. Hugging Face, Isaac Teleop for SO-101, 5-DoF IK, 시작 정렬, stale stream hold, 2026년 접근.
3. Hugging Face, LeRobot SO-101 Isaac Teleop source, 2026년 접근.
4. TheRobotStudio, SO-ARM100/SO-101, 하드웨어와 시뮬레이션 자산, 2026년 접근.
5. TheRobotStudio, SO-101 URDF and MuJoCo Description, 영점 규약과 그리퍼 표현, 2026년 접근.
6. ROS 2, `sensor_msgs/msg/JointState` 명세, Jazzy.
7. ROS 2, `robot_state_publisher` 문서, Jazzy.
8. OpenUSD, USD Physics Schema, articulation 및 joint schema, 2026년 접근.
9. NVIDIA, Isaac Sim URDF Importer, 2026년 접근.
10. J. Denavit and R. S. Hartenberg, "A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices," *Journal of Applied Mechanics*, 22(2), 1955, DOI:10.1115/1.4011045.
11. C. W. Wampler II, "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods," *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1), 1986, DOI:10.1109/TSMC.1986.289285.
12. K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017.


## [SIM] 실습 3-1: FK와 DLS IK

**목표**

- DH 변환 체인과 위치 자코비안을 코드로 확인한다.
- 감쇠, 최대 스텝, 관절 한계가 수렴성에 미치는 영향을 비교한다.
- "수렴하지 않음"을 정상적인 결과 상태로 처리한다.

**준비**

```
cd book/03_so101_kinematics_teleop
python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy
```

**절차**

1. `python code/fk_dls_ik.py`를 실행한다.
2. `target`을 1 cm씩 이동하며 반복 횟수와 최종 오차를 표로 기록한다.
3. `damping`을 0.005, 0.02, 0.08, 0.2로 바꾼다.
4. `max_step_rad` 제한을 제거했을 때 큰 점프가 생기는지 관찰한다. 이 단계는 계산만 수행하며 실물에 연결하지 않는다.
5. 목표를 `[1.0, 0.0, 1.0]` m로 두고 도달 불가능한 목표의 반환값을 확인한다.
6. 관절 중 하나를 상한에 고정하도록 `Q_MIN/Q_MAX`를 임시 수정하고 workspace 변화를 설명한다.

### 예상 관찰

- 작은 감쇠는 빠르게 수렴할 수 있지만 특이점 근처에서 큰 스텝이 발생한다.
- 큰 감쇠는 안정적이지만 느리거나 목표 오차를 남긴다.
- 관절 한계에서 clip이 반복되면 오차가 줄지 않는 정체 상태가 생긴다.

### 보고서 질문

1. 감쇠와 최종 오차의 관계는 단조적인가?
2. `JJ^T`를 푸는 현재 구현과 `J^TJ`를 푸는 구현의 행렬 크기를 비교하라.
3. 실제 SO-101 URDF로 교체할 때 어떤 데이터와 회귀 검사가 필요한가?

## [SIM] [SAFETY] 실습 3-2: Leader-Follower 안전 필터

```
cd book/03_so101_kinematics_teleop
```

### 목표

- 이름 기반 관절 매핑, soft limit, slew-rate limit, watchdog을 실험한다.
- 정상 운전과 HOLD/FAULT 전이를 구분한다.

### 안전 경계

이 실습은 `code/safe_teleop_core.py`의 메모리 내 숫자만 처리한다. 모터 송신 코드를 추가하지 않는다.

### 절차

1. `python code/safe_teleop_core.py`를 실행한다.
2. Leader의 `shoulder_pan` 목표를 상한보다 크게 바꾸고 clamp 결과를 확인한다.
3. 한 관절을 삭제한 dict를 입력해 `FAULT`가 되는지 확인한다.
4. 한 값을 `float("nan")`으로 바꿔 프레임 거부를 확인한다.
5. `deadman=False`로 호출해 측정 자세를 유지하는지 확인한다.
6. timeout을 0.05 s로 줄이고 워치독을 실험한다.
7. `MAX_RATE`를 높였을 때 첫 프레임의 변화량과 위험을 설명한다.

### 설계 확장

- `FAULT` 이후에는 새 프레임만으로 자동 복구되지 않고 `reset_fault()`를 요구하도록 바꾼다.
- 캘리브레이션 ID와 URDF 해시를 입력 계약에 추가한다.
- 관절 clamp가 연속 5회 발생하면 작업 공간 명령을 거부하도록 한다.

### 제출물

- 상태 전이표
- 각 실패 입력에 대한 예상 상태와 실제 상태
- 실물 적용 전 추가해야 할 하드웨어 독립 안전장치 목록

## [SIM→HW] [SAFETY] 실습 3-3: ROS 2 JointState 안전 브리지

### 목표

- ROS 2 Jazzy 패키지를 빌드하고 이름 기반 `JointState` 릴레이를 관찰한다.
- 잘못된 관절 집합, NaN, timeout을 주입해 실패 경로를 확인한다.

### 빌드

```
cd book/03_so101_kinematics_teleop
cd code/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

### 정상 입력

터미널 A:

```
ros2 run so101_training_bridge safe_joint_bridge
```

터미널 B:

```
source /opt/ros/jazzy/setup.bash
ros2 topic echo /follower/joint_command
```

터미널 C:

```
source /opt/ros/jazzy/setup.bash
ros2 topic pub -r 20 /leader/joint_states sensor_msgs/msg/JointState \
"{name: ['wrist_roll','elbow_flex','gripper','shoulder_pan','wrist_flex','shoulder_lift'],
        position: [0.1,0.4,50.0,0.2,-0.1,-0.3]}"
```

출력 배열은 입력 순서가 아니라 `JOINTS`의 계약 순서로 재정렬되어야 한다.

### 실패 주입

1. `gripper`를 이름 배열에서 제거한다.
2. 이름은 6개, 위치는 5개만 보낸다.
3. 정상 발행을 중단하고 0.25 s 이후 timeout 로그를 확인한다.
4. `control_period_s`와 입력 발행률을 다르게 설정하고 추종 지연을 관찰한다.

### 모니터

```
ros2 run so101_training_bridge joint_state_monitor
```

`/follower/joint_states`에 fake 상태를 발행해 유효 프레임과 stale 경고를 확인한다.

### 실물 연결 전 검토

- 출력 토픽을 모터 드라이버에 직접 연결하지 않는다.
- Follower의 실측 초기 자세를 브리지에 공급하도록 인터페이스를 수정한다.
- HOLD 명령과 torque-off의 의미를 드라이버 계약에 정의한다.
- E-stop 복구에는 수동 승인과 재정렬 단계를 둔다.

## 프로젝트 평가 기준: 안전한 SO-101 Digital Twin Bridge

총점 100점이다.

| 항목 | 배점 | 우수 기준 |
|---|---|---|
| 인터페이스 계약 | 15 | 관절 이름, 단위, 프레임, QoS, 시간 기준이 명시됨 |
| 상태 머신 | 15 | 모든 전이 조건과 자동 재개 금지 조건을 시험함 |
| 안전 제한 | 20 | 위치·속도·timeout·NaN·첫 자세 정렬을 독립적으로 처리함 |
| 기구학 | 15 | URDF 기준 FK와 DLS IK를 비교하고 실패 상태를 반환함 |
| 디지털 트윈 | 10 | 측정/예측 상태와 TF namespace가 분리됨 |
| 장비 없는 시험 | 10 | fake 노드로 정상·실패 경로를 재현함 |
| 관측성 | 10 | 주기, jitter, stale, clamp 횟수를 기록함 |
| 문서와 안전 검토 | 5 | E-stop, 전원, 재승인 절차와 알려진 한계를 명시함 |

다음 중 하나라도 있으면 안전 항목 점수는 0점이다.

- 관절 배열을 이름 확인 없이 복사
- timeout 이후 마지막 명령을 무기한 재전송
- E-stop 해제만으로 자동 추종 재개
- 관절 제한을 끄고 실물 시험

## 제3장 확인문제 해설

1. 그리퍼는 개폐 기능을 수행하지만 팔 엔드이펙터의 6차원 pose를 만드는 직렬 관절 자유도에는 포함되지 않는다. 따라서 팔은 5-DoF이고 구동 모터는 6개다.
2. `origin`은 부모 링크 프레임에서 관절 프레임으로 가는 고정 변환이고, `axis`는 관절 프레임에서 표현된 운동 축이다.
3. $6 \times 5$ 행렬의 rank는 최대 5이므로 6차원 출력 공간 전체를 span할 수 없다.
4. 큰 $\lambda$는 특이점에서 관절 증분을 제한하지만 추종 오차가 커지고 수렴이 느려질 수 있다.
5. 한 프레임에 큰 위치 오차를 명령하면 최대 속도/가속도와 충돌 여유를 위반할 수 있다. 제한 속도 정렬이 필요하다.

6. 발행자마다 배열 순서가 다를 수 있다. `name[i]`와 값의 대응을 사용해야 한다.
7. ROS time은 시뮬레이션 정지, 점프, 재설정이 가능하다. 경과 시간 기반 안전 timeout에는 단조 시계가 적합하다.
8. 프로세스 정지, OS 멈춤, 통신 고장 때 소프트웨어 명령이 실행되지 않을 수 있다. 독립적인 하드웨어 전원 차단이 필요하다.
9. 원시 tick↔정규화 개폐율↔URDF rad/m 변환을 명시하고 포화, 극성, 캘리브레이션 버전을 관리해야 한다.
10. 측정과 예측을 서로 다른 토픽, 프레임 또는 표시 스타일로 분리하고, 각각의 타임스탬프와 추정 지연을 표시한다.

제4장

# 멀티모달 센서 정합과 데이터 파이프라인

카메라와 로봇 데이터가 시간축에 동기화되는 모습

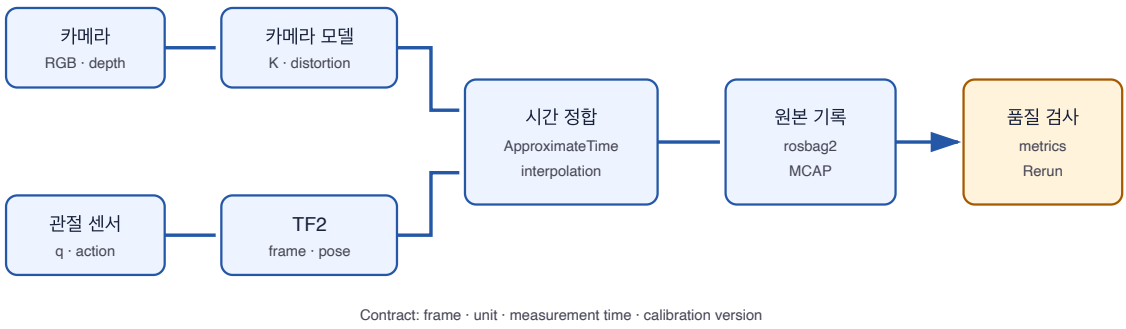
*“33 ms 늦은 프레임도 프레임이다. 다만 같은 순간의 프레임은 아니다.”*

# 제4장 멀티모달 센서 정합과 데이터 파이프라인

## 장 개요

로봇 학습 데이터는 이미지, 관절 상태, 행동 명령, 언어 지시를 한 행에 모았다고 해서 자동으로 올바른 데이터가 되지 않는다. 이미지의 픽셀이 어느 3차원 광선을 뜻하는지, 관절 상태가 어느 시각의 것인지, 카메라가 어느 링크에 붙어 있는지, 시뮬레이션과 실물의 축 정의가 어떻게 다른지를 설명할 수 있어야 한다. 이 장에서는 **공간 정합(spatial alignment)**과 **시간 정합(temporal alignment)**을 하나의 데이터 계약으로 설계한다.

먼저 핀홀 카메라 모델과 내부·외부 파라미터를 유도한다. OpenUSD/Isaac Sim, ROS 2, OpenCV 사이의 좌표 프레임을 명시적으로 변환하고 TF2 트리를 구성한다. 50~100 Hz 관절 스트림과 30 fps 영상의 타임스탬프를 `message_filters`로 근사 동기화하고, 정책 학습용 기준 시각에 관절 상태를 보간한다. 이어 `rosbag2`와 MCAP으로 원본 메시지를 보존하고, Omniverse Replicator로 도메인 랜덤화(domain randomization)를 수행하며, Rerun으로 프레임 드롭·타임스탬프 역전·관절 이상치를 시각적으로 점검한다.




공간과 시간 정합 파이프라인

**그림 4-1.** 카메라 기하, TF2, 타임스탬프, 보간, 기록, 무결성 검사를 분리하면 어느 단계에서 오차가 생겼는지 추적할 수 있다.

## 학습 목표

이 장을 마치면 다음을 수행할 수 있다.

1. 핀홀 카메라의 내부 파라미터 행렬 $K$와 외부 변환 $[R|t]$를 구분한다.
2. 3차원 점을 이미지 평면에 투영하고 깊이로부터 카메라 좌표 점을 복원한다.
3. OpenUSD/Isaac Sim, ROS 2, OpenCV의 축과 쿼터니언 규약 차이를 변환 행렬로 처리한다.
4. `world`, `base_link`, `tool0`, `camera_link`, `camera_optical_frame` TF2 트리를 설계한다.
5. `ApproximateTimeSynchronizer`의 queue와 slop을 센서 주기에 맞게 선택한다.
6. 서로 다른 주기의 시계열을 기준 시각에 보간하고 데이터 누출을 방지한다.
7. rosbag2/MCAP 기록과 재생을 수집 파이프라인의 원본 증거로 활용한다.
8. Replicator 랜덤화 분포를 정의하고 Rerun으로 에피소드 무결성을 검사한다.

## 선수 지식과 준비물

- 3차원 벡터, 행렬 곱, 동차 변환 행렬
- Python 3.12, NumPy, C++17
- Ubuntu 24.04 LTS, ROS 2 Jazzy Jalisco
- 선택: OpenCV, `message_filters`, `tf2_ros`, `rosbag2_storage_mcap`, Rerun SDK, Isaac Sim

> **데이터 보존 원칙**: 동기화·보간된 학습 데이터만 저장하지 않는다. 가능한 경우 원본 메시지와 원본 타임스탬프를 rosbag2에 보존하고, 파생 데이터가 어떤 코드·파라미터로 만들어졌는지 기록한다.

# 4.1 핀홀 카메라와 좌표 프레임

## 4.1.1 투영 모델

카메라 좌표의 점 $\boldsymbol{P}_c = (X_c, Y_c, Z_c)^T$에서 $Z_c > 0$라 하자. 왜곡이 없는 핀홀 카메라에서 정규화 좌표는

$$x_n = \frac{X_c}{Z_c}, \qquad y_n = \frac{Y_c}{Z_c}$$

이고, 픽셀 좌표는

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{K} \begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix}$$

이다. $f_x, f_y$는 픽셀 단위 초점거리, $(c_x, c_y)$는 주점(principal point), $s$는 skew다. 현대 카메라에서는 보통 $s = 0$으로 모델링한다.

월드 점 $\boldsymbol{P}_w$를 카메라로 옮기는 외부 파라미터가 ${}^{c}T_{w}$라면

$$\tilde{\boldsymbol{P}}_c = {}^{c}T_{w}\tilde{\boldsymbol{P}}_w, \qquad {}^{c}T_{w} = \begin{bmatrix} {}^{c}R_{w} & {}^{c}\boldsymbol{t}_{w} \\ 0 & 1 \end{bmatrix}$$

이다. 표기 ${}^{a}T_{b}$는 "프레임 $b$의 좌표를 프레임 $a$로 표현한다"는 뜻으로 이 책 전체에서 사용한다.

내부 파라미터는 카메라 내부 광학·센서 특성을, 외부 파라미터는 카메라가 로봇/세계 어디에 놓였는지를 나타낸다. 해상도나 crop을 바꾸면 $K$도 함께 변할 수 있지만, 카메라가 움직였다고 $K$가 바뀌는 것은 아니다.

> **이해 도움말 — 내부 파라미터와 외부 파라미터를 구분하는 질문**
>
> "이 카메라가 픽셀을 어떤 광선으로 해석하는가?"에 답하는 것이 내부 파라미터(intrinsic parameter) $K$이고, "그 광선이 로봇이나 월드에서 어느 방향을 향하는가?"에 답하는 것이 외부 파라미터(extrinsic parameter) $[R|t]$다. 렌즈·해상도·crop이 바뀌면 먼저 $K$를 의심하고, 카메라 마운트를 다시 조립했거나 충격을 받았다면 $[R|t]$를 다시 측정한다.

## 4.1.2 렌즈 왜곡

실물 렌즈는 핀홀 모델에서 벗어난다. OpenCV의 일반적인 모델은 방사 왜곡 $k_1, k_2, k_3$과 접선 왜곡 $p_1, p_2$를 사용한다. 정규화 좌표에서 $r^2 = x^2 + y^2$라 할 때 대표적인 형태는

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2)$$

$$y_d = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy$$

이다. 정확한 모델과 계수 순서는 사용하는 OpenCV calibration model을 기록한다. 시뮬레이터가 왜곡 없는 영상을 생성해도, 실물 전환 시 동일한 전처리를 만들기 위해 시뮬레이션 영상에 보정 전 왜곡을 합성하거나 실물 영상을 rectify할 수 있다.

## 4.1.3 역투영과 깊이의 의미

픽셀 $(u, v)$와 optical-axis depth $Z_c$가 주어지면

$$\boldsymbol{P}_c = Z_c K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

로 복원한다. 그러나 센서의 depth가 광축 방향 $Z_c$인지, 카메라 중심에서 점까지의 Euclidean range인지 확인해야 한다. 둘은 화면 가장자리에서 다르다.

`code/camera_geometry.py`는 투영, 역투영, 좌표 변환을 NumPy로 구현한다. 행렬의 방향과 depth 정의를 함수 이름과 docstring에 명시했다.

### 4.1.4 캘리브레이션 품질

캘리브레이션의 단일 RMS reprojection error만 보고 품질을 판단하지 않는다.

- 패턴이 화면 전체와 다양한 자세를 덮는가?
- 학습과 검증 이미지가 분리되어 있는가?
- 잔차가 특정 모서리나 방향에 편향되는가?
- 초점·노출·해상도가 운영 조건과 같은가?
- 외부 파라미터는 카메라 마운트 재조립 뒤 다시 측정했는가?

Eye-in-hand 카메라는 `base_link -> tool0 -> camera_link` 체인에 있고, eye-to-hand 카메라는 보통 `world` 또는 `base_link`에 고정된다. 두 경우의 외부 파라미터가 업데이트되는 방식이 다르다.

# 4.2 Isaac Sim/USD, ROS 2, OpenCV 좌표 변환

## 4.2.1 축 이름을 추측하지 않는다

ROS REP 103의 로봇 몸체 관례는 x-forward, y-left, z-up이다. ROS optical frame은 z-forward, x-right, y-down을 사용하며 OpenCV의 카메라 좌표 관례와 맞는다. OpenUSD Stage는 up axis를 메타데이터로 갖고, 카메라와 자산의 로컬 축 의미는 schema와 importer 설정을 함께 확인해야 한다. Isaac Sim 장면이 흔히 Z-up이라고 해서 모든 USD 파일이 Z-up인 것은 아니다.

ROS body frame 벡터를 optical frame으로 바꾸는 한 가지 회전은 다음과 같다.

$$ {}^{opt}R_{body} = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix} $$

즉 $x_{opt} = -y_{body}$, $y_{opt} = -z_{body}$, $z_{opt} = x_{body}$다. 코드에서 축을 swap하는 세 줄로 숨기지 말고 이름 있는 행렬로 표현한다.

## 4.2.2 동차 변환의 방향

두 변환을 합성할 때 오른쪽 변환이 먼저 적용된다.

$$ {}^{world}T_{camera} = {}^{world}T_{base}\,{}^{base}T_{tool}\,{}^{tool}T_{camera} $$

월드 점을 카메라 좌표로 만들려면 이 행렬을 그대로 곱하는 것이 아니라 역변환 ${}^{camera}T_{world} = ({}^{world}T_{camera})^{-1}$을 사용한다. 코드 리뷰에서 변수 이름 `camera_pose`만으로는 방향을 알 수 없으므로 `T_world_camera`처럼 source와 target을 모두 쓴다.

## 4.2.3 쿼터니언과 행렬

ROS `geometry_msgs/Quaternion` 필드는 `(x, y, z, w)`다. 다른 라이브러리는 `(w, x, y, z)`를 생성자 순서로 사용한다. 쿼터니언 $q$와 $-q$는 같은 회전을 뜻하므로, 시계열 비교나 SLERP 전에 내적이 음수면 한쪽 부호를 뒤집어 최단 경로를 택한다.

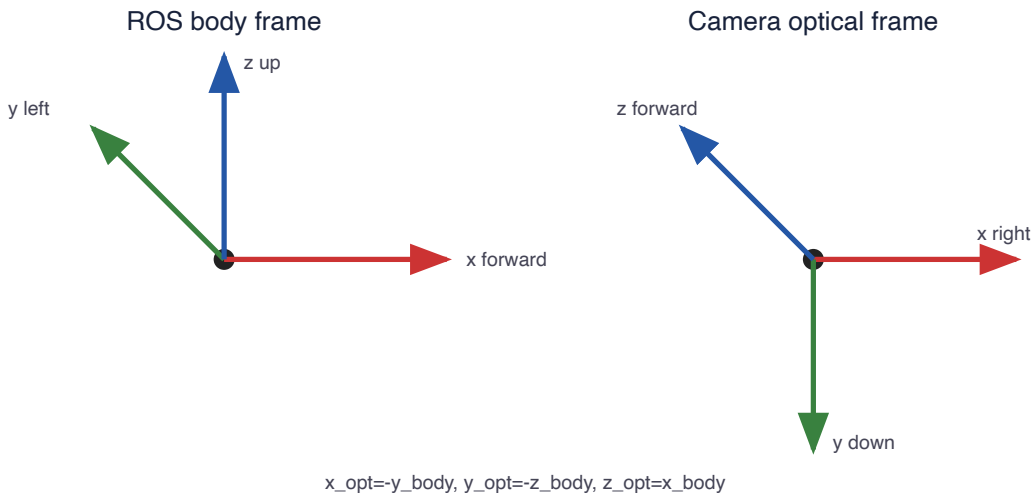
회전 행렬은 다음 조건을 만족해야 한다.

$$ R^T R = I, \qquad \det(R) = +1 $$

축 swap을 직접 작성한 뒤 이 조건을 확인하면 reflection을 회전으로 잘못 사용한 오류를 찾을 수 있다.

## 4.2.4 프레임 계약 표

| 프레임 | 부모 | 축 의미 | 단위 | 소유자 |
|---|---|---|---|---|
| `world` | 없음 | Stage up-axis 명시 | m, rad | simulator |
| `base_link` | `world` | x forward, y left, z up | m, rad | localization/static |
| `tool0` | `base_link` | URDF 정의 | m, rad | robot state publisher |
| `camera_link` | `tool0` | 카메라 기구 프레임 | m, rad | calibration |
| `camera_optical_frame` | `camera_link` | z forward, x right, y down | m, rad | static transform |




좌표 프레임 변환

**그림 4-2.** 기구 프레임과 optical frame을 분리하면 ROS·OpenCV 관례를 유지하면서 마운트 자세를 독립적으로 캘리브레이션할 수 있다.

# 4.3 TF2 트리 설계

## 4.3.1 트리는 하나의 부모를 갖는다

TF2는 시간에 따라 변하는 프레임 관계를 버퍼에 저장한다. 각 child frame은 한 시점에 하나의 parent를 가져야 한다. 같은 `camera_link`를 두 Node가 서로 다른 부모로 broadcast하면 트리가 흔들리거나 조회 결과가 불안정해진다.

- 고정 마운트: `StaticTransformBroadcaster`
- 관절에 따라 변함: `robot_state_publisher` 또는 동적 broadcaster
- 월드 localization: localization/SLAM Node

`code/ros2_jazzy/sensor_pipeline/sensor_pipeline/static_camera_tf.py`는 `tool0 -> camera_link -> camera_optical_frame`의 고정 변환을 방송한다. 실제 값은 캘리브레이션 결과로 교체한다.

## 4.3.2 timestamp가 있는 조회

"최신 transform"으로 모든 과거 이미지를 변환하면 움직이는 로봇에서 공간 오차가 생긴다. 이미지 header timestamp에 해당하는 transform을 조회한다.

```
transform = tf_buffer.lookup_transform(
    "base_link",
    image.header.frame_id,
    image.header.stamp,
    timeout=Duration(seconds=0.05),
)
```

필요한 transform이 아직 버퍼에 없으면 기다리거나 해당 표본을 누락 처리한다. 미래 extrapolation 오류를 피하려고 timestamp를 0으로 바꾸면 데이터는 흐르지만 의미가 틀릴 수 있다.

## 4.3.3 TF 진단

```
ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo base_link camera_optical_frame
```

**예상 출력 예시**

```
At time 1710000000.033
Translation: [0.083, -0.012, 0.115]
Rotation (xyzw): [0.500, -0.500, 0.500, -0.500]
```

숫자만 보지 말고 RViz의 axes와 알려진 물체의 projection으로 방향을 검증한다.

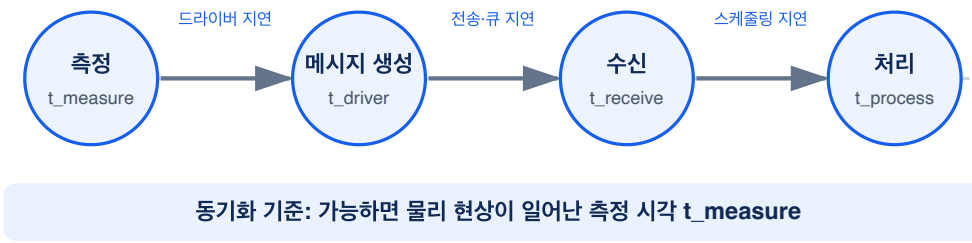
# 4.4 멀티레이트 센서 시간 정합

## 4.4.1 네 개의 시간

하나의 메시지에도 여러 시간이 존재할 수 있다.

1. 센서 노출/측정 시각
2. 드라이버가 메시지를 만든 시각
3. middleware가 수신한 시각
4. 애플리케이션이 처리한 시각

동기화에는 가능한 한 측정 시각을 header에 기록한다. 수신 시각을 대신 쓰면 네트워크 jitter가 물리적인 시간 차이처럼 섞인다. 장치 clock을 호스트 clock으로 변환했다면 offset 추정법과 불확실성을 기록한다.



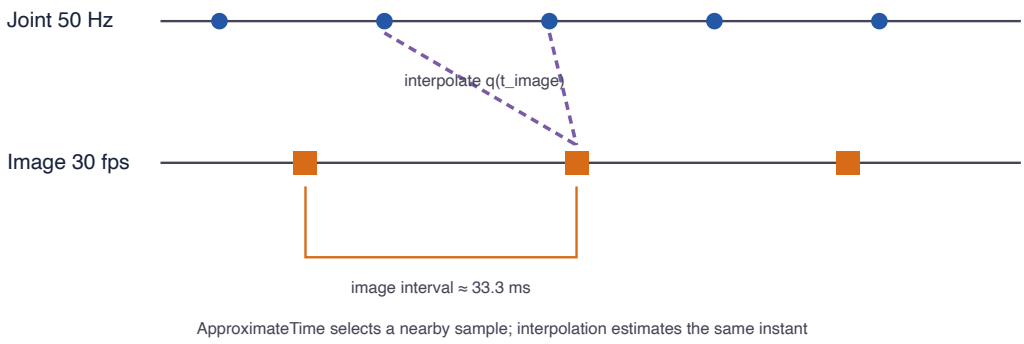

메시지가 거치는 네 가지 시각

**학습 도식.** 측정 시각은 물리 현상이 일어난 때를 나타내고, 나머지 시각의 차이는 드라이버·전송·처리 지연을 진단하는 단서다.

## 4.4.2 ExactTime과 ApproximateTime

서로 다른 센서는 timestamp가 정확히 같지 않으므로 `ApproximateTimeSynchronizer`가 유용하다. 알고리즘은 queue 안에서 timestamp 차이가 허용 범위(slop) 안인 조합을 찾는다. 이는 시간 여행을 보정하는 것이 아니라 가까운 메시지를 짝짓는 것이다.

30 fps 영상의 프레임 간격은 약 33.3 ms이고, 50 Hz 관절 상태는 20 ms다. 예를 들어 slop 15 ms는 가까운 상태를 찾을 가능성이 높지만, 시스템 jitter가 더 크면 pair가 사라진다. slop을 100 ms로 키우면 pair 수는 늘지만 빠른 동작에서 공간 오차가 커진다.




ApproximateTime과 보간

**그림 4-3.** 근사 동기화는 가장 가까운 이산 표본을 고르고, 보간은 이미지 시각 양쪽 관절 표본으로 같은 시각의 상태를 계산한다.

`message_filters`의 모든 Subscriber는 호환되는 QoS를 사용해야 한다. sensor-data publisher가 best-effort인데 subscriber만 reliable이면 연결되지 않을 수 있다. queue size는 최대 지연과 rate로 추정하되, 무한히 키워 오래된 데이터를 쌓지 않는다.

### 4.4.3 선형 보간

이미지 timestamp $t$를 사이에 두는 관절 상태 $(t_0, \boldsymbol{q}_0)$, $(t_1, \boldsymbol{q}_1)$가 있으면

$$\alpha = \frac{t - t_0}{t_1 - t_0}, \qquad \boldsymbol{q}(t) = (1 - \alpha)\boldsymbol{q}_0 + \alpha \boldsymbol{q}_1$$

로 보간한다. 회전 관절이 $-\pi$와 $+\pi$ 경계를 지날 때는 unwrap 후 보간한다. 쿼터니언은 선형 보간보다 SLERP가 적합하다.

실시간 정책에서는 미래 표본 $t_1$을 기다릴 수 없으므로 보간이 지연을 추가한다. 오프라인 데이터셋 정렬에서는 양쪽 표본을 쓸 수 있지만, 학습 입력에 미래 행동이나 미래 관측이 섞이지 않도록 feature timestamp를 엄격히 관리한다.

`code/interpolate_joint_state.cpp`는 관절 timestamp를 단조 증가로 검사하고 두 표본 사이를 선형 보간하는 독립 C++17 예제다.

> **잠깐 점검 — 가까운 표본이면 충분할까?**
>
> 1. 오프라인 데이터셋에서는 이미지 시각 양쪽의 관절 표본을 쓸 수 있지만, 실시간 제어에서 같은 보간을 사용하면 어떤 비용이 생기는가?
> 2. `ApproximateTime`의 slop을 크게 하면 callback 수는 늘어난다. 그런데 빠른 로봇 동작에서는 왜 데이터 품질이 나빠질 수 있는가?
>
> **확인 포인트:** 실시간 양방향 보간은 미래 표본을 기다리는 만큼 지연(latency)을 추가한다. 큰 slop은 서로 다른 물리 시각의 영상과 관절 상태를 한 쌍으로 묶어 공간 정합 오차를 숨길 수 있다.

### 4.4.4 품질 지표

에피소드마다 다음을 계산한다.

- 예상 프레임 대비 실제 프레임 수와 drop rate
- timestamp 차이 $|t_{image} - t_{joint}|$의 p50/p95/max

- timestamp 역전 수
- 관절별 속도·가속도·jerk의 이상치
- TF lookup 실패율
- 영상 decode 실패와 해상도 변화
- 명령과 관측의 지연 분포

단순히 “파일이 열린다”는 데이터 품질 검사가 아니다.

# 4.5 rosbag2와 MCAP 원본 기록

## 4.5.1 왜 원본 메시지를 보존하는가

동기화 정책이나 보정 파라미터는 나중에 바뀐다. 원본 메시지와 timestamp가 있으면 다시 정렬할 수 있지만, 30 fps 배열로 이미 resample한 데이터만 남으면 원래 jitter와 누락을 복원할 수 없다.

Jazzy의 `rosbag2`는 메시지 기록·재생 계층이고 storage plugin을 선택할 수 있다. MCAP은 여러 channel과 schema, index를 담을 수 있는 컨테이너이며 ROS 2 storage plugin으로 사용할 수 있다.

```
sudo apt install ros-jazzy-rosbag2-storage-mcap
ros2 bag record -s mcap -o episode_0001 \
  /camera/color/image_raw /camera/color/camera_info \
  /joint_states /tf /tf_static /command /clock
```

기록 후:

```
ros2 bag info episode_0001
ros2 bag play episode_0001 --clock
```

**예상 출력 예시**

```
Storage id: mcap
Duration: 42.31s
/camera/color/image_raw: 1268 messages
/joint_states: 2115 messages
```

## 4.5.2 에피소드 메타데이터

bag 폴더 바깥에 다음을 함께 저장한다.

- task prompt, 성공/실패, 종료 이유
- 로봇/카메라 serial과 calibration hash
- 소프트웨어 버전과 Git revision
- QoS override와 record 명령
- 시뮬레이션 seed와 randomization 분포
- operator와 안전 이벤트

개인정보나 사람 얼굴이 포함될 수 있으면 수집 전에 보존 기간과 접근 권한을 정한다.

# 4.6 Domain Randomization

## 4.6.1 목적과 한계

도메인 랜덤화는 시뮬레이션의 렌더링·물리 파라미터를 분포에서 표본화해 정책이 특정 시뮬레이션 외형에 과적합하지 않도록 한다. 이 접근은 시뮬레이터가 정확히 현실과 같아야 한다는 요구를 완화하지만, 실제 환경이 학습 분포 밖이면 여전히 실패한다.

랜덤화 범주:

- 조명: 위치, 방향, 색온도, intensity
- 카메라: 작은 pose jitter, focal length, exposure/noise
- 외형: 배경, texture, object color/roughness
- 기하: 물체 초기 pose, 크기 허용오차
- 물리: mass, friction, restitution, actuator delay

안전과 의미를 깨는 랜덤화는 피한다. 예를 들어 색상 지시 작업에서 target 색을 바꾸면서 언어 prompt를 그대로 두면 label이 틀린다. 카메라를 벽 안으로 이동시키거나 물체 질량을 비현실적으로 뽑는 것도 유효한 다양성이 아니다.

## 4.6.2 분포를 버전 관리한다

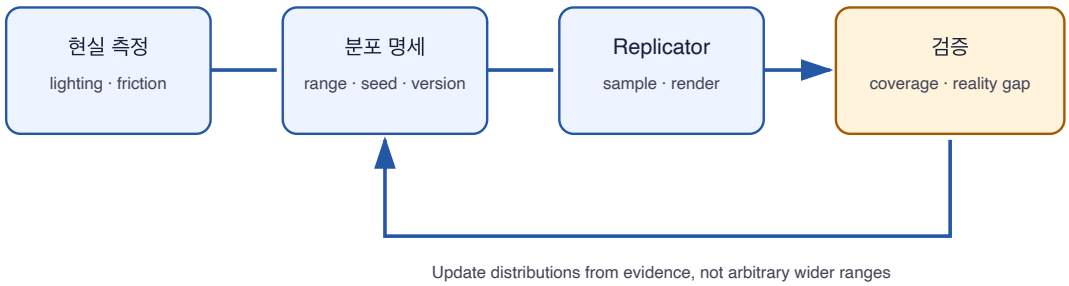
“randomize lighting” 대신 분포를 수치로 기록한다.

```yaml
randomization_version: 3
seed: 4102
camera_translation_jitter_m: [-0.005, 0.005]
camera_rotation_jitter_deg: [-2.0, 2.0]
light_intensity: [800, 3000]
object_hue_shift_deg: [-15, 15]
static_friction: [0.6, 1.1]
```

분포는 train/validation에서 겹치되 validation seed는 분리한다. 실제 측정값이 들어오면 범위를 업데이트하고 버전을 올린다.

## 4.6.3 Replicator 실행 구조

`code/replicator_randomization.py`는 Replicator의 trigger 안에서 camera, light, object pose를 표본화하고 writer를 연결하는 예시다. Isaac Sim 릴리스에 따라 import와 API가 달라질 수 있으므로 설치된 버전의 공식 Replicator 문서와 함께 사용한다.




도메인 랜덤화 설계

**그림 4-4.** 측정한 현실 범위에서 분포를 정의하고 seed와 파라미터를 메타데이터에 남겨야 합성 데이터를 재현할 수 있다.

## 4.7 Rerun 기반 무결성 검사

Rerun은 이미지, scalar, transform, point, time-series를 같은 timeline에 로그해 공간·시간 관계를 탐색하는 도구다. Rerun 자체가 데이터 품질을 보장하지는 않는다. 검사 규칙이 이상치를 계산하고, Rerun은 그것을 사람이 빠르게 해석하도록 돕는다.

`code/rerun_integrity_check.py`는 CSV joint trajectory와 이미지 경로를 읽는 대신, 의존성 없이 생성한 예시 시계열을 Rerun에 기록한다. 다음 entity를 권장한다.

- `world/camera/image`
- `robot/joints/<joint_name>/position`
- `quality/image_joint_delta_ms`
- `quality/frame_drop`
- `quality/timestamp_reversal`
- `episode/task`와 `episode/success`

`code/rerun_integrity_check.py`는 frame 184에 영상 간격 증가, frame 250에 `joint_3` spike, frame 291에 timestamp 역전을 의도적으로 주입하고 해당 품질 entity를 기록한다.

**예상 관찰 예시**

```
frame 184: image interval 67 ms -> one probable drop
frame 250: joint_3 velocity threshold exceeded
frame 291: joint timestamp decreased by 12 ms
```

무결성 검사는 실패를 숨기지 않는다. 품질 기준을 통과하지 못한 에피소드를 삭제하기보다 quarantine 목록에 두고 원인과 재수집 여부를 기록한다.

## 4.8 구현 예제

### [SIM] 4.8.1 카메라 수학

```
python3 code/camera_geometry.py
```

**예상 출력 예시**

```
pixel: [420.0, 290.0]
reconstructed camera point: [0.2, 0.1, 1.0]
round-trip error: 0.000000
det(R_opt_body): 1.000
```

### [SIM] 4.8.2 ApproximateTime ROS 2 Node

`code/ros2_jazzy/sensor_pipeline/`는 Python 패키지로 다음 console script를 제공한다.

- `synthetic_sensors`: 30 Hz Image와 50 Hz JointState를 고정된 jitter/offset과 함께 생성
- `approx_sync`: Image와 JointState를 근사 동기화하고 timestamp delta를 발행
- `static_camera_tf`: camera link와 optical frame의 고정 TF 방송

```
cd code/ros2_jazzy
colcon build --symlink-install --packages-select sensor_pipeline
source install/setup.bash
ros2 run sensor_pipeline synthetic_sensors
```

다른 터미널에서 동기화 Node를 실행한다.

```
cd book/04_sensor_data_pipeline/code/ros2_jazzy
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run sensor_pipeline approx_sync --ros-args -p slop_s:=0.02
```

`synthetic_sensors`는 실물 장비 없이 slop 실험을 재현하기 위한 교육용 입력이다. `image_offset_ms`와 `jitter_ms` 파라미터로 두 stream의 timestamp 차이를 바꿀 수 있다.

### [SIM] 4.8.3 독립 C++ 보간

```
g++ -std=c++17 -O2 code/interpolate_joint_state.cpp -o /tmp/interpolate_joint_state
/tmp/interpolate_joint_state
```

**예상 출력 예시**

```
t=0.033 q=[0.165, -0.066]
reversal rejected: timestamps must be strictly increasing
```

## 4.9 트러블슈팅

| 증상 | 원인 후보 | 진단 | 해결 방향 |
|---|---|---|---|
| 3D 점이 좌우 반전 | body/optical 축 혼동 | 알려진 축 점을 투영 | 명시적 회전 행렬 사용, det 검사 |
| 영상 위 overlay가 속도에 따라 밀림 | 최신 TF 사용, timestamp offset | 정지/동작 잔차 비교 | 이미지 시각 TF 조회, clock offset 보정 |
| ApproximateTime callback이 호출되지 않음 | QoS 비호환, slop 과소, header 없음 | topic info -v, stamp 출력 | QoS 통일, header 측정 시각 제공 |
| callback은 많지만 pose가 부정확 | slop 과대 | delta histogram | slop 축소 또는 보간 사용 |
| bag replay 시 TF extrapolation | `/clock`/use_sim_time 설정, `/tf_static` 누락 | bag topic 확인 | clock 재생, static TF 기록 |
| joint spike | wrap 경계, packet corruption, 잘못된 순서 | 이름 기반 재정렬과 unwrap | 범위·속도 검사, quarantine |
| 합성 데이터 성능만 개선 | 랜덤화 분포가 현실과 다름 | 실제 파라미터 측정 | 분포 보정, photometric/physics 분리 실험 |
| Rerun timeline이 겹치지 않음 | 서로 다른 time column/단위 | ns/s 변환 확인 | 공통 timeline과 명시적 단위 사용 |

## 4.10 요약

- 내부 파라미터는 광선과 픽셀의 관계, 외부 파라미터는 프레임 사이의 자세를 정의한다.
- 좌표 변환은 축 swap이 아니라 방향이 명시된 ${}^{a}T_{b}$로 관리한다.
- ROS body와 optical frame은 축 관례가 다르며, USD Stage의 up axis도 조회해야 한다.
- TF2 조회는 데이터의 측정 timestamp에 수행한다.
- ApproximateTime은 가까운 표본을 짝지을 뿐이며, 큰 slop은 공간 오차를 숨길 수 있다.
- 오프라인 정렬에는 양쪽 관절 표본 보간이 가능하지만 미래 정보 누출을 방지해야 한다.
- rosbag2/MCAP 원본과 에피소드 메타데이터를 보존해야 파이프라인을 재처리할 수 있다.
- 랜덤화 분포와 seed를 버전 관리하고, Rerun으로 공간·시간 이상을 함께 조사한다.

## 4.11 확인문제

1. $K$의 $f_x$가 두 배가 될 때 같은 3D 점의 픽셀 좌표가 어떻게 변하는가?
2. optical-axis depth와 Euclidean range의 차이를 그림과 식으로 설명하라.
3. ${}^{world}T_{camera}$가 있을 때 월드 점을 카메라로 변환하는 식을 쓰라.
4. ROS body frame의 단위 벡터 x, y, z를 optical frame으로 각각 변환하라.
5. 쿼터니언 시계열에서 $q$와 $-q$가 번갈아 저장될 때 단순 차분이 만드는 문제를 설명하라.
6. TF2에서 camera frame에 두 parent가 생기면 안 되는 이유는 무엇인가?
7. 30 fps 영상과 50 Hz 관절 상태에서 slop 5 ms와 50 ms의 trade-off를 비교하라.
8. 실시간 정책과 오프라인 데이터셋에서 보간 방식이 달라지는 이유를 설명하라.
9. rosbag2 원본을 유지하면 calibration이 바뀌었을 때 어떤 작업을 다시 할 수 있는가?
10. 색상 기반 지시 작업에서 무작위 색 변경이 label corruption을 만들 수 있는 사례를 제시하라.
11. timestamp 역전과 frame drop을 각각 검출하는 최소 알고리즘을 작성하라.
12. Rerun 시각화와 자동 품질 기준의 역할 차이를 설명하라.

해설은 제4장 확인문제 해설 및 평가 기준에 있다.

## 4.12 프로젝트 과제

### 과제 A: 멀티모달 에피소드 정합기

카메라 30 fps, 관절 50 Hz, action 20 Hz의 synthetic stream을 만든다. 이미지 timestamp를 기준으로 관절을 보간하고, action은 해당 시각 이전의 최신 명령만 연결한다. 출력에 원본 index, timestamp delta, interpolation flag를 포함한다.

### 과제 B: 데이터 품질 리포트

에피소드별 frame drop, reversal, joint velocity outlier, TF missing, image-joint delta를 계산하고 Rerun entity로 기록한다. 실패 에피소드를 삭제하지 않고 사유가 있는 quarantine manifest를 만든다.

### 평가 기준

| 항목 | 배점 |
|---|---|
| 좌표 프레임과 단위 정확성 | 20 |
| timestamp·동기화·보간 의미 | 25 |
| 누락·역전·이상치 처리 | 20 |
| 재현 가능한 메타데이터 | 15 |
| 시각화와 설명 | 10 |
| 코드 구조와 출처 | 10 |

## 4.13 실습·평가·참고자료

- 실습 4-1: 카메라 투영과 프레임 정합
- 실습 4-2: TF2와 ApproximateTime
- 실습 4-3: rosbag2/MCAP 에피소드 기록
- 실습 4-4: Domain Randomization과 Rerun 품질 검토
- 제4장 확인문제 해설 및 평가 기준

### 4.13.1 참고문헌


1. OpenCV, Camera Calibration and 3D Reconstruction.
2. ROS REP 103, Standard Units of Measure and Coordinate Conventions.
3. ROS 2 Jazzy, tf2 Tutorials.
4. ROS 2 Jazzy, Approximate Time Synchronizer (Python).
5. ROS 2 Jazzy, Quality of Service settings.
6. ROS 2, rosbag2 및 MCAP Storage Plugin.
7. Foxglove, MCAP Specification.
8. NVIDIA Isaac Sim, Replicator Tutorials.
9. NVIDIA Omniverse, Replicator Documentation.
10. Rerun, Python SDK 및 Time.
11. Tobin, J. et al., Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World, 2017.
12. Sadeghi, F., Levine, S., CAD2RL: Real Single-Image Flight without a Single Real Image, 2016.
13. Horaud, R., Dornaika, F., Hand-Eye Calibration, IJRR, 1995.


### [SIM] 실습 4-1: 카메라 투영과 프레임 정합

```
cd book/04_sensor_data_pipeline
```

**목표**

내부 파라미터로 3D 점을 투영하고 ROS body/optical frame 변환을 검증한다.

**절차**

1. `python3 code/camera_geometry.py`를 실행한다.
2. $f_x, f_y$를 20% 변경하고 픽셀 이동을 기록한다.
3. optical depth와 Euclidean range를 같은 값으로 넣었을 때 화면 가장자리의 3D 복원 오차를 계산한다.
4. x, y, z 단위 벡터에 `R_opt_body`를 곱하고 축 의미를 표로 작성한다.
5. 의도적으로 한 축의 부호를 바꾸고 determinant가 -1이 되는지 확인한다.

**제출물**

- 투영/역투영 round-trip 오차

- 좌표 프레임 표
- reflection과 rotation의 차이 설명

## [SIM] 실습 4-2: TF2와 ApproximateTime

```
cd book/04_sensor_data_pipeline
source /opt/ros/jazzy/setup.bash
cd code/ros2_jazzy
colcon build --symlink-install --packages-select sensor_pipeline
source install/setup.bash
```

새 터미널마다 위의 ROS 환경과 `install/setup.bash`를 source한 뒤 다음을 실행한다.

```
# 터미널 A: 30 Hz Image + 50 Hz JointState
ros2 run sensor_pipeline synthetic_sensors

# 터미널 B: 고정 카메라 TF
ros2 run sensor_pipeline static_camera_tf

# 터미널 C: 근사 동기화
ros2 run sensor_pipeline approx_sync --ros-args -p slop_s:=0.015
```

**목표**

카메라 TF 트리를 확인하고 image-joint timestamp 차이를 측정한다.

**절차**

1. `ros2 run sensor_pipeline synthetic_sensors`로 30 Hz Image와 50 Hz JointState를 생성한다.
2. `static_camera_tf`를 실행하고 `tf2_echo`로 변환을 확인한다.
3. 별도 터미널에서 `approx_sync`의 `slop_s`를 0.005, 0.015, 0.050으로 바꾼다.
4. 각 설정의 pair 수, delta p95, 최대 delta를 기록한다.
5. `synthetic_sensors`의 `image_offset_ms`와 `jitter_ms`를 바꾸고 callback 비율을 비교한다.
6. publisher와 subscriber QoS를 일부러 다르게 하고 `ros2 topic info -v`로 진단한다.

**제출물**

- TF tree 그림
- slop별 품질 표
- 선택한 운영 slop과 근거

## [SIM] 실습 4-3: rosbag2/MCAP 에피소드 기록

```
cd book/04_sensor_data_pipeline
```

#### 목표

원본 메시지를 기록하고 metadata와 품질 통계를 분리해 보존한다.

#### 절차

1. 카메라, JointState, TF, command, `/clock`을 MCAP으로 기록한다.
2. `ros2 bag info`에서 topic별 count와 duration을 저장한다.
3. `--clock`으로 재생하며 `use_sim_time` Node의 동작을 확인한다.
4. 카메라 calibration hash, software revision, task prompt, success를 YAML manifest에 작성한다.
5. `/tf_static`을 제외한 bag을 별도로 만들고 재생 시 차이를 관찰한다.

#### 주의

사람 영상이 포함되면 수집 동의, 접근 권한, 보존 기간을 먼저 정한다.

## [SIM] 실습 4-4: Domain Randomization과 Rerun 품질 검토

```
cd book/04_sensor_data_pipeline
```

#### 목표

랜덤화 분포를 버전 관리하고 시계열 이상을 공간·시간 뷰에서 검토한다.

#### 절차

1. 카메라 jitter, light intensity, object color, friction 분포를 YAML로 작성한다.
2. 의미를 깨는 표본과 물리적으로 불가능한 표본을 거부하는 조건을 설계한다.
3. Isaac Sim 환경이 있으면 `replicator_randomization.py`를 설치 버전에 맞게 실행한다.
4. `code/rerun_integrity_check.py`에서 frame 184 drop, frame 250 `joint_3` spike, frame 291 timestamp reversal을 찾는다.
5. 자동 threshold와 사람의 시각 검토가 불일치하는 사례를 기록한다.

#### 제출물

- randomization manifest
- Rerun 캡처 또는 관찰 기록
- quarantine 에피소드 목록과 사유

## 제4장 확인문제 해설 및 평가 기준

1. 주점 기준 픽셀 변위가 두 배가 된다. $u - c_x = f_x X/Z$다.
2. optical depth는 Z 성분이고 range는 $\sqrt{X^2 + Y^2 + Z^2}$ 다. 광축 밖에서는 range가 더 크다.
3. $\tilde{P}_c = ({}^{w}T_c)^{-1}\tilde{P}_w$다.
4. body x→optical z, body y→optical -x, body z→optical -y다.
5. 같은 자세인데 component 차분이 큰 점프로 나타나 각속도와 보간이 잘못된다. 내적 부호를 맞춘다.
6. 한 child의 자세가 동시에 두 정의를 가져 트리 경로와 authority가 모호해진다.
7. 5 ms는 pair 누락이 많지만 시간 오차가 작고, 50 ms는 pair 수는 늘어도 빠른 동작의 공간 오차가 커진다.

8. 오프라인은 미래 표본을 이용한 양방향 보간이 가능하지만 실시간에서는 그 표본을 기다리면 지연이 생긴다.
9. 새 calibration으로 projection, rectify, TF 기반 pose를 다시 계산할 수 있다.
10. “빨간 블록” prompt를 유지한 채 target을 파란색으로 바꾸면 관측과 label이 충돌한다.
11. 인접 timestamp의 차이가 0 이하인지 검사하고, 예상 frame interval의 임계 배수보다 큰지 검사한다.
12. 자동 기준은 일관된 탐지·격리를, Rerun은 여러 신호의 관계를 사람이 해석하는 역할을 한다.

제5장

# LeRobot Dataset과 SmolVLA 파인튜닝

로봇과 작은 신경망 캐릭터가 행동 데이터를 분류하는 모습

*"모델은 데이터에서 배운다. 데이터가 헷갈리면 모델도 자신 있게 헷갈린다."*

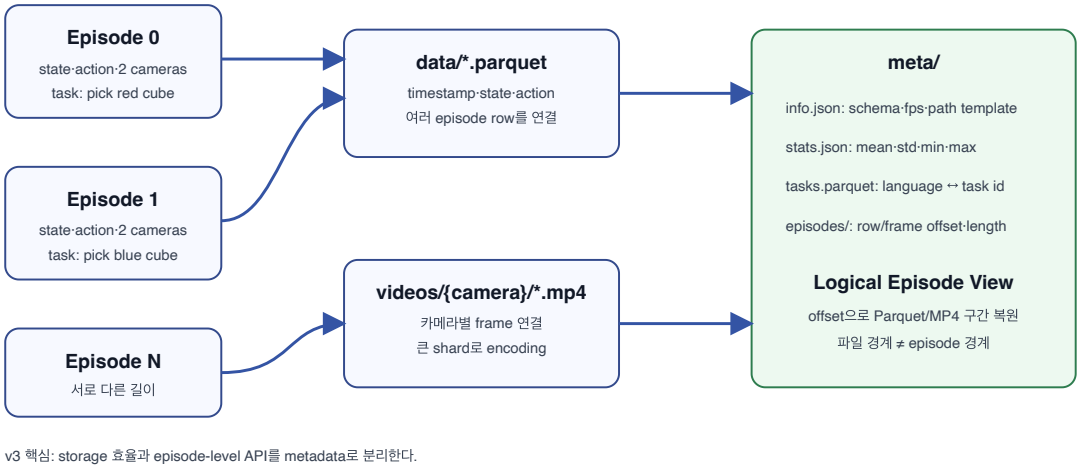
# 제5장 LeRobot Dataset과 SmolVLA 파인튜닝

## 장 개요

로봇을 사람이 직접 조작해 성공 시연(demonstration)을 모으면, 관측에서 행동으로 가는 정책(policy)을 지도학습 형태로 훈련할 수 있다. 그러나 일반 이미지 분류와 달리 로봇 행동은 한 번의 예측이 다음 관측을 바꾸고, 작은 오차가 시간에 따라 누적된다. 데이터 수집, 시계열 정렬, 정규화, 행동 표현, 실시간 추론을 하나의 시스템으로 다뤄야 하는 이유다.

이 장에서는 행동 복제(behavior cloning, BC)의 기본 목적함수와 공변량 이동(covariate shift)을 출발점으로, Action Chunking with Transformers(ACT), Diffusion Policy, Vision-Language-Action(VLA) 모델의 차이를 설명한다. 이어서 Hugging Face LeRobotDataset v3의 Parquet·MP4·메타데이터 구조를 해부하고, SmolVLA를 LoRA(Low-Rank Adaptation)로 미세조정하는 절차를 설계한다. 마지막에는 20 Hz 폐루프(closed-loop) 실행, action chunk, temporal ensembling, 비동기 추론, 안전 필터를 하나의 배포 파이프라인으로 연결한다.

이 장의 버전 기준은 **2026년 10월에 확인한 LeRobot 공식 `main` 문서와 LeRobotDataset v3 계열**이다. LeRobot의 CLI와 configuration key는 빠르게 변할 수 있으므로 실제 실습에서는 설치한 commit과 `lerobot-train --help` 출력을 원고와 함께 보관한다.




LeRobotDataset v3 구조

**그림 5-1.** 여러 episode가 소수의 Parquet/MP4 shard에 합쳐지고, metadata가 episode 경계를 복원하는 LeRobotDataset v3 구조.

## 학습 목표

이 장을 마치면 학습자는 다음을 수행할 수 있다.

1. behavior cloning의 목적함수와 covariate shift에 의한 오류 누적을 설명할 수 있다.
2. 단일 행동 회귀, ACT, Diffusion Policy, VLA 정책의 표현력과 계산 비용을 비교할 수 있다.
3. LeRobotDataset v3의 `data`, `videos`, `meta` 관계와 episode reconstruction 방식을 설명할 수 있다.

4. 카메라·관절·행동·언어 데이터를 일관된 feature schema로 설계할 수 있다.
5. episode 단위 train/validation/test 분할과 training-only normalization을 구현할 수 있다.
6. SmolVLA의 VLM context와 action expert, action chunk 생성을 설명할 수 있다.
7. full fine-tuning과 LoRA의 parameter/memory trade-off를 평가할 수 있다.
8. training configuration, checkpoint, dataset revision을 재현 가능한 형태로 기록할 수 있다.
9. 20 Hz 폐루프에서 chunk 실행, temporal ensembling, inference latency를 다룰 수 있다.
10. 데이터 공개 전 라이선스, 개인정보, 음성·영상·Hub 접근 권한을 점검할 수 있다.

# 선수 지식과 준비물

## 선수 지식

- PyTorch의 tensor, module, optimizer, dataloader
- Transformer의 self-attention과 cross-attention
- 확률분포, 조건부확률, 평균·표준편차
- 제4장의 센서 timestamp 정렬과 episode 수집 구조

## 기준 환경

- Ubuntu 24.04 LTS
- Python 가상환경 또는 uv
- CUDA 호환 NVIDIA GPU(학습 시 권장)
- Hugging Face 계정과 token(비공개 Hub 저장소를 사용할 경우)
- LeRobot repository의 고정 commit 또는 태그

설치 명령은 프로젝트의 현재 문서를 우선한다. SmolVLA 추가 의존성은 공식 문서에서 다음과 같이 안내된다.

```
git clone https://github.com/huggingface/lerobot.git
cd lerobot
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[smolvla,peft]"
git rev-parse HEAD
lerobot-train --help
```

마지막 두 명령의 출력을 실험 기록에 저장한다. `latest` 또는 `main`만 기록하면 같은 실험 환경을 다시 만들 수 없다.

# 5.1 모방학습과 행동 복제

## 5.1.1 demonstration dataset

시연 데이터셋 $D$는 관측 $o_t$, 행동 $a_t$, 필요하면 언어 지시 $l$의 시계열로 구성된다.

$$D = \{\tau_i\}_{i=1}^{N}, \qquad \tau_i = \{(o_t, a_t, l_i)\}_{t=0}^{T_i-1}$$

관측은 한 장의 이미지가 아니라 여러 카메라 영상, 현재 관절 상태, 그리퍼 상태를 포함할 수 있다. 행동은 다음 joint position, joint delta, end-effector pose, velocity 중 하나다. `action`의 의미를 schema와 unit에 명시하지 않으면 같은 shape의 tensor라도 서로 호환되지 않는다.

## 5.1.2 behavior cloning 목적함수

결정론적 정책이 행동을 직접 회귀한다면 가장 단순한 loss는 다음과 같다.

$$\mathcal{L}_{BC}(\theta) = \mathbb{E}_{(o,a)\sim D}\left[\ \| \pi_\theta(o) - a \|_1\ \right]$$

L1 또는 Smooth L1은 큰 이상치에 대한 민감도가 L2보다 낮다. 하지만 loss가 낮다고 task success가 반드시 높지는 않다. 같은 관측에서 왼쪽과 오른쪽으로 우회하는 두 성공 행동이 존재하면 평균 회귀 결과는 장애물 중앙으로 향할 수 있다. 행동 분포의 다봉성(multimodality)이 중요한 이유다.

## 5.1.3 covariate shift와 복구 데이터

훈련 데이터의 관측 분포는 사람 정책 $\pi_E$가 만든다. 배포 중 모델 $\pi_\theta$가 작은 오차를 내면 훈련에서 거의 보지 못한 상태로 이동하고, 그 상태에서 더 큰 오차를 낼 수 있다.

$$o_t \sim d_{\pi_E}\ (\text{training}), \qquad o_t \sim d_{\pi_\theta}\ (\text{deployment})$$

대응 방법은 하나가 아니다.

- 시작 pose, 물체 위치, 조명 변화를 충분히 포함한다.
- 완벽한 시연뿐 아니라 안전한 복구(recovery) 행동을 수집한다.
- 실패 직전/실패 상태를 구분해 annotation한다.
- closed-loop evaluation에서 관측된 실패 분포를 다음 수집 round에 반영한다.
- 필요하면 DAgger 계열처럼 정책 방문 상태에 expert label을 추가한다.

단, 위험 상태를 의도적으로 만들 때는 로봇 속도·토크 제한과 비상 정지 절차가 우선이다.

> **이해 도움말 — 왜 한 번의 작은 오차가 계속 커지는가**
>
> 행동 복제(Behavior Cloning)는 주로 전문가가 만든 “정상 경로”를 본다. 그러나 배포 중 정책이 물체를 몇 mm 빗맞히면 다음 카메라 영상은 훈련 데이터에 드물었던 장면이 된다. 정책은 낯선 장면에서 다시 오차를 내고, 그 결과는 더 낯선 다음 관측을 만든다. 이것이 공변량 이동(covariate shift)의 연쇄다. 따라서 데이터 수집 때는 성공 궤적의 개수만 늘리기보다, 살짝 어긋난 상태에서 정상 경로로 돌아오는 복구(recovery) 예시를 포함하는 것이 중요하다.

# 5.2 행동 정책의 계보

## 5.2.1 단일 step 정책

단일 step 정책은 현재 관측에서 다음 action 하나를 예측한다. latency가 작고 구현이 단순하지만, 사람 시연의 고주파 jitter를 그대로 학습하거나 시간적 일관성을 잃기 쉽다. 긴 작업을 매 step 독립적으로 결정하면 예측 noise가 command noise가 된다.

## 5.2.2 ACT: action sequence를 한 번에 예측

ACT는 Action Chunking with Transformers의 약자다. Zhao 등은 저가형 양팔 로봇의 정밀 조작에서 action sequence를 생성하는 generative model을 제시했다. 한 시점에서 $H$개의 미래 행동을 예측하면 유효 decision horizon이 줄고, 시간적으로 연결된 동작을 모델링할 수 있다.

$$\widehat{A}_t = [\widehat{a}_t, \widehat{a}_{t+1}, \ldots, \widehat{a}_{t+H-1}]$$

ACT는 conditional VAE와 Transformer를 사용해 시연의 변동을 잠재변수로 표현한다. 배포에서는 여러 시점에 예측한 겹치는 chunk를 temporal ensemble로 합칠 수 있다. 원 논문은 Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware다.

## 5.2.3 Diffusion Policy

Diffusion Policy는 noise에서 시작해 observation에 조건부인 denoising을 반복하여 action sequence를 생성한다. 단일 평균값 대신 다봉 행동분포를 표현하기에 유리하고 receding-horizon control과 결합한다. 반면 여러 denoising step은 inference latency와 compute 비용을 만든다. Diffusion Policy는 conditional denoising, visual conditioning, receding horizon의 조합을 제시한다.

## 5.2.4 VLA

Vision-Language-Action 모델은 이미지와 자연어를 context로 사용해 로봇 행동을 생성한다.

$$\pi_\theta(A_t \mid I_t^{1:C}, s_t, l)$$

VLA의 장점은 task를 언어로 조건화하고 사전학습된 시각-언어 표현을 활용할 수 있다는 점이다. 그러나 자연어가 안전 제약을 대신하지 않는다. “빨리 집어라”라는 지시가 joint limit, workspace boundary, collision rule보다 우선해서는 안 된다.

| 정책 | 출력 | 다봉성 | 언어 조건 | 주된 비용 |
|---|---|---|---|---|
| 단일 BC | action 1개 | 낮음 | 선택적 | 낮음 |
| ACT | action chunk | CVAE로 표현 | 보통 별도 설계 | 중간 |
| Diffusion Policy | denoised chunk | 강함 | 조건으로 확장 가능 | 반복 추론 |
| SmolVLA | 언어·시각 조건 action chunk | action expert가 표현 | 기본 구성 | VLM+expert 추론 |

# 5.3 LeRobotDataset v3 설계

Hugging Face의 LeRobotDataset v3 문서는 v3의 핵심을 storage와 user-facing episode API의 분리로 설명한다. v2 계열의 episode당 파일 구조와 달리 v3는 여러 episode를 큰 Parquet/MP4 파일에 묶어 파일 수를 줄이고 metadata로 경계를 복원한다.

## 5.3.1 세 개의 기둥

1. **Tabular data**: state, action, timestamp처럼 저차원·고주파 신호를 Parquet에 저장한다.
2. **Visual data**: 카메라별 frame을 MP4 shard로 저장한다.
3. **Relational metadata**: schema, stats, task, episode 경계, file/row/video offset을 저장한다.

간소화한 구조는 다음과 같다.

```
dataset/
├── meta/
│   ├── info.json
│   ├── stats.json
│   ├── tasks.parquet
│   └── episodes/chunk-000/file-000.parquet
├── data/chunk-000/file-000.parquet
└── videos/
    ├── observation.images.top/chunk-000/file-000.mp4
    └── observation.images.wrist/chunk-000/file-000.mp4
```

문서 revision에 따라 `tasks.jsonl`처럼 이전 이름이 검색될 수 있으므로 경로를 직접 조립하지 말고 설치 버전의 API와 `meta/info.json` path template을 사용한다.

## 5.3.2 feature schema

예시 feature는 다음과 같다.

| key | shape 예 | dtype | 의미 |
|---|---|---|---|
| `observation.state` | `[6]` | float32 | 현재 관절/그리퍼 상태 |
| `action` | `[6]` | float32 | 다음 target joint 또는 delta |
| `observation.images.top` | `[3,H,W]` | video | 외부 카메라 |
| `observation.images.wrist` | `[3,H,W]` | video | 손목 카메라 |
| `timestamp` | scalar | float64 | episode 내 시간 |
| `episode_index` | scalar | int | episode 식별자 |
| `task_index` | scalar | int | 언어 task 연결 |

shape만으로 충분하지 않다. joint order, unit, absolute/delta, camera color order, resize/crop, gripper sign convention을 dataset card에 기록한다.

### 5.3.3 episode 경계와 split

v3에서는 한 Parquet/MP4 shard 안에 여러 episode가 들어갈 수 있다. 파일 이름을 episode ID로 해석하면 안 된다. `meta/episodes`의 row range와 video offset으로 episode를 찾는다.

train/validation/test는 **frame이 아니라 episode 단위**로 분할한다. 같은 episode의 인접 frame이 train과 validation에 나뉘면 거의 동일한 이미지가 양쪽에 들어가 성능이 과대평가된다. 더 엄격한 일반화를 평가하려면 물체 위치, 배경, 작업자, 날짜 같은 group 단위로 분할한다.

### 5.3.4 finalize의 의미

v3 writer는 incremental Parquet writing과 metadata buffer를 사용한다. 수집 종료 후 `finalize()`를 호출해야 buffer를 flush하고 Parquet footer를 완성한다. 프로그램이 비정상 종료되면 최근 episode가 불완전할 수 있다. 공식 Dataset API는 `finalize()`를 생략한 dataset이 유효하지 않을 수 있다고 명시한다.

# 5.4 데이터 수집과 품질 관리

## 5.4.1 다양성과 반복의 균형

SmolVLA 공식 안내는 task variation마다 충분한 시연을 수집할 것을 강조한다. “50 episode”는 출발점 예시이지 보장 수량이 아니다. 다음 축을 설계표로 만든다.

- 물체 위치와 방향
- 물체 색·크기·질량
- 조명, 배경, camera exposure
- grasp approach와 recovery 경로
- 작업자 및 teleoperation 속도

variation 수만 늘리고 각 조건 반복이 한 번뿐이면 모델이 조건별 일관성을 배우기 어렵다. 먼저 핵심 조건을 반복하고 그다음 범위를 넓힌다.

## 5.4.2 성공/실패 label

성공 episode만 모으면 안정적인 목표 행동을 배우기 쉽지만, 작은 오차에서 복구하는 방법을 배우지 못한다. 실패를 포함할 경우 무작정 action target으로 학습하지 말고 다음을 구분한다.

- 성공 시연
- 실패하지만 유용한 복구 구간
- 센서 오류/프레임 드랍으로 사용 불가능한 구간
- 안전 개입으로 종료된 구간

dataset card에 inclusion/exclusion rule을 명시한다.

## 5.4.3 시간 정렬

action $a_t$가 관측 $o_t$에 대한 명령인지, $o_{t+1}$ 결과인지 명확히 한다. 20 Hz command와 30 fps 카메라를 단순 row 번호로 맞추지 않는다. timestamp를 기준으로 nearest/previous/interpolation 정책을 정하고 최대 허용 오차를 기록한다.

## 5.4.4 영상 인코딩

MP4/H.264 같은 inter-frame codec은 저장 효율이 높지만 임의 frame 접근 비용과 seek 정확성에 영향을 줄 수 있다. 다음을 기록한다.

- codec, pixel format, frame rate
- resolution과 resize/crop pipeline
- keyframe interval
- variable/constant frame rate 여부
- 원본 timestamp 보존 방식

압축을 “무손실”이라고 표현하려면 실제 codec setting과 lossless mode를 검증해야 한다. 일반 H.264 설정은 대개 손실 압축이다.

# 5.5 정규화와 통계

## 5.5.1 Z-score와 min-max

feature $x$의 Z-score는 다음과 같다.

$$\tilde{x} = \frac{x - \mu_{train}}{\max(\sigma_{train}, \epsilon)}$$

[-1,1] min-max normalization은 다음과 같다.

$$\tilde{x} = 2\frac{x - x_{min}}{\max(x_{max} - x_{min}, \epsilon)} - 1$$

통계는 train split에서만 계산하고 validation/test에 재사용한다. 전체 dataset 통계를 먼저 계산하면 evaluation distribution 정보가 training pipeline에 누출된다. `code/normalization.py`가 seed를 고정한 episode-level shuffle과 두 정규화를 구현한다. 실제 프로젝트에서는 task variation, 작업자, 날짜를 group으로 묶은 stratified split을 우선한다.

## 5.5.2 작은 분산과 outlier

거의 움직이지 않는 joint의 표준편차가 0에 가까우면 작은 noise가 큰 normalized value가 된다. epsilon, std floor 또는 해당 feature 고정 처리를 명시한다. min/max는 잘못된 encoder spike 하나에 민감하므로 percentile-based clipping을 검토하되, joint safety limit과 통계적 clipping을 혼동하지 않는다.

## 5.5.3 action representation

- absolute joint target: 누적 drift가 적지만 embodiment와 calibration에 민감하다.
- delta joint: local motion 학습이 쉽지만 drift와 scale 선택이 중요하다.
- end-effector delta: task geometry에 직관적이지만 IK와 singularity 처리가 필요하다.

학습과 배포에서 같은 representation과 inverse normalization을 사용한다. action 통계를 state 통계로 역정규화하는 실수는 shape가 같으면 조용히 지나갈 수 있다.

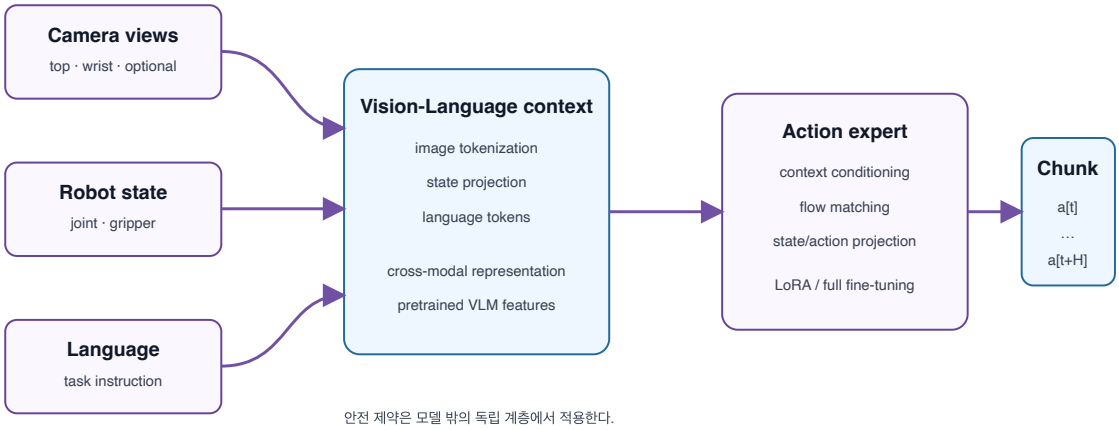
## 5.5.4 Dataset 전역 통계와 정책 학습 통계

LeRobotDataset v3의 `meta/stats.json`은 **dataset repository 전체**를 요약한다. 데이터 탐색과 무결성 점검에는 유용하지만 한 repository에서 일부 episode만 train으로 쓰는 경우, 이 전역 통계를 정책 정규화에 그대로 사용하면 validation/test 정보가 들어간다.

| 통계 | 범위 | 용도 | 저장 위치 예 |
|---|---|---|---|
| Dataset global stats | repository 전체 | 탐색, 시각화, 데이터 감사 | `meta/stats.json` |
| Policy train stats | train episode만 | 학습 및 배포 정규화 | checkpoint/run manifest |

split을 먼저 고정하고 train episode에서 stats를 다시 계산해 policy artifact에 저장한다. 또는 split별 dataset revision/repository를 materialize한다. 어느 방법이든 dataset revision, episode ID, stats hash를 checkpoint와 묶는다. `dataset.meta.stats`를 출력했다는 사실만으로 train-only normalization이 보장되지는 않는다.

# 5.6 SmolVLA 구조




SmolVLA 개념 구조

**그림 5-2.** 다중 카메라, robot state, language instruction을 context로 만들고 action expert가 미래 action chunk를 생성하는 개념 구조.

SmolVLA는 Hugging Face가 공개한 경량 VLA다. SmolVLA 공식 문서는 여러 카메라, 현재 sensorimotor state, 자연어 지시를 입력으로 받아 action expert가 action chunk를 생성한다고 설명한다. 공개 base checkpoint는 약 450M parameter 규모로 안내된다. 구조 세부와 benchmark는 SmolVLA 논문을 참고한다.

## 5.6.1 입력 context

- image tokens: top/wrist 등 여러 camera view
- state token: 현재 robot state를 projected embedding으로 변환
- language tokens: task instruction

camera key가 checkpoint가 기대하는 이름과 다르면 `rename_map`을 사용한다. 이름만 맞추는 것으로 끝나지 않고 view 의미가 일치하는지 확인한다. top camera를 wrist camera 이름으로 바꾸면 shape 검사는 통과해도 의미가 틀린다.

## 5.6.2 action expert와 flow matching

SmolVLA는 VLM context에 조건부인 action expert를 통해 연속 action chunk를 생성한다. 논문은 flow matching 기반 action generation을 사용한다. 개념적으로 noise sample $x_0$를 data action $x_1$ 방향으로 운반하는 velocity field $v_\theta(x_t, t, c)$를 학습한다.

$$x_t = (1-t)x_0 + tx_1, \qquad \mathcal{L}_{FM} = \mathbb{E} \parallel v_\theta(x_t, t, c) - (x_1 - x_0) \parallel^2$$

실제 구현의 path, noise, solver는 checkpoint/config에 따른다. 이 식은 원리를 보여 주는 단순화다.

### 5.6.3 chunk size

chunk size $H$가 크면 한 번의 inference로 긴 시간을 커버하지만 오래된 관측에 기반한 open-loop 구간이 길어진다. 작으면 자주 재계획하지만 inference load가 커진다. 원 교안의 16 frame은 실습 시작값으로 사용할 수 있으나 SmolVLA checkpoint default를 임의로 덮어쓰지 않는다. 저장된 policy config의 `chunk_size`와 `n_action_steps`를 확인한다.

# 5.7 LoRA 기반 파인튜닝

## 5.7.1 저랭크 업데이트

LoRA는 pretrained weight $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$를 고정하고 저랭크 행렬만 학습한다.

$$W = W_0 + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r \times d_{in}}, \quad B \in \mathbb{R}^{d_{out} \times r}$$

full matrix의 parameter는 $d_{out} d_{in}$, LoRA adapter는 $r(d_{in} + d_{out})$다. `code/lora_budget.py`로 규모를 비교할 수 있다. LoRA 원 논문은 Hu et al., 2021이다.

계산기의 값은 동일 크기 square projection을 가정한 단순 추정이다. 실제 SmolVLA 기본 PEFT target은 LM expert의 q/v projection과 task-dependent state/action projection을 포함할 수 있다. 학습 시작 시 framework가 출력한 trainable/total parameter 목록을 권위 있는 값으로 기록한다.

## 5.7.2 rank와 alpha

- rank $r$: 높을수록 update 표현력이 커지지만 parameter와 memory가 증가한다.
- alpha $\alpha$: update scaling에 관여한다.
- target modules: 어떤 projection에 adapter를 넣을지 결정한다.
- modules to save: 새 action/state projection처럼 full training이 필요한 모듈을 함께 저장한다.

현재 LeRobot의 PEFT training 문서는 SmolVLA에서 PEFT를 지원하고 `--peft.r`, `--peft.lora_alpha`를 제공한다. 공식 설정의 default target을 확인하고 임의의 정규식이 backbone 전체를 예상치 않게 포함하지 않는지 trainable parameter 목록을 출력한다.

## 5.7.3 full fine-tuning과 LoRA 선택

LoRA는 GPU memory와 checkpoint 크기를 줄이고 task별 adapter 관리에 유리하다. 그러나 새로운 embodiment의 action/state projection은 pretrained VLM과 성격이 다르므로 adapter만 학습할지, 일부 새 layer를 full train할지 검토해야 한다. 데이터가 충분하고 compute가 허용되면 full fine-tuning을 baseline으로 비교한다.

## 5.8 학습 파이프라인

### 5.8.1 데이터 사전 점검

```
python code/inspect_lerobot_v3.py \
  --repo-id USER/my_so101_dataset \
  --revision DATASET_COMMIT_SHA \
  --episode 0
```

환경 의존 예제이며 설치한 LeRobot API에 따라 import 경로가 달라질 수 있다. `--revision`에는 이동하는 `main` 대신 dataset commit SHA를 사용한다. 최소 점검 항목은 다음과 같다.

- 모든 camera key와 resolution
- state/action shape와 dtype
- NaN/Inf, joint limit 초과
- timestamp 역전과 frame gap
- episode length 분포
- task string과 실제 행동 일치
- normalization stats의 합리성

### 5.8.2 기본 fine-tuning 예

```
lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=${HF_USER}/my_so101_dataset \
  --dataset.revision=${DATASET_COMMIT_SHA} \
  --batch_size=16 \
  --steps=20000 \
  --output_dir=outputs/train/so101_smolvla \
  --job_name=so101_smolvla \
  --policy.device=cuda
```

공식 예시는 더 큰 batch를 제시하기도 하지만 VRAM에 맞춰 작은 값에서 시작한다. gradient accumulation을 쓸 경우 effective batch를 기록한다. “epoch 50”보다 dataset 크기와 batch에 독립적인 update step이 실험 비교에 편리할 수 있다.

### 5.8.3 LoRA 예

```
lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=${HF_USER}/my_so101_dataset \
  --dataset.revision=${DATASET_COMMIT_SHA} \
  --steps=20000 \
  --batch_size=16 \
  --peft.method_type=LORA \
  --peft.r=16 \
  --peft.lora_alpha=32 \
  --output_dir=outputs/train/so101_smolvla_lora
```

위 명령은 **예시**다. 정확한 flag는 고정한 LeRobot commit의 `--help`로 확인한다.

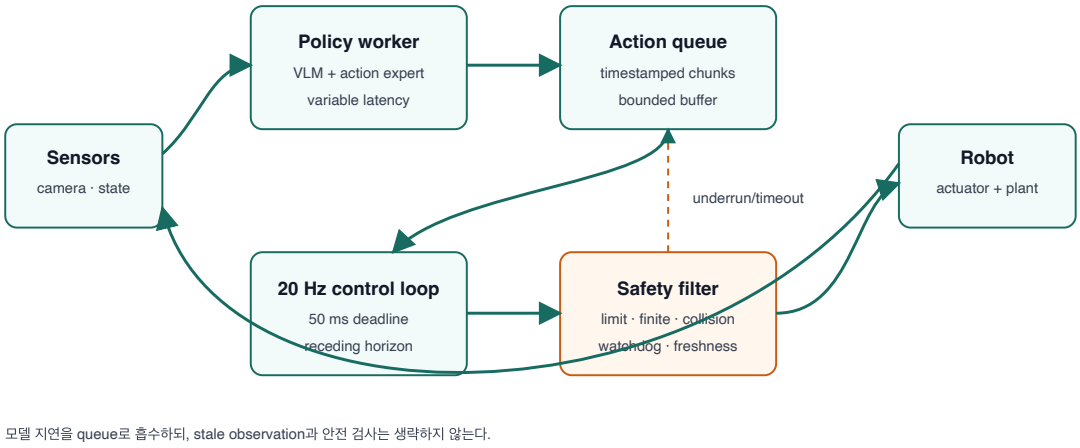
### 5.8.4 monitoring

training loss만 보지 않는다.

- validation action loss
- dimension별 action error
- predicted action smoothness와 limit violation
- fixed offline episode replay
- 실제/시뮬레이션 closed-loop task success
- variation별 성공률과 failure taxonomy

checkpoint 선택은 validation loss 최소점과 실제 success가 다를 수 있다. 최소한 두 지표를 함께 기록한다.

# 5.9 20 Hz 폐루프 추론




20 Hz 폐루프와 비동기 추론

**그림 5-3.** sensor/control loop와 policy worker를 분리하고 action queue, safety filter, watchdog을 거치는 폐루프.

## 5.9.1 주기와 deadline

20 Hz의 period는 50 ms다. 한 cycle의 예산은 capture, preprocessing, inference/queue, safety filter, communication을 포함한다.

$$T_{capture} + T_{pre} + T_{infer} + T_{safety} + T_{comm} \le 50 \text{ ms}$$

VLA inference가 50 ms를 넘으면 매 cycle 동기 추론은 deadline을 놓친다. 대안은 다음과 같다.

- action chunk를 queue에 넣고 control loop가 소비한다.
- policy worker를 별 thread/process/machine에 둔다.
- execution horizon이 소진되기 전에 다음 chunk를 요청한다.
- inference가 늦으면 이전 안전 action을 유지하거나 정지한다.

LeRobot은 Asynchronous Inference와 RTC 방식 안내를 제공한다. 사용 API는 버전에 따라 달라질 수 있으므로 개념과 실제 CLI를 구분한다.

## 5.9.2 receding horizon

chunk 전체를 끝까지 open-loop로 실행하지 않고 첫 $h$개만 실행한 뒤 새 관측으로 재계획한다. $h = 1$이면 가장 폐루프적이지만 inference rate가 높다. latency와 환경 변화 속도에 맞춰 execution horizon을 정한다.

## 5.9.3 temporal ensembling

시점 $t - j$에서 예측한 chunk가 현재 step $t$의 action도 포함할 수 있다. 여러 예측을 가중 평균한다.

$$\bar{a}_t = \frac{\sum_j w_j \hat{a}_t^{(t-j)}}{\sum_j w_j}, \qquad w_j = \exp(-\lambda j)$$

`code/temporal_ensemble.py`가 이를 구현한다. 평균은 discontinuity를 줄일 수 있지만 multimodal 행동의 서로 다른 mode를 섞으면 위험한 중간 행동을 만들 수 있다. grasp 전환점처럼 mode switch가 있는 구간은 history를 reset하거나 phase-aware ensemble을 사용한다.

후보를 무조건 평균하지 않는다. 각 proposal에 schema, unit, finite, source timestamp, task/phase, 기본 hard limit 검사를 먼저 적용하고 stale/invalid proposal을 버린다. 유효 후보만 ensemble/filter한 뒤 최종 결과에 rate, joint/workspace, collision 검사를 다시 적용한다.

### 5.9.4 safety filter와 watchdog

모델 출력은 곧바로 motor command가 아니다. 최소한 다음 단계를 거친다.

1. inverse normalization
2. NaN/Inf 검사
3. joint/workspace limit clamp 또는 reject
4. velocity/acceleration/jerk limit
5. self/environment collision check
6. timestamp freshness 확인
7. watchdog heartbeat
8. command 전송

`code/closed_loop_20hz.py`는 hardware-free mock loop로 joint limit과 per-cycle delta limit을 보여 준다. 실제 안전 정지는 software exception 하나에 의존하지 않고 hardware E-stop과 독립 watchdog을 둔다.

> **잠깐 점검 — 부드러운 출력과 안전한 출력은 같은가**
>
> 1. Temporal ensembling으로 행동이 매끄러워졌다면 관절 한계 검사를 생략해도 되는가?
> 2. 값과 단위가 올바른 action이라도 source timestamp가 오래되었다면 왜 실행하면 안 되는가?
>
> **확인 포인트:** 평균화는 불연속을 줄일 뿐 물리 한계를 보장하지 않는다. 오래된 action은 현재 관측과 상태를 반영하지 않으므로 폐기하거나 HOLD로 전환해야 한다.

### 5.9.5 최신 LeRobot 배포 backend

2026년 공식 LeRobot 배포 인터페이스는 동기 실행, local RTC(Real-Time Chunking), remote inference를 구분한다.

- **Sync**: control tick마다 policy call을 기다린다.
- **Local RTC**: rollout process의 background thread가 다음 chunk를 만들며 현재 trajectory와 이어지는 action을 준비한다.
- **Remote inference**: 별 process/host의 policy server와 통신하고 robot client가 bounded action buffer를 소비한다.

RTC의 `execution_horizon`, observation age, action/startup timeout, starvation grace는 실측 latency와 usable chunk horizon 안에서 정한다. queue가 비었다고 마지막 action을 무기한 유지하지 않는다. task instruction이 바뀌면 이전 instruction의 queued/in-flight action을 폐기하고 fresh observation으로 재계획한다. 최신 flag와 지원 checkpoint 계약은 Policy Deployment, Real-Time Chunking, Remote Inference를 따른다. 이전 async server/client tutorial의 `actions_per_chunk`, `chunk_size_threshold`와 최신 local RTC flag를 섞지 않는다.

# 5.10 평가 설계

## 5.10.1 task success

성공 기준을 평가 전에 명시한다. “대충 bin 안에 있음”이 아니라 다음처럼 정의한다.

- 대상 물체 중심이 bin 경계 내부
- 로봇이 물체를 놓은 뒤 2초 이상 유지
- 다른 물체를 넘어뜨리지 않음
- safety intervention 없음
- 제한 시간 30초 이내

## 5.10.2 stratified evaluation

전체 성공률만 보고하지 않는다.

| 조건 | trial 수 | 성공 | 성공률 | 주요 실패 |
|---|---|---|---|---|
| 학습 위치 | | | | |
| 미학습 위치 | | | | |
| 밝은 조명 | | | | |
| 낮은 조명 | | | | |
| distractor 포함 | | | | |

trial이 적을 때 90%와 80%의 차이를 과도하게 해석하지 않는다. seed, object placement protocol, 평가자 기준을 저장한다.

## 5.10.3 offline metric의 한계

action MSE/L1은 expert action과 다른 성공 경로를 벌점 처리할 수 있다. 반대로 낮은 error도 작은 누적 편향 때문에 closed-loop 실패로 이어질 수 있다. offline metric은 빠른 regression check로 사용하고 최종 판단은 closed-loop success와 안전 지표로 한다.

# 5.11 데이터 라이선스, 개인정보, 보안

## 5.11.1 dataset card와 license

Hub에 올리기 전에 다음 권리를 확인한다.

- 촬영 공간, 물체, 로봇 CAD/브랜드의 공개 권한
- 배경에 보이는 모니터·문서·상표
- third-party dataset과 pretrained model license
- 상업적 사용, 재배포, derivative model 허용 범위

dataset license와 model license는 별개일 수 있다. 여러 dataset을 섞으면 가장 제한적인 조건이 전체 배포를 제한할 수 있다. 법률 판단이 필요한 경우 조직의 담당자에게 검토를 요청한다.

## 5.11.2 개인정보

카메라에 얼굴, 이름표, 화면, 차량번호가 들어갈 수 있고 마이크를 쓰면 음성이 biometric/personal data가 될 수 있다.

- capture area를 물리적으로 제한한다.
- 불필요한 audio는 수집하지 않는다.
- 공개 전 자동 탐지와 사람 검수를 모두 수행한다.
- blur/redaction 후 원본 접근 권한과 보존 기간을 정한다.
- 삭제 요청이 들어왔을 때 dataset revision과 파생 checkpoint를 추적할 수 있게 한다.

## 5.11.3 token과 Hub 권한

Hugging Face token을 코드, shell history, dataset metadata에 넣지 않는다. 최소 권한 token과 private repository를 사용하고, 공개 전 별도 승인 절차를 둔다. 로봇의 serial number, 내부 IP, 작업장 위치 같은 운영 정보도 제거한다.

## 5.12 재현성 체크리스트

다음을 하나의 run manifest에 저장한다.

- Ubuntu, GPU, driver, CUDA, PyTorch 버전
- LeRobot git commit과 설치 extras
- base model repository와 exact revision/commit
- dataset repository와 exact revision
- train/validation/test episode ID
- feature schema, camera rename map, train-only normalization stats와 hash
- random seed와 deterministic setting
- batch size, gradient accumulation, optimizer, scheduler
- PEFT target, rank, alpha, trainable parameter 수
- checkpoint selection rule
- evaluation object placement와 success criteria

재현성은 bitwise 동일성만 의미하지 않는다. GPU kernel 비결정성 때문에 수치가 약간 달라도 같은 경향과 task success 범위를 얻는지 반복 run으로 확인한다.

## 5.13 단계별 실습

### 실습 A: v3 dataset 구조와 무결성 감사

실습 5-A: LeRobotDataset v3 무결성 감사에 따라 schema, episode boundary, video stream, stats, task mapping을 점검한다.

### 실습 B: normalization과 split 누출 실험

실습 5-B: Episode split과 정규화 누출에서 frame split과 episode split의 차이를 비교하고 train-only stats를 적용한다.

### 실습 C: SmolVLA LoRA 학습 계획

실습 5-C: SmolVLA LoRA 학습 설계에서 baseline, LoRA rank, ablation, checkpoint 평가표를 설계한다.

### 실습 D: 20 Hz action chunk 실행

실습 5-D: 20 Hz Action Chunk 폐루프에서 inference latency를 모사하고 queue underrun, temporal ensemble, safety filter를 비교한다.

## 5.14 예상 출력 예시

아래 출력은 환경에 따른 **예상 출력 예시**다.

```
python3 code/normalization.py
```

```
train episodes: [0, 1, 2, 3, 4, 6, 7, 8]
validation episodes: [5, 9]
train mean: [-0.0757  0.3933 -0.219 ]
validation normalized mean: [ 0.226  -0.0607  0.1596]
round-trip max error: 1.1102230246251565e-16
```

```
python3 code/temporal_ensemble.py
```

```
step=0: action=[0. 0.]
step=1: action=[0.3401 0.1401]
step=2: action=[0.674 0.274]
step=3: action=[1.074 0.474]
```

```
python3 code/lora_budget.py --hidden 1024 --rank 16 --layers 16
```

```
Simplified LoRA estimate: 1,048,576
corresponding full matrices: 33,554,432
ratio: 3.125%
scaling example (alpha=32): 2.0
Use the framework's trainable-parameter report as the authoritative count.
```

## 5.15 트러블슈팅

### dataset이 load되지 않는다

- 수집/변환 뒤 `finalize()`를 호출했는가?
- `meta/info.json`의 codebase version과 설치 LeRobot이 호환되는가?
- path를 손으로 만들지 않고 API가 제공한 template을 썼는가?
- Parquet footer와 MP4 index가 정상적으로 닫혔는가?

### 영상과 관절이 어긋난다

- frame index가 아니라 timestamp로 연결했는가?
- camera buffering latency와 motor timestamp clock source가 같은가?
- variable frame rate video를 constant rate로 가정하지 않았는가?
- action이 observation 이전/이후 어느 시점의 command인지 정의했는가?

### loss는 감소하지만 로봇이 거의 움직이지 않는다

- action normalization과 inverse normalization이 같은가?
- state/action joint order가 배포 robot과 같은가?
- LoRA alpha/r scaling과 target module을 확인했는가?
- 새 action projection이 실제로 trainable인가?
- dataset에서 정지 frame이 지나치게 많지 않은가?

### 동작이 떨리거나 chunk 경계에서 튄다

- chunk의 첫 action이 현재 state와 연속적인가?
- temporal ensemble 또는 execution horizon을 비교했는가?
- absolute와 delta action을 혼동하지 않았는가?
- 제어기 자체 rate와 policy rate가 정수비인지 확인했는가?

### GPU memory가 부족하다

- batch size를 줄이고 gradient accumulation을 기록한다.
- mixed precision과 activation checkpointing 지원을 확인한다.
- LoRA rank와 target modules를 줄인다.
- image resolution/camera 수를 줄일 때 성능 영향 ablation을 수행한다.
- OOM 직전 run의 partial checkpoint를 신뢰하지 않는다.

### inference가 20 Hz를 만족하지 못한다

- capture/preprocess/model/postprocess/communication을 따로 계측한다.
- action chunk queue와 asynchronous worker를 사용한다.
- stale observation과 queue underrun 정책을 정의한다.

- latency를 숨기기 위해 safety check를 제거하지 않는다.

## 5.16 요약

- behavior cloning은 supervised learning처럼 보이지만 정책이 만든 관측 분포 때문에 오류가 누적된다.
- ACT와 Diffusion Policy는 action sequence와 다봉 행동분포를 다루는 서로 다른 방법이다.
- VLA는 시각·언어·robot state를 context로 action을 생성하지만 안전 제약을 대체하지 않는다.
- LeRobotDataset v3는 여러 episode를 큰 Parquet/MP4 shard에 저장하고 metadata로 경계를 복원한다.
- split과 normalization은 episode/group 단위로 설계해 정보 누출을 막는다.
- SmolVLA는 VLM context와 action expert를 결합해 action chunk를 생성한다.
- LoRA는 저랭크 update로 trainable parameter를 줄이지만 target과 새 projection 학습 여부를 점검해야 한다.
- 20 Hz 배포에는 chunk queue, receding horizon, temporal ensemble, deadline, safety filter, watchdog이 필요하다.
- dataset/model revision, license, 개인정보, 평가 protocol은 코드와 같은 수준의 산출물이다.

## 5.17 확인문제

1. behavior cloning의 covariate shift가 일반적인 supervised image classification과 다른 점을 설명하라.
2. 같은 관측에 두 개의 성공 action mode가 있을 때 L2 단일 회귀가 실패할 수 있는 예를 들라.
3. action chunk가 effective horizon과 inference rate에 주는 영향을 설명하라.
4. ACT와 Diffusion Policy가 multimodality를 표현하는 방식을 비교하라.
5. v3에서 파일 이름으로 episode 경계를 판단하면 안 되는 이유는 무엇인가?
6. frame random split이 validation score를 과대평가하는 이유를 설명하라.
7. train/validation 전체에서 normalization stats를 계산하는 것이 왜 누출인가?
8. LoRA에서 rank와 alpha가 update에 미치는 영향을 식으로 설명하라.
9. temporal ensembling이 multimodal action에서 위험할 수 있는 사례를 들라.
10. 20 Hz loop의 inference가 80 ms 걸릴 때 가능한 system architecture 두 개를 제안하라.
11. dataset license와 pretrained model license를 별도로 확인해야 하는 이유는 무엇인가?
12. offline action loss가 낮아도 task success가 낮을 수 있는 원인을 세 가지 쓰라.

추가 연습문제와 구현 과제는 제5장 연습문제 및 구현 과제에 있다.

## 5.18 프로젝트 과제

### 과제 1: Dataset v3 품질 보고서

50개 이상 episode를 대상으로 schema, duration, frame gap, state/action distribution, task balance, video sample을 분석한다. 데이터 삭제/제외 기준과 privacy review 결과를 포함한다.

### 과제 2: ACT 또는 SmolVLA baseline 비교

동일 split과 action representation에서 단일-step BC와 chunk policy를 비교한다. offline loss, smoothness, closed-loop success, inference latency를 보고한다.

### 과제 3: LoRA ablation

rank 8/16/32와 full fine-tuning 일부 조건을 비교한다. trainable parameter, peak VRAM, wall time, validation loss, task success를 함께 제시한다.

### 과제 4: 20 Hz deployment harness

실제 policy 대신 latency를 갖는 mock worker로 action queue를 구현한다. inference latency spike, camera delay, queue underrun, NaN action을 주입하고 watchdog과 safe stop이 작동하는지 기록한다.

# 5.19 실습·평가·참고자료

이 절 아래에 장별 참고문헌, 단계별 실습, 연습문제와 구현 과제를 한데 모았다.

## 5.19.1 참고문헌


1. Hugging Face, LeRobot Documentation, 2026년 10월 확인.
2. Hugging Face, LeRobotDataset v3.0, 2026년 10월 확인.
3. Hugging Face, LeRobot Dataset API, 2026년 10월 확인.
4. Hugging Face, SmolVLA, 2026년 10월 확인.
5. Shukor et al., SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025.
6. Zhao et al., Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware, 2023.
7. Chi et al., Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, 2023/2024.
8. Hu et al., LoRA: Low-Rank Adaptation of Large Language Models, 2021.
9. Ross, Gordon, Bagnell, A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, AISTATS 2011.
10. Hugging Face, Parameter-efficient fine-tuning with PEFT, 2026년 10월 확인.
11. Hugging Face, Asynchronous Inference, 2026년 10월 확인.
12. Hugging Face, Policy Deployment, 2026년 10월 확인.
13. Hugging Face, Real-Time Chunking, 2026년 10월 확인.
14. Hugging Face, Remote Inference, 2026년 10월 확인.


## 실습 5-A: LeRobotDataset v3 무결성 감사

### 목표

- storage shard와 logical episode를 구분한다.
- schema, timestamp, video, task, stats의 불일치를 찾는다.
- 공개 전 개인정보와 라이선스를 점검한다.

### 절차

1. dataset repository의 exact revision을 기록한다.
2. `meta/info.json`에서 codebase version, fps, feature schema, path template을 확인한다.
3. `meta/episodes`를 API로 읽고 episode length 분포를 그린다.
4. episode 0, 중간, 마지막을 선택해 첫/마지막 frame과 action을 확인한다.
5. 각 camera stream의 frame 수, resolution, timestamp gap을 확인한다.
6. state/action에서 NaN, Inf, joint limit 초과, 순간 jump를 찾는다.
7. task string을 영상과 대조한다.
8. `stats.json`의 mean/std/min/max를 raw sample에서 다시 산출한 값과 비교한다.
9. dataset card의 license, 수집 동의, 공개 범위, 삭제 절차를 검토한다.
10. 얼굴·이름표·모니터·문서·음성 등 개인정보 노출을 sample과 무작위 episode에서 검수한다.

### 실행 예

```
python code/inspect_lerobot_v3.py \
  --repo-id USER/my_so101_dataset \
  --revision DATASET_COMMIT_SHA \
  --episode 0
```

### 감사표

| 검사 | 기대 | 관찰 | 판정 | 조치 |
|---|---|---|---|---|
| codebase version | v3 호환 | | | |
| camera frame gap | 정의한 tolerance 이내 | | | |
| action finite | 전부 finite | | | |
| task/video 일치 | 전부 일치 | | | |
| privacy | 식별정보 없음 | | | |

### 완료 조건

- 최소 3개 episode를 시작부터 끝까지 확인했다.
- 이동하는 branch가 아니라 exact dataset commit SHA를 기록했다.
- critical/major/minor issue를 구분했다.
- 공개 차단 조건과 재수집 조건을 명시했다.

## 실습 5-B: Episode split과 정규화 누출

### 목표

- frame random split과 episode split을 비교한다.
- train-only statistics를 적용한다.
- outlier와 작은 표준편차 문제를 찾는다.

### 절차

1. `python code/normalization.py`를 실행한다.
2. `split_by_episode()` 대신 frame을 무작위 분할한 조건을 만든다.
3. 인접 frame의 state 차이를 기준으로 train/validation 유사도를 비교한다.
4. 전체 dataset stats와 train-only stats로 validation을 각각 정규화한다.
5. 한 joint에 큰 encoder spike를 한 개 주입하고 mean/std/min/max 변화를 기록한다.
6. 한 joint를 상수로 만들고 epsilon 없이 나누었을 때 결과를 확인한다.
7. min-max, z-score, percentile clipping을 비교한다.
8. LeRobot `meta/stats.json`의 dataset-global stats와 train episode에서 다시 계산한 policy stats를 비교하고, 후자와 split/hash를 checkpoint manifest에 저장한다.

### 질문

1. frame split의 validation loss가 낮아진다면 왜 일반화 향상으로 해석할 수 없는가?
2. joint limit clamp와 statistical clipping은 어떤 순서와 목적으로 적용해야 하는가?
3. validation에 train 범위를 벗어나는 값이 있을 때 자동으로 제거하면 안 되는 이유는 무엇인가?

### 제출물

- 분할별 episode ID 목록
- train-only normalization stats
- outlier 전후 통계표
- 선택한 normalization과 근거

## 실습 5-C: SmolVLA LoRA 학습 설계

### 목표

- 재현 가능한 baseline과 LoRA ablation을 설계한다.
- trainable parameter와 VRAM/성능 trade-off를 기록한다.

### 사전 기록

```
git -C lerobot rev-parse HEAD
python -c "import torch; print(torch.__version__, torch.version.cuda)"
nvidia-smi
lerobot-train --help > train-help.txt
```

### 실험 행렬

| run | method | rank | alpha | target | batch/effective | seed |
|---|---|---|---|---|---|---|
| B0 | full 또는 공식 baseline | - | - | | | 7 |
| L8 | LoRA | 8 | 16 | default | | 7 |
| L16 | LoRA | 16 | 32 | default | | 7 |
| L32 | LoRA | 32 | 64 | default | | 7 |

#### LoRA 예

```
lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=${HF_USER}/my_so101_dataset \
  --dataset.revision=${DATASET_COMMIT_SHA} \
  --steps=20000 \
  --batch_size=16 \
  --peft.method_type=LORA \
  --peft.r=16 \
  --peft.lora_alpha=32 \
  --output_dir=outputs/train/lora_r16
```

설치한 commit의 `--help`와 다르면 해당 버전의 flag를 사용하고 차이를 기록한다.

#### 측정

- trainable/total parameter
- dataset/base model/adapter revision과 train-only stats hash
- peak VRAM
- 1,000 step wall time
- validation loss
- action limit violation rate
- 20회 closed-loop success

#### 중단 조건

- NaN loss
- 반복되는 action limit violation
- dataset/video decode error
- GPU thermal/power 이상
- evaluation 중 safety intervention

### 실습 5-D: 20 Hz Action Chunk 폐루프

#### 목표

- 50 ms deadline과 inference latency 관계를 측정한다.
- receding horizon과 temporal ensemble을 비교한다.
- queue underrun 시 safe behavior를 설계한다.

#### 준비

```
python code/temporal_ensemble.py
python code/closed_loop_20hz.py
```

### 실험 조건

1. inference latency를 10, 40, 70, 120 ms로 모사한다.
2. chunk horizon을 1, 8, 16으로 바꾼다.
3. execution horizon을 1, 4, 8로 바꾼다.
4. temporal ensemble decay를 0, 0.25, 1.0으로 바꾼다.
5. 2% 확률의 latency spike와 NaN action을 주입한다.
6. stale chunk, task/phase가 다른 chunk, hard limit을 크게 벗어난 proposal을 주입하고 ensemble 전에 제거되는지 확인한다.
7. grasp/retreat phase 전환과 E-stop/reset에서 ensemble history를 비운다.

### 측정

- missed deadline 수
- queue underrun 수
- command delta/jerk
- goal error
- safety reject 수
- stale observation age

### 안전 정책 설계

다음 사건별 동작을 표로 작성한다.

| 사건 | 감지 조건 | 즉시 동작 | 복구/종료 |
|---|---|---|---|
| queue underrun | 실행할 action 없음 | hold 또는 감속 정지 | 새 chunk 대기 |
| stale image | age > threshold | 신규 inference 금지 | sensor 복구 |
| NaN action | finite check 실패 | command reject | safe stop |
| watchdog loss | heartbeat timeout | torque/velocity 안전 상태 | operator reset |

### 토의

temporal averaging으로 smoothness는 좋아졌지만 goal error가 커질 수 있는 경우를 설명한다. 서로 다른 action mode를 평균하는 것이 안전하지 않은 사례도 제시한다.

## 제5장 연습문제 및 구현 과제

### 개념 문제

1. expert distribution과 learned-policy distribution을 식으로 구분하라.
2. action chunk가 오류 누적을 줄이는 측면과 open-loop 위험을 늘리는 측면을 설명하라.
3. Diffusion Policy와 flow matching action expert의 공통점과 차이를 조사하라.
4. VLA에서 language instruction이 모호하거나 잘못되었을 때 안전 계층이 해야 할 일을 설명하라.
5. LeRobotDataset v3가 episode당 파일 대신 shard를 사용하는 이유를 설명하라.
6. dataset metadata가 손상됐지만 MP4와 Parquet가 남아 있을 때 episode를 완전히 복원하기 어려운 이유는 무엇인가?
7. normalization stats도 model artifact로 versioning해야 하는 이유를 쓰라.
8. LoRA adapter와 base checkpoint revision을 함께 저장해야 하는 이유는 무엇인가?
9. temporal ensemble과 low-pass filter의 입력/의미 차이를 설명하라.

10. dataset 공개 이후 삭제 요청이 model checkpoint에 미치는 영향을 논하라.

### 계산 문제

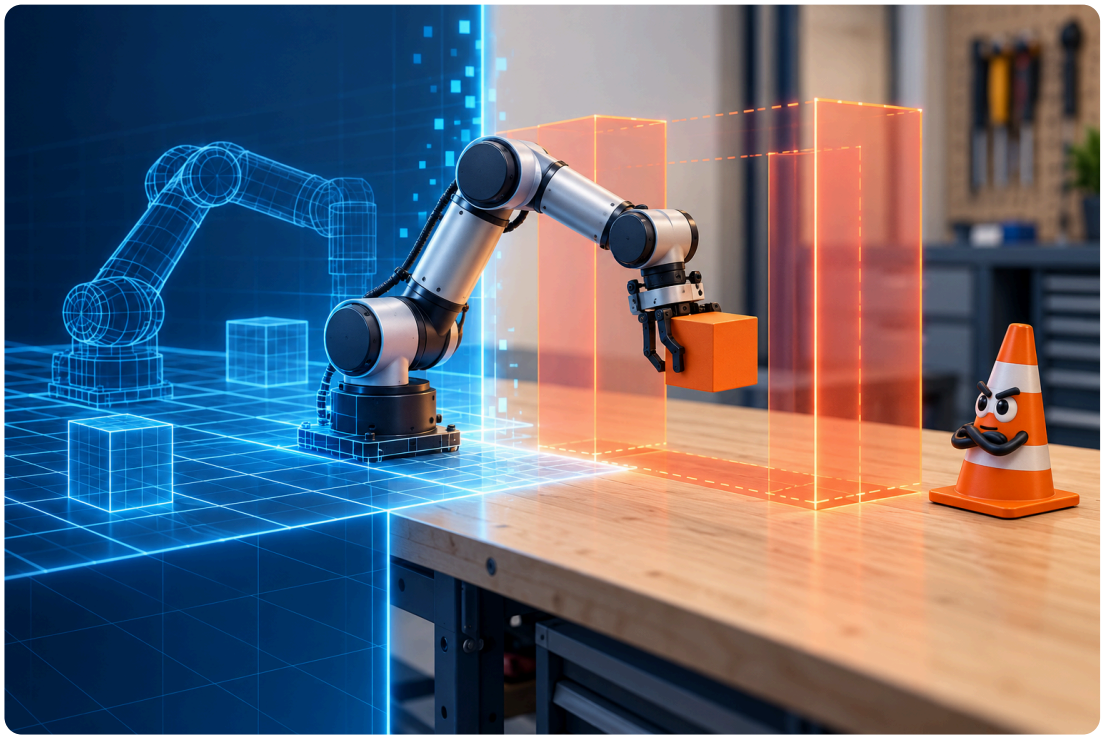
1. 20 Hz loop에서 capture 8 ms, preprocess 6 ms, inference 42 ms, safety 3 ms, communication 4 ms가 걸린다. deadline miss와 개선안을 계산하라.
2. `1024×1024` projection 두 개가 16 layer에 있고 rank 16 LoRA를 적용한다. full/LoRA parameter 수와 비율을 구하라.
3. 50 Hz로 20초 수집한 60 episode의 tabular row 수를 구하라.
4. chunk size 16, control 20 Hz일 때 chunk가 나타내는 시간 길이를 구하라.
5. ensemble decay $\lambda = 0.5$이고 age가 0,1,2인 세 예측의 normalized weight를 구하라.

### 구현 문제

1. episode group을 task variation별로 stratified split하는 함수를 작성하라.
2. timestamp gap과 역전을 보고하는 dataset audit 도구를 작성하라.
3. `TemporalEnsembler`에 최대 history와 mode-change reset 기능을 추가하라.
4. closed-loop mock에 별도 inference thread와 bounded queue를 추가하라.
5. run manifest의 SHA-256을 계산하고 checkpoint와 함께 저장하는 도구를 작성하라.

제6장

# Sim2Real 배포 최적화와 최종 실증



시뮬레이션에서 현실 작업대로 이동하며 안전 경계를 통과하는 로봇팔

*"시뮬레이션에서는 Undo가 된다. 현실에서는 안전 감독기가 그 역할을 맡는다."*

# 제6장 Sim2Real 배포 최적화와 최종 실증

## 장 개요

시뮬레이션에서 95% 성공한 정책이 실물에서 실패하는 원인은 "현실이 더 복잡해서"라는 한 문장으로 끝나지 않는다. 카메라 외부 파라미터 8 mm 오차, USB 통신의 60 ms 꼬리 지연, 서보 백래시, 그리퍼 마찰, 조명 반사처럼 서로 다른 오차가 관측과 행동 루프에서 합성된다. 실무의 Sim2Real은 모델을 한 번 더 학습하는 일이 아니라, 오차를 측정하고 안전하게 가설을 검증하는 시스템 엔지니어링이다.

이 장에서는 Reality Gap을 시각, 기하, 동역학, 시간, 제어, 과업 분포의 여섯 축으로 분해한다. 이어 저역통과 필터(Low-Pass Filter), 지수이동평균(Exponential Moving Average, EMA), 시간 앙상블(Temporal Ensembling)을 구분하고, 작업공간 경계와 워치독을 가진 안전 감독기를 만든다. 마지막으로 SO-101 Pick & Place 정책을 dry-run에서 정상 속도까지 단계적으로 승격하고 성공률과 신뢰구간을 보고한다.

> **안전 면책:** SO-101과 같은 저가형 교육용 로봇이라도 끼임, 충돌, 낙하, 과전류 위험이 있다. 이 장은 위험성 평가나 인증 지침이 아니다. ISO 10218-1:2025는 산업용 로봇 자체의 안전 요구를, ISO 10218-2:2025는 통합과 로봇 셀을 다룬다. 적용 범위와 현장 규정은 별도로 판단해야 한다.[ISO, 2025]

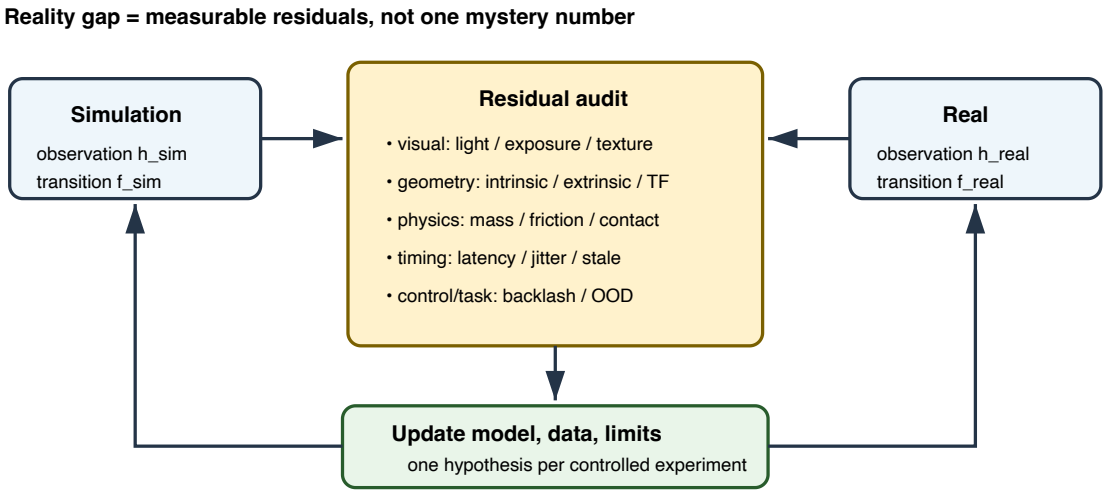



그림 6-1. Reality Gap을 측정 가능한 오차 항목으로 분해하는 폐루프

## 학습 목표

이 장을 마치면 학습자는 다음을 수행할 수 있다.

1. Reality Gap을 관측·기하·물리·시간·제어·과업 분포 오차로 분류한다.
2. 지연, jitter, 백래시, 마찰, 카메라 외부 파라미터 오차를 정량 진단한다.
3. EMA와 action chunk 시간 앙상블의 목적과 지연 비용을 설명한다.
4. 작업공간 경계, 관절/속도 제한, 워치독, 데드맨, 독립 E-stop을 계층화한다.

5. dry-run → 저속 → 제한 작업공간 → 정상 운전의 승격 게이트를 설계한다.
6. 음성/텍스트 명령을 정책 입력과 안전 권한으로 분리한다.
7. 성공률과 Wilson 신뢰구간, 조건별 실험 설계를 보고한다.
8. 재현 가능한 로보틱스 포트폴리오를 패키징한다.

## 선수 지식과 준비물

- 제3장: SO-101 기구학, 관절 제한, Leader-Follower 안전
- 제4장: 카메라 좌표계, TF2, 타임스탬프 동기화
- 제5장: LeRobot Dataset, SmolVLA/ACT 계열 정책, action chunk
- Ubuntu 24.04 LTS, ROS 2 Jazzy, Python 3.12
- 선택: SO-101 Follower, 고정 작업대, 카메라, 물리 E-stop 또는 즉시 전원 차단 수단
- 평가용 물체와 트레이, 고정된 조명 조건과 변화 조건

실물 실습자는 장비 전압, 고정 상태, 케이블 경로, 작업공간 비움, 그리퍼 payload, 비상 정지 수단을 점검한다. 한 사람이 실험을 실행하고 다른 한 사람이 E-stop을 담당하는 2인 운용을 권장한다.

# 6.1 Reality Gap의 구조

Reality Gap을 하나의 스칼라로 보지 말고 정책이 받는 관측 $o_t$, 내보내는 행동 $a_t$, 실제 시스템 상태 $x_t$의 경로로 분해한다.

$$o_t = h_{real}(x_{t-\tau_o}; \theta_{camera}, \theta_{light}) + \epsilon_o$$

$$x_{t+1} = f_{real}(x_t, a_{t-\tau_a}; \theta_{mass}, \theta_{friction}, \theta_{backlash}) + \epsilon_x.$$

시뮬레이터의 $h_{sim}, f_{sim}$과 실제 $h_{real}, f_{real}$의 차이가 정책 입력 분포와 전이 분포를 바꾼다. 중요한 것은 각 $\theta$와 지연 $\tau$를 측정할 수 있는 실험으로 바꾸는 것이다.

> **이해 도움말 — Reality Gap은 하나의 점수가 아니라 원인 지도다**
>
> 실물 성공률이 낮다는 사실만으로는 무엇을 고쳐야 할지 알 수 없다. 먼저 "관측이 달라졌는가, 명령 전달이 늦었는가, 같은 명령에도 물체가 다르게 움직였는가"로 경로를 나눈다. 예를 들어 영상의 물체 위치는 맞지만 그리퍼가 늦게 닫히면 시각 랜덤화를 늘릴 문제가 아니라 지연(latency)과 구동 계층을 측정할 문제다. 한 번에 한 범주만 바꾸는 진단 실험이 원인과 처방을 연결한다.

## 6.1.1 오차 분류표

| 범주 | 대표 원인 | 측정량 | 첫 조치 |
|---|---|---|---|
| 시각 | 조도, 반사, 색온도, 노출 | 히스토그램, blur, detector confidence | 카메라 고정, 노출 잠금, 조건별 데이터 |
| 기하 | intrinsic/extrinsic, 프레임 부호 | reprojection error, target pose residual | 재캘리브레이션, TF 감사 |
| 동역학 | 질량, 마찰, 관성, 유연성 | 낙하/슬라이드 시간, 접촉 성공률 | 식별 후 시뮬레이터 범위 조정 |
| 구동 | 백래시, deadband, 포화 | 방향 반전 시 히스테리시스 | 접근 방향 통일, deadband 모델 |
| 시간 | 카메라/추론/통신 지연, jitter | p50/p95/p99 latency, deadline miss | timestamp 분리, 버퍼/주기 조정 |
| 과업 | 물체 위치·색·배경 분포 | 조건별 성공률 | 층화 데이터와 평가 세트 |

## 6.1.2 도메인 랜덤화의 역할과 한계

도메인 랜덤화(Domain Randomization)는 시뮬레이터의 질감, 조명, 카메라, 물리 파라미터를 분포에서 샘플링해 실제 환경을 그 분포의 한 사례로 보게 하는 전략이다. Tobin 등은 비현실적인 질감을 포함한 시뮬레이션 영상만으로 실제 물체 위치 추정과 파지를 시연했다.[Tobin et al., 2017]

하지만 "많이 랜덤화"가 항상 좋은 것은 아니다.

- 실제 범위를 포함하지 않으면 의미가 없다.
- 너무 넓은 범위는 학습 용량을 불필요한 변동에 쓴다.
- 서로 상관된 값, 예를 들어 노출과 조도, 질량과 관성은 독립 샘플링이 비현실적 장면을 만든다.
- 검증 세트까지 같은 난수 생성기로 만들면 일반화를 과대평가한다.
- Isaac Sim Replicator의 API namespace는 버전별로 달라질 수 있으므로 설치 버전 문서를 따른다.[NVIDIA, 2026]

# 6.2 정량 진단

## 6.2.1 지연과 jitter

하나의 프레임이 행동이 되기까지 다음 타임스탬프를 기록한다.

| 기호 | 사건 |
|---|---|
| $t_c$ | 카메라 exposure 또는 캡처 완료 |
| $t_r$ | 애플리케이션 수신 |
| $t_i^0, t_i^1$ | 추론 시작/끝 |
| $t_s$ | 모터 명령 송신 |
| $t_m$ | 모터 상태에서 움직임 관측 |

핵심 지표는 평균만이 아니다.

$$T_{sense} = t_r - t_c, \quad T_{infer} = t_i^1 - t_i^0, \quad T_{e2e} = t_m - t_c.$$

p50은 보통 경험, p95/p99는 드문 위험 지연을 보여준다. 다만 20개 표본의 p99는 사실상 최댓값 근처의 보간값일 뿐 tail latency를 안정적으로 추정하지 못한다. latency는 episode 수와 별개로 충분한 control tick/frame 표본을 모으고 n, max, deadline miss를 함께 보고한다. 작은 표본의 p95/p99는 탐색적 값으로만 취급한다. 시간원(clock domain)이 다르면 NTP/PTP 동기화 또는 동일 프로세스의 monotonic timestamp를 사용한다. 음수 지연은 로봇이 미래를 본 것이 아니라 시계가 정합되지 않았다는 신호다.

`code/sim2real_diagnostics.py`는 CSV의 latency와 성공 결과를 요약한다. 예상 CSV schema는 `episode_id,condition,success,capture_ns,receive_ns,infer_start_ns,infer_end_ns,send_ns,motion_ns`다.

## 6.2.2 백래시와 deadband

백래시는 명령 방향이 바뀐 뒤 모터축은 움직여도 출력 링크가 늦게 움직이는 구간이다. 측정 절차는 다음과 같다.

1. 안전한 중간 자세에서 한 관절만 선택한다.
2. 작은 양의 증가를 여러 번 명령하고 외부 표식 또는 엔코더로 출력 움직임을 기록한다.
3. 방향을 반대로 바꿔 출력이 실제로 반전되기까지의 명령 누적량을 측정한다.
4. 속도와 payload 조건을 반복한다.

방향 전환 지연을 각도 차이로 환산해 backlash band를 얻는다. 정책 출력에 무조건 역보상을 더하면 노이즈에서 채터링이 생길 수 있다. 동일 방향 접근, 속도 저감, 히스테리시스 모델, 접촉 전 최종 시각 보정을 우선 검토한다.

## 6.2.3 마찰과 그리퍼

시뮬레이션의 마찰 계수를 임의로 "성공할 때까지" 높이지 않는다. 동일 재질 쌍과 접촉 조건에서 다음을 기록한다.

- 경사면에서 미끄러지기 시작하는 임계각 $\theta$: $\mu_s \approx \tan\theta$
- 일정 하중에서 미끄러짐 속도

- 그리퍼 명령값과 물체 유지 성공률
- 물체 질량, 표면, 접촉 면적, 그리퍼 패드 상태

저가형 그리퍼에 힘 센서가 없다면 “명령 위치가 덜 닫혔다”는 사실은 파지력의 직접 측정이 아니다. 물체 걸림과 진짜 파지를 구분할 추가 관측이 필요하다.

### 6.2.4 카메라 외부 파라미터

Eye-to-Hand에서는 베이스와 고정 카메라 사이 변환, Eye-in-Hand에서는 엔드이펙터와 카메라 사이 변환을 추정한다. Hand-eye calibration은 보통

$$AX = XB$$

형태의 여러 pose 쌍을 푼다. Tsai와 Lenz는 카메라와 로봇의 상대 운동으로 3D hand-eye 관계를 구하는 고전적 방법을 제시했다. [Tsai and Lenz, 1989]

한 개의 reprojection error 숫자만 보지 않는다. 작업 공간 전체에 보드를 배치하고 다음을 확인한다.

- 중앙과 모서리의 reprojection residual
- 거리/방향별 pose residual
- 장착 나사를 다시 조인 뒤 반복성
- 온도와 케이블 장력 변화
- TF의 프레임 방향과 timestamp

# 6.3 출력 안정화

## 6.3.1 EMA 저역통과 필터

관절 목표 $u_t$에 EMA를 적용하면

$$y_t = \alpha u_t + (1-\alpha) y_{t-1}, \quad 0 < \alpha \le 1$$

이다. 작은 $\alpha$는 노이즈를 줄이지만 phase lag를 늘린다. 안전 제한보다 필터를 먼저 적용하면 필터 내부 상태가 경계 밖 값을 저장할 수 있다. 권장 순서는 정책 출력 검증 → 단위 변환 → 시간 앙상블/필터 → 속도 제한 → 위치·작업공간 제한 → 드라이버다.

필터는 문제를 숨길 수 있다. 타임스탬프 역전, NaN, 통신 끊김은 smoothing 대상이 아니라 fault다.

## 6.3.2 Temporal Ensembling

Action chunk 정책은 매 시점 미래 행동 시퀀스를 예측한다. 시점 $t$의 행동에는 과거 여러 추론 chunk가 제안한 값이 겹친다. 이를 가중 평균하면 chunk 경계의 불연속을 줄일 수 있다.

$$\widehat{a}_t = \frac{\sum_{k \in K_t} w_{t,k} a_t^{(k)}}{\sum_{k \in K_t} w_{t,k}}.$$

ACT는 action chunking과 temporal ensembling으로 모방학습의 누적 오차와 비정상적 시연 문제를 다룬다.[Zhao et al., 2023] `code/temporal_filters.py`는 EMA와 겹치는 chunk의 지수 가중 평균을 구현한다.

시간 앙상블은 안전 감독기를 대체하지 않는다. 먼저 proposal마다 schema, unit, finite, timestamp freshness, task/phase, 기본 hard limit을 확인해 stale/invalid proposal을 제거한다. 유효 후보만 합성한 뒤 최종 출력에 joint/workspace/rate/collision limit을 다시 적용한다. grasp/open/retreat처럼 mode가 바뀌거나 E-stop/reset이 발생하면 ensemble history를 비운다. 서로 다른 chunk가 모두 경계 밖 행동을 제안하면 평균도 위험하다.

# 6.4 안전 배포 감독기

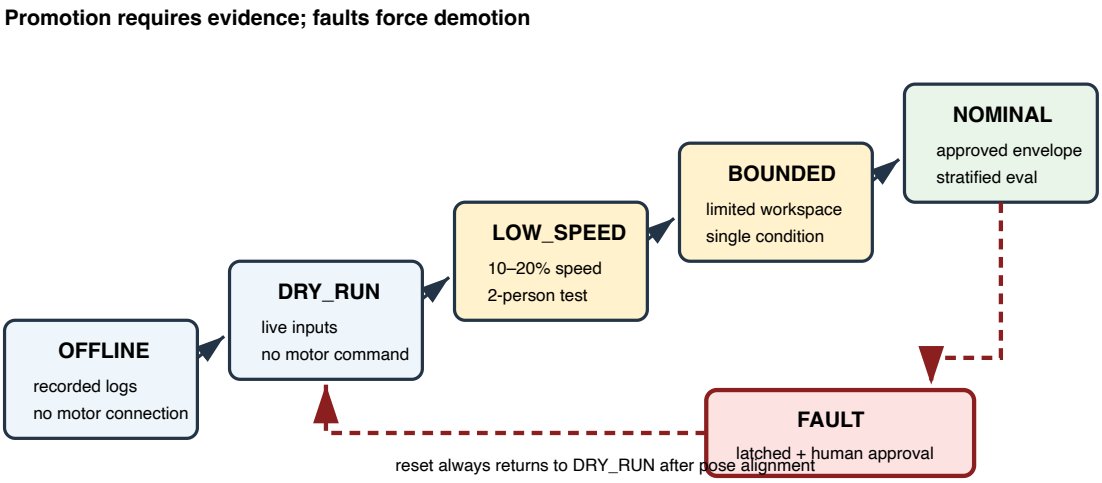



그림 6-2. Dry-run에서 정상 운전까지 단계적 승격

## 6.4.1 승격 상태

| 단계 | 모터 | 속도 | 작업공간 | 통과 기준 예시 |
|---|---|---|---|---|
| 0. OFFLINE | 분리 | - | 로그 재생 | NaN 0건, schema/단위 확인 |
| 1. DRY_RUN | 미명령 | 계산만 | 전체를 검사 | 경계 위반률 0, latency 예산 충족 |
| 2. LOW_SPEED | 활성 | 10~20% | 보수적 박스 | 무부하 20회, E-stop 시험 |
| 3. BOUNDED | 활성 | 30~50% | 실제 과업 일부 | 단일 물체 20회, 충돌 0 |
| 4. NOMINAL | 활성 | 승인값 | 승인 작업공간 | 조건별 평가 및 운영 승인 |

승격은 단방향 자동 증가가 아니다. 다음 사건은 즉시 HOLD 또는 FAULT로 강등한다.

- 워치독 timeout 또는 deadline 연속 위반
- NaN/Inf, 관절 이름/차원 불일치
- 작업공간/관절 한계 위반
- E-stop, 모터 오류, 카메라 입력 손실
- 정책/캘리브레이션/URDF 해시 변경

FAULT에서 자동 재개하지 않는다. 원인 기록, 작업공간 확인, 자세 재정렬, 사람 승인 후 낮은 단계부터 다시 시작한다.

### 6.4.2 작업공간 Bounding Box

베이스 프레임에서 엔드이펙터 위치 $p = (x, y, z)$가

$$p_{min} \le p \le p_{max}$$

를 만족해야 한다. 축 정렬 박스는 구현이 쉽지만 장애물과 로봇 자체 충돌을 표현하지 못한다. 따라서 최소한 다음을 결합한다.

- 관절 soft limit
- 엔드이펙터 bounding box
- 테이블 아래 $z$ 금지
- 최대 관절 변화/속도
- self-collision/환경 collision 검사
- gripper payload와 접근 방향 규칙

정책의 Cartesian target을 단순 clamp하면 방향이 바뀌거나 표면을 따라 미끄러질 수 있다. 큰 위반은 clamp보다 거부와 HOLD가 안전하다.

작업공간 검사는 반드시 **어느 좌표계의 값인지**와 함께 수행한다. `PoseStamped`의 `frame_id`가 `base_link`와 다르면 숫자만 base 좌표로 재표기해서는 안 된다. 이 장의 교육용 guard는 보수적으로 동일 `base_link` frame만 허용한다. 다른 frame을 지원하려면 TF2로 메시지 timestamp 시점의 transform을 조회하고, transform timeout/extrapolation은 latched FAULT로 처리한다.

### 6.4.3 워치독과 생명주기

워치독은 "마지막 메시지가 왔는가"뿐 아니라 current/target의 source timestamp, 수신 시각, 추론 완료, 명령 송신, Follower 상태 피드백을 각각 본다. 서로 다른 timeout 원인을 하나의 `stale`로 합치지 않는다. position뿐 아니라 quaternion의 finite와 norm도 검증한다.

ROS 2 노드는 구성되지 않은 상태, inactive, active, error를 구분하는 managed lifecycle을 사용할 수 있다. 그러나 lifecycle state 자체가 안전 기능의 인증을 뜻하지 않는다. 실제 토크 차단은 드라이버와 하드웨어 계약으로 검증해야 한다.

`code/safety_supervisor.py`는 하드웨어 비종속 상태 머신이고 `code/ros2_ws/.../workspace_guard.py`는 교육용 ROS 2 노드다.

> **잠깐 점검 — 안전 감독기는 언제 멈춰야 하는가**
>
> 1. 목표가 작업공간 경계를 크게 벗어났을 때 경계값으로 clamp하는 것보다 HOLD가 나은 이유는 무엇인가?
> 2. 정책이나 캘리브레이션 파일의 hash가 바뀌면 이전 승격 상태를 그대로 유지해도 되는가?
>
> **확인 포인트:** 큰 위반을 clamp하면 의도와 다른 경로를 만들 수 있으므로 명령을 거부하고 원인을 확인한다. 정책·캘리브레이션 변경은 검증했던 시스템 계약을 바꾸므로 낮은 단계에서 다시 승인해야 한다.

# 6.5 End-to-End Pick & Place 배포

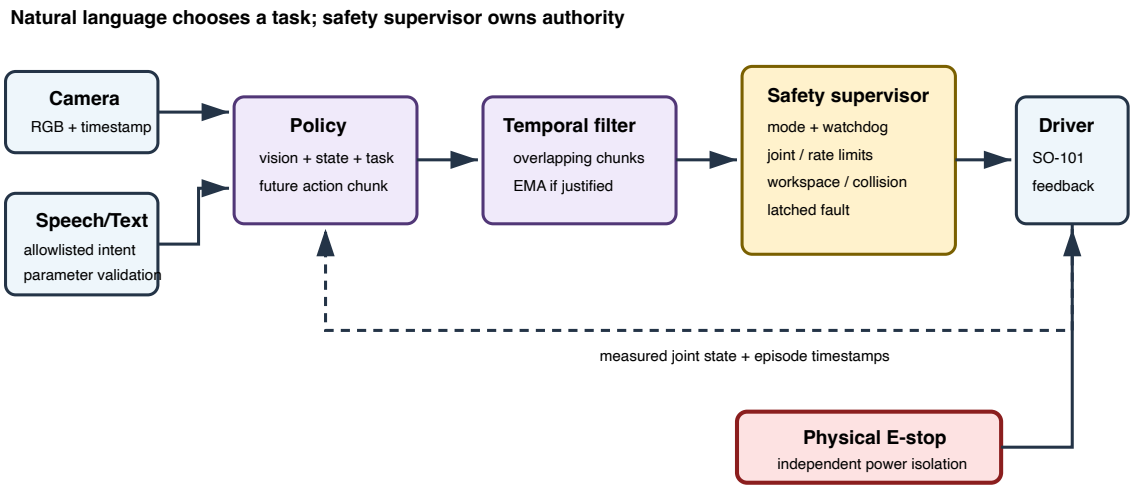



그림 6-3. 인지-정책-안전-실행-관측 배포 아키텍처

## 6.5.1 배포 전 고정해야 할 계약

- 정책 체크포인트 해시와 학습 데이터 버전
- base model/adapter/dataset exact revision과 train episode ID
- train-only normalization stats/hash (`meta/stats.json` 전역 통계와 구분)
- 카메라 serial, 해상도, FPS, exposure, mount
- 관절 이름, 순서, 단위, 캘리브레이션 ID
- camera `rename_map`, action representation, inverse-normalization 계약
- 언어 prompt grammar
- action chunk 크기, 실행 주기, temporal ensemble 파라미터
- workspace/joint/rate limit
- reset pose와 에피소드 성공/실패 정의

LeRobot의 실물 로봇 튜토리얼은 시연 기록, 데이터 시각화, 정책 학습, 평가의 전체 흐름을 제공한다. 최신 CLI는 변경될 수 있으므로 공식 문서를 기준으로 명령을 기록한다.[Hugging Face LeRobot]

최신 배포에서는 local RTC와 remote inference를 구분한다. local RTC는 rollout process의 background thread에서 동작하고, remote backend는 별 process/host의 policy server와 통신한다. `execution_horizon`, `max_observation_age_s`, action/startup timeout, starvation grace를 실측 지연과 chunk horizon에 맞춰 설정한다. buffer exhaustion이나 freshness 위반은 무기한 마지막 명령 유지가 아니라 HOLD/FAULT로 연결한다. task 변경 시 이전 instruction의 queue와 in-flight 결과를 폐기한다. [Hugging Face Policy Deployment/RTC]

## 6.5.2 권장 실행 순서

1. 녹화 영상과 관절 로그를 재생하는 offline inference
2. live camera + 모터 미명령 shadow inference
3. 시작 pose에서 단일 action chunk를 계산하고 사람이 검토

4. 저속, 빈 그리퍼, 넓은 장애물 여유
5. 제한 workspace에서 단일 색/단일 위치
6. 위치 변화, 조명 변화, distractor를 한 축씩 추가
7. 승인된 분포에서 정상 속도 평가

모든 단계에서 원시 관측, 정책 출력, 필터 출력, 최종 명령, 관절 피드백, 상태 전이, 사람 개입을 같은 episode ID로 기록한다.

### 6.5.3 음성/텍스트 명령 경계

자연어는 과업 선택 입력이지 안전 권한이 아니다. 음성 인식 결과를 shell, ROS topic 이름, 파일 경로, arbitrary prompt template에 직접 삽입하지 않는다.

안전한 경계는 다음과 같다.

```
speech/text → normalization → allowlisted intent → parameter validation
            → human confirmation(if required) → fixed task template → policy
```

예를 들어 허용 intent를 `PICK_RED_TO_BLUE`, `PICK_BLUE_TO_RED`, `STOP`으로 제한한다. “더 빨리”, “안전 제한 해제”, “테이블 밖 물체 집기”는 정책 prompt가 아니라 거부 대상이다. ASR confidence가 낮거나 색/대상이 모호하면 움직이지 않고 재질문한다. `STOP`은 자연어 모델을 거치지 않는 별도 저지연 경로로 처리하되 물리 E-stop을 대체하지 않는다.

# 6.6 평가 설계

## 6.6.1 성공 정의

Pick & Place의 성공을 “그럴듯한 영상”으로 평가하지 않는다. 에피소드 시작 전에 판정 규칙을 고정한다.

- 목표 물체가 지정 tray 내부에 완전히 놓임
- 제한 시간 내 완료
- 비목표 물체 이동 임계값 이하
- workspace/joint limit 위반 없음
- 사람 개입, E-stop, 물체 낙하는 실패

부분 성공은 별도 지표로 기록하지만 주 성공률의 정의를 사후 변경하지 않는다.

## 6.6.2 성공률과 Wilson 신뢰구간

$n$회 중 $s$회 성공이면 $\widehat{p} = s/n$이다. 작은 표본에서 단순 Wald 구간 $\widehat{p} \pm 1.96\sqrt{\widehat{p}(1-\widehat{p})/n}$은 0%나 100% 근처에서 나쁘다. 95% Wilson 구간은

$$\frac{\widehat{p} + z^2/(2n) \pm z\sqrt{\widehat{p}(1-\widehat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \quad z = 1.96$$

를 사용한다. NIST는 이항 비율 구간에서 Wilson 방법을 소개하고 관련 성능 문헌을 연결한다.[NIST]

예를 들어 10회 중 9회 성공은 점추정 90%지만 불확실성이 크다. 조건당 최소 20회 이상을 시작점으로 삼되 필요한 표본 수는 허용 오차와 위험에 따라 사전에 정한다. 성공률의 episode 표본과 latency의 control tick/frame 표본을 같은 n으로 오해하지 않는다. LeRobot의 정책 기여 안내도 실물 전용 정책 결과에 재현 명령과 하드웨어, 10회 이상의 성공률 보고를 요구한다.[Hugging Face]

## 6.6.3 층화 실험

| 요인 | 수준 예시 |
|---|---|
| 물체 위치 | 중앙, 좌, 우, near reach limit |
| 조명 | 기준, 어두움, 측면 반사 |
| 물체 색/재질 | 학습 내, 학습 경계, 미관측 |
| 카메라 | 기준, ±5 mm, ±2° perturbation |
| 지연 | 기준, +20 ms, +50 ms |

한 번에 여러 조건을 바꾸면 원인을 분리하기 어렵다. 순서를 무작위화하고 reset 절차를 동일하게 유지한다. 실패 영상만 골라 보여주는 survivorship bias와, 성공할 때까지 반복한 뒤 마지막 시도만 세는 선택 편향을 피한다.

**Stratify conditions; keep every episode**

| Position \ Light | Baseline | Dim | Side glare |
|---|---|---|---|
| **Center** | 18/20 = 0.90<br>Wilson CI + failure tags | 17/20 = 0.85<br>grasp(2), timeout(1) | 13/20 = 0.65<br>vision drift dominates |
| **Left** | 17/20 = 0.85<br>all episodes retained | 15/20 = 0.75<br>pose residual elevated | 9/20 = 0.45<br>recalibration gate |
| **Reach edge** | 14/20 = 0.70<br>joint limit failures | 10/20 = 0.50<br>vision + reach coupled | 6/20 = 0.30<br>outside approved envelope |

Counts are illustrative. Fix trial counts and success rules before running the experiment.

그림 6-4. 조건을 층화한 평가와 실패 원인 기록

## 6.7 트러블슈팅 매트릭스

| 현상 | 가설 | 확인할 로그/시험 | 조치 |
|---|---|---|---|
| 관절 떨림 | chunk 경계 불연속, 높은 gain | raw/ensemble/final action 비교 | temporal ensemble, rate limit, gain 조정 |
| 목표를 지나침 | latency, EMA lag, backlash 보상 과다 | p95 e2e, 방향 반전 응답 | 속도 하향, latency 보정, 보상 축소 |
| 파지 후 미끄러짐 | 마찰/토크/payload | 물체별 유지 시간, 패드 상태 | 패드·접근 개선, payload 축소 |
| 카메라 중앙만 성공 | extrinsic/intrinsic 잔차 | 위치별 reprojection residual | 작업공간 전체 재캘리브레이션 |
| 특정 조명에서 실패 | exposure/반사 OOD | histogram, confidence, raw frame | 노출 고정, 조건 데이터/랜덤화 |
| 첫 동작이 큼 | reset pose/첫 chunk 불일치 | 첫 관측과 명령 차이 | 정렬 slew, 첫 프레임 승인 |
| 가끔 오래 멈춤 | 추론 p99, GPU contention | stage timestamps | deadline, 모델 경량화, newest-only queue |
| 물체 대신 배경 파지 | shortcut learning | occlusion/배경 교체 실험 | 데이터 균형, distractor, 시각화 |
| 음성 오인식 후 오동작 | open-ended prompt | ASR/intent 로그 | allowlist, 확인, 모호성 거부 |
| E-stop 후 즉시 재동작 | 자동 복구 | 상태 전이 로그 | latched FAULT, 수동 reset/재정렬 |

# 6.8 포트폴리오 패키징

좋은 포트폴리오는 최고 성공 영상 하나가 아니라 재현 가능한 엔지니어링 주장이다.

## 필수 구조

```
project/
├── README.md                 # 문제, 결과, 안전 범위, 빠른 시작
├── docs/architecture.svg
├── configs/                  # limit, camera, policy version
├── src/                      # inference, supervisor, bridge
├── launch/
├── experiments/
│   ├── protocol.md
│   ├── results.csv
│   └── failure_taxonomy.md
├── media/                    # 성공뿐 아니라 대표 실패 영상
├── model_card.md
├── dataset_card.md
└── LICENSES/
```

README에는 Ubuntu/ROS 2/GPU/카메라/로봇 버전, 설치 명령, 실행 명령, 기대 출력, 알려진 한계, 안전 경고를 둔다. 비밀키, Hub token, 사람 얼굴/음성, 사유 공간 영상은 제거한다. 외부 모델, CAD, 이미지, 폰트의 라이선스를 기록한다.

## 보고할 결과

- 전체와 조건별 성공률 + Wilson 95% CI
- latency p50/p95/p99와 deadline miss
- failure taxonomy와 대표 실패 사례
- 필터/안전 감독기 전후 비교
- sim-only, shadow, low-speed, nominal 단계별 증거
- 재현 가능한 checkpoint/dataset/config hash

## 6.9 단계별 실습

### 실습 1: 로그 진단과 신뢰구간

실습 6-1: Sim2Real 로그 진단을 따른다.

```
python code/sim2real_diagnostics.py --demo
```

**예상 출력 예시**

```
episodes=20 success=15 rate=0.750 wilson95=[0.531, 0.888]
infer_ms  n=20 p50=31.1 p95=41.5 p99=n/a(n<100) max=42.3
e2e_ms    n=20 p50=82.4 p95=105.3 p99=n/a(n<100) max=107.0
condition=baseline     n=10 success= 8 rate=0.800 ci95=[0.490, 0.943]
condition=side_light   n=10 success= 7 rate=0.700 ci95=[0.397, 0.892]
```

### 실습 2: 필터와 안전 승격

실습 6-2: Temporal Filter와 안전 승격 감독기를 따른다.

```
python code/temporal_filters.py
python code/safety_supervisor.py
```

**예상 출력 예시**

```
mode=DRY_RUN accepted=True send_to_hardware=False
promotion DRY_RUN -> LOW_SPEED: approved
workspace violation: latched FAULT
reset denied: human approval and pose alignment required
```

### 실습 3: ROS 2 Workspace Guard

실습 6-3: ROS 2 Cartesian Workspace Guard를 따른다.

```
cd code/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 run sim2real_safety workspace_guard
```

## 6.10 요약

- Reality Gap은 시각, 기하, 동역학, 구동, 시간, 과업 분포의 측정 가능한 오차다.
- 평균 지연보다 충분한 표본의 p95/p99, n/max, clock domain, deadline miss가 안전 배포에 중요하다.
- 도메인 랜덤화는 실제 범위를 포함하는 분포 설계이며 무제한 변동이 아니다.
- EMA와 temporal ensemble은 출력 불연속을 줄이지만 지연과 안전 한계 문제를 해결하지 않는다.
- 배포는 offline/dry-run/저속/제한 작업공간/정상 운전의 승격 게이트를 거친다.
- 자연어는 과업 입력이며 권한 상승이나 안전 제한 해제 수단이 아니다.
- 성공률은 사전 정의한 판정과 조건별 시행 수, Wilson 신뢰구간과 함께 보고한다.
- 포트폴리오는 성공 영상뿐 아니라 실패, 버전, 안전 범위, 재현 명령을 포함한다.

## 6.11 확인문제

1. Reality Gap을 하나의 수치로만 관리하면 어떤 원인 분리가 어려운가?
2. p50 latency만 보고 배포 여부를 결정하면 안 되는 이유는 무엇인가?
3. 백래시 보상을 크게 설정했을 때 채터링이 생기는 이유를 설명하라.
4. EMA의 $\alpha$를 작게 하면 노이즈와 지연은 각각 어떻게 변하는가?
5. temporal ensemble 뒤에도 rate/workspace limit이 필요한 이유는 무엇인가?
6. Cartesian target의 큰 workspace 위반을 clamp보다 거부해야 하는 이유를 제시하라.
7. E-stop 해제 후 자동으로 NOMINAL 상태로 돌아가면 안 되는 이유는 무엇인가?
8. 음성 명령에서 allowlist intent가 필요한 이유는 무엇인가?
9. 10회 중 9회 성공이라는 결과가 “성공률 90%가 확실하다”를 뜻하지 않는 이유는 무엇인가?
10. 성공 사례만 담은 포트폴리오가 시스템 역량을 과대평가하는 이유를 설명하라.

해설은 제6장 확인문제 해설에 있다.

# 6.12 최종 프로젝트

## 안전한 SO-101 Pick & Place 실증

다음 산출물을 만든다.

1. 고정된 task contract와 성공/실패 판정
2. offline 20 episode replay와 latency 분석
3. dry-run/shadow inference 보고서
4. 저속·제한 workspace 실물 시험
5. 최소 3개 조건, 조건당 20 episode 평가
6. 성공률, Wilson 95% CI, failure taxonomy
7. E-stop/timeout/workspace violation 시험 증거
8. 재현 가능한 저장소와 3분 기술 설명 영상

평가 기준은 최종 프로젝트 평가 기준에 있다.

# 6.13 실습·평가·참고자료

이 절 아래에 장별 참고문헌, 단계별 실습, 확인문제 해설과 최종 프로젝트 평가 기준을 한데 모았다.

## 6.13.1 참고문헌


1. Hugging Face, Getting Started with Real-World Robots, 2026년 접근.
2. Hugging Face, Adding a Policy, 실물 평가 결과 보고 지침, 2026년 접근.
3. NVIDIA, Isaac Sim Randomization Snippets, 2026년 접근.
4. NVIDIA, Replicator Domain Randomization Migration, 2026년 접근.
5. J. Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," IROS 2017, arXiv:1703.06907.
6. T. Z. Zhao et al., "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," RSS 2023, arXiv:2304.13705.
7. R. Y. Tsai and R. K. Lenz, "A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration," *IEEE Transactions on Robotics and Automation*, 5(3), 1989, DOI:10.1109/70.34770.
8. NIST/SEMATECH, Confidence Intervals for a Binomial Proportion, 2026년 접근.
9. L. D. Brown, T. T. Cai, A. DasGupta, "Interval Estimation for a Binomial Proportion," *Statistical Science*, 16(2), 2001, DOI:10.1214/ss/1009213286.
10. ISO, ISO 10218-1:2025 Robotics — Safety requirements — Part 1, 2025.
11. ISO, ISO 10218-2:2025 Robotics — Safety requirements — Part 2, 2025.
12. ROS 2, Managed Nodes, lifecycle 설계 문서.
13. ROS 2 Jazzy, Quality of Service Settings, deadline/liveliness/QoS.
14. ROS 2 Jazzy, Using stamped datatypes with tf2_ros::MessageFilter, timestamp와 frame 기반 변환.
15. Hugging Face, Policy Deployment, local RTC freshness/timeout 계약, 2026년 접근.
16. Hugging Face, Real-Time Chunking, 2026년 접근.
17. Hugging Face, Remote Inference, 2026년 접근.


## 실습 6-1: Sim2Real 로그 진단

**목표**

- stage timestamp로 추론 및 end-to-end latency를 계산한다.
- 평균 대신 p50/p95/p99를 비교한다.
- 전체 성공률과 조건별 Wilson 95% 신뢰구간을 계산한다.

**절차**

1. 데모 데이터를 분석한다.

```
cd book/06_sim2real_deployment
python code/sim2real_diagnostics.py --demo
```

2. 다음 header를 가진 `episodes.csv`를 만든다.

```
episode_id,condition,success,capture_ns,receive_ns,infer_start_ns,infer_end_ns,send_ns,motio
        n_ns
```

3. 자신의 로그를 분석한다.

```
python code/sim2real_diagnostics.py --csv episodes.csv
```

4. 한 행의 `infer_end_ns`를 `infer_start_ns`보다 작게 만들어 timestamp inversion이 거부되는지 확인한다.
5. 성공 9/10과 90/100의 점추정은 같지만 신뢰구간 폭이 어떻게 다른지 `wilson_interval`로 계산한다.
6. 조건별 성공률이 전체 성공률에 가려지는 사례를 만든다.

#### 분석 질문

- p99가 p50보다 3배 클 때 제어 주기를 어떻게 정할 것인가?
- 표본이 20개뿐일 때 p99를 운영 tail latency로 해석하면 안 되는 이유는 무엇인가? n, max, deadline miss를 함께 보고하라.
- `motion_ns`를 어떤 센서 사건으로 정의했는가?
- 서로 다른 컴퓨터의 timestamp를 직접 빼도 되는가?
- 실패 episode를 삭제하면 성공률과 latency가 각각 어떤 편향을 갖는가?

#### 제출물

- 전체/조건별 성공률과 Wilson 구간 표
- latency p50/p95/p99 표
- clock domain과 누락 episode 처리 정책
- success literal 오탈자, duplicate episode ID, 빈 condition이 거부되는 시험
- 가장 가능성 높은 Reality Gap 가설 3개와 다음 실험

## 실습 6-2: Temporal Filter와 안전 승격 감독기

#### 목표

- EMA와 temporal ensemble을 구분한다.
- 승격 게이트와 latched FAULT를 시험한다.
- workspace 위반을 clamp하지 않고 거부한다.

#### 1부: 필터

```
python code/temporal_filters.py
```

1. EMA `alpha`를 0.1, 0.3, 0.8로 바꾸고 step input 응답을 기록한다.
2. chunk의 두 번째 예측에 큰 이상치를 넣고 ensemble 출력 변화를 본다.
3. ensemble 출력을 joint/rate limit에 다시 통과시켜야 하는 이유를 설명한다.
4. stale/NaN/다른 phase proposal을 ensemble 전에 제거하고 E-stop/reset에서 history를 비운다.

#### 2부: 감독기

```
python code/safety_supervisor.py
```

1. `OFFLINE → DRY_RUN`을 승인 없이 시도한다.
2. 단계를 건너뛰어 `DRY_RUN → BOUNDED`로 승격을 시도한다.
3. NaN target, timeout, workspace 위반을 각각 주입한다.
4. `human_approval=True`, `pose_aligned=False`로 fault reset을 시도한다.
5. 두 조건을 모두 만족시킨 뒤 reset이 항상 `DRY_RUN`으로 돌아가는지 확인한다.

#### 상태 전이 설계 과제

다음 사건을 상태 전이표에 추가한다.

- 카메라 serial 변경
- 정책 checkpoint hash 변경
- 세 번 연속 deadline miss
- E-stop 입력
- 그리퍼 과전류

각 사건에서 자동 복구 허용 여부와 사람이 확인할 항목을 적는다.

## 실습 6-3: ROS 2 Cartesian Workspace Guard

#### 목표

- 정책 목표와 로봇 현재 pose를 별도 토픽으로 관리한다.
- workspace, 최대 step, current-state watchdog을 시험한다.
- guard 출력이 모터 명령이 아니라 검증된 중간 표현임을 이해한다.

#### 빌드

```
cd book/06_sim2real_deployment/code/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 run sim2real_safety workspace_guard
```

#### 정상 입력

터미널 B에서 live timestamp를 가진 현재 pose를 20 Hz로 발행한다. `ros2 topic pub`의 고정 YAML timestamp는 곧 stale해지므로 실습용 publisher를 사용한다.

```
ros2 run sim2real_safety stamped_pose_publisher --ros-args \
  -p topic:=/robot/ee_pose -p rate_hz:=20.0 \
  -p frame_id:=base_link -p xyz:="[0.20, 0.00, 0.12]"
```

터미널 C에서 2 cm 이내 목표를 2 Hz로 발행한다.

```
ros2 run sim2real_safety stamped_pose_publisher --ros-args \
  -p topic:=/policy/ee_target -p rate_hz:=2.0 \
  -p frame_id:=base_link -p xyz:="[0.21, 0.00, 0.12]"
```

터미널 D에서 상태를 본다.

```
ros2 topic echo /safety/state --qos-durability transient_local
```

#### 실패 주입

1. x=0.9인 목표를 보내 workspace fault를 발생시킨다.
2. 노드를 재시작하고 5 cm step을 보내 `target step too large`를 확인한다.
3. current pose 발행을 중단해 watchdog fault를 확인한다.
4. target publisher의 `frame_id:=camera_link`로 바꾸고 frame mismatch가 latched FAULT가 되는지 확인한다. 좌표 숫자를 `base_link`로 재표기해서는 안 된다.
5. `quaternion_xyzw:="[0.0, 0.0, 0.0, 0.0]"`와 NaN 성분을 각각 보내 quaternion/finite 검사를 확인한다.
6. `ros2 topic pub --once`로 stamp가 0인 메시지를 보내 `timestamp missing`을 확인한다.
7. fault 뒤 정상 목표가 자동으로 통과하지 않는지 확인한다.

#### 실물 연결 전 추가 항목

- orientation 유효성, quaternion normalization
- source timestamp freshness와 future timestamp tolerance
- frame mismatch 거부 또는 timestamped TF2 변환
- FK 기반 joint limit과 self-collision
- command/feedback sequence ID
- driver HOLD/torque-off acknowledgment
- 물리 E-stop 입력과 재승인 절차
- ROS 노드가 멈춰도 작동하는 독립 안전 회로

#### QoS 계약

- current pose는 sensor-data QoS로 최신값을 우선한다.
- policy target과 accepted target은 reliable depth 1을 사용한다.
- fault state는 transient-local reliable QoS로 late subscriber도 마지막 latched fault를 읽게 한다.
- 실제 driver 연결에서는 accepted target 수신만으로 실행 성공을 가정하지 말고 HOLD/torque-off acknowledgement를 별도 확인한다.

## 최종 프로젝트 평가 기준

총점 100점이다.

| 항목 | 배점 | 우수 기준 |
|---|---|---|
| 과업 계약 | 10 | 성공/실패, timeout, reset, 조건을 사전 정의 |

| 항목 | 배점 | 우수 기준 |
|---|---|---|
| 안전 승격 | 20 | dry-run→저속→bounded→nominal 증거와 강등 조건 |
| 독립 안전 | 10 | E-stop/전원 차단, latched fault, 수동 재승인 |
| Reality Gap 진단 | 15 | 지연·기하·마찰·백래시 중 3개 이상 정량 분석 |
| 정책 배포 | 10 | 버전/해시, action chunk, filter, 주기 계약 |
| 실험 설계 | 15 | 조건별 반복, 순서 무작위화, 누락 없는 episode |
| 통계 보고 | 10 | 성공률 + Wilson CI + latency p50/p95/p99 |
| 재현성과 포트폴리오 | 10 | 코드·설정·실패 사례·라이선스·한계 공개 |

다음 중 하나라도 있으면 안전 승격 항목은 0점이다.

- workspace/joint limit 없이 실물 실행
- `PoseStamped.frame_id`를 무시하거나 좌표 변환 없이 `base_link`로 재표기
- stale target 또는 유효하지 않은 quaternion을 실행 경로에서 허용
- E-stop 뒤 자동 NOMINAL 복귀
- timeout 뒤 마지막 명령 무기한 유지
- 실패 episode 삭제 또는 성공 기준 사후 변경
- 자연어 명령으로 안전 제한 해제 가능

## 제6장 확인문제 해설

1. 시각, 기하, 동역학, 지연, 과업 분포 오차는 원인과 개선 방법이 다르다. 하나의 수치는 어느 축이 개선/악화됐는지 숨긴다.
2. p50은 절반의 프레임만 설명한다. 드문 p95/p99 지연이 deadline miss와 충돌을 만들 수 있다.
3. 방향 반전 근처의 작은 노이즈에 보상이 반복 적용되면 출력 방향이 계속 바뀌어 채터링이 생긴다.
4. 작은 $\alpha$는 평활을 강하게 해 노이즈를 줄이지만 응답 지연을 늘린다.
5. 평균할 proposal 자체가 모두 위험할 수 있고 평균 결과도 관절/작업공간/속도 한계를 넘을 수 있다.
6. 큰 위반을 경계로 투영하면 정책이 의도하지 않은 방향과 접촉 경로가 생긴다. 이는 모델/프레임 오류 신호일 수 있어 HOLD가 적절하다.
7. 위험 원인, 현재 자세, 작업공간이 아직 안전하다는 보장이 없다. fault 원인 확인과 재정렬, 사람 승인이 필요하다.
8. ASR/자연어 모델은 모호하고 open-ended다. allowlist는 실행 가능한 과업과 파라미터 경계를 고정한다.
9. 표본이 작아 모비율의 불확실성이 크다. 성공률과 함께 이항 신뢰구간을 보고해야 한다.
10. 실패 분포, 운영 조건, 재시도 횟수를 숨기면 일반화와 신뢰성을 판단할 수 없다.

# 부록 A. ROS 2 Lyrical 마이그레이션 노트

## A.1 왜 본문은 Jazzy를 사용하는가

ROS 2는 짝수 해에 Ubuntu LTS와 맞춘 장기지원(Long Term Support, LTS) 배포판을 제공한다. ROS 2 Jazzy Jalisco는 Ubuntu 24.04를 Tier 1 플랫폼으로 지원하며 2029년 5월까지 지원된다. ROS 2 Lyrical Luth는 2026년 5월에 공개된 후속 LTS이며 2031년 5월까지 지원된다.

이 책은 다음 이유로 Jazzy를 주 환경으로 삼는다.

- Ubuntu 24.04 기반 개발·배포 환경이 이미 널리 축적되어 있다.
- 교육용 로봇 드라이버, 시뮬레이터 확장, 서드파티 패키지의 바이너리와 문서가 충분하다.
- 새 LTS 공개 직후 발생하는 패키지 이식 지연을 줄일 수 있다.
- Jazzy의 지원 기간이 교육 및 단기 제품 개발 주기에 충분하다.

새 프로젝트가 Ubuntu 26.04와 Lyrical의 특정 기능을 필요로 한다면, 아래 점검표로 이식 범위를 판단한다.

## A.2 이식 점검표

1. 필요한 ROS 패키지가 Lyrical 바이너리로 배포되는지 `index.ros.org`와 `apt`에서 확인한다.
2. 로봇 하드웨어 드라이버가 사용 중인 커널, USB/시리얼 계층과 호환되는지 확인한다.
3. Isaac Sim의 공식 지원 운영체제와 ROS 브리지 조합을 확인한다.
4. `rclcpp`, `rclpy`, `rosbag2`, `launch`, `message_filters`의 변경 사항을 릴리스 노트에서 검토한다.
5. 워크스페이스를 새 환경에서 깨끗하게 빌드하고, 컴파일 경고를 오류처럼 다룬다.
6. QoS, 타임스탬프, rosbag 기록/재생, TF 트리를 통합 시험한다.
7. 실물 로봇에서는 저속·저토크 모드로 회귀 시험한다.

## A.3 Lyrical에서 주목할 기능

Lyrical에는 비동기 Python 노드 지원 개선, 이벤트 기반 실행기, `rosbag2` 제어 및 관측 기능 확장, 런타임 로깅 백엔드 선택 등 새 기능이 포함된다. 그러나 새 기능을 사용하기 위해 시스템 전체를 즉시 이식할 필요는 없다. 제품의 요구사항, 의존 패키지 준비 상태, 유지보수 기간을 함께 비교한다.

## 참고문헌


- ROS 2, REP 2000: ROS 2 Releases and Target Platforms.
- ROS 2 Documentation, Jazzy Jalisco Release Notes.
- ROS 2 Documentation, Lyrical Luth Release Notes.

# 부록 B. 용어집

| 용어 | 설명 |
|---|---|
| Action Chunk | 한 번의 추론으로 예측하는 연속된 미래 행동 묶음 |
| Articulation | 관절로 연결된 강체 집합을 물리 엔진에서 표현하는 구조 |
| CCD | Continuous Collision Detection. 빠르게 움직이는 물체의 프레임 간 관통을 줄이는 충돌 검출 방식 |
| DDS | Data Distribution Service. ROS 2 통신의 기반이 되는 데이터 중심 미들웨어 표준 |
| Digital Twin | 실물 시스템의 상태와 동작을 대응시키는 가상 모델 |
| DLS | Damped Least Squares. 특이점 부근의 역기구학 수치 불안정을 줄이는 감쇠 최소제곱법 |
| Domain Randomization | 시뮬레이션의 외형·물리 조건을 무작위화해 현실 변화에 대한 정책 강건성을 높이는 기법 |
| End Effector | 로봇팔 끝에서 물체 또는 환경과 상호작용하는 도구나 그 기준 프레임 |
| Extrinsic | 카메라와 기준 좌표계 사이의 회전과 이동 관계 |
| FK | Forward Kinematics. 관절 상태에서 엔드이펙터 포즈를 계산하는 순기구학 |
| IK | Inverse Kinematics. 목표 엔드이펙터 포즈를 만족하는 관절 상태를 구하는 역기구학 |
| Intrinsic | 초점 거리, 주점 등 카메라 내부 투영 파라미터 |
| LoRA | Low-Rank Adaptation. 저랭크 행렬을 추가해 적은 학습 파라미터로 모델을 미세조정하는 방법 |
| OpenUSD | 장면의 계층, 속성, 관계와 합성을 표현하는 Universal Scene Description 생태계 |
| Policy | 관측과 목표를 받아 로봇 행동을 출력하는 의사결정 함수 또는 모델 |
| Prim | USD Stage 안의 기본 장면 객체 |
| QoS | Quality of Service. 신뢰성, 이력, 내구성 등 ROS 2 통신 정책 |
| Reality Gap | 시뮬레이션과 실제 환경의 관측·동역학 차이 |
| Sim2Real | 시뮬레이션에서 개발·학습한 시스템을 실제 장비로 이전하는 과정 |
| TF2 | ROS 2에서 시간에 따른 좌표 프레임 변환을 관리하는 라이브러리 |
| VLA | Vision-Language-Action. 시각 관측과 언어 지시에서 로봇 행동을 생성하는 모델 계열 |